Introduction, Haskell, and Type Classes

These notes serve as material for the *Advanced Programming* (AP) course at DIKU. They do not form a complete textbook and may not be comprehensible outside the context of the course. In particular, they assume that basic Haskell programming skills are acquired through other means, for example via textbooks such as Programming in Haskell. The notes serve to emphasize the course perspectives, as well as contain material that we could not find of sufficient quality and brevity elsewhere.

The text will contain many links to other resources online. Unless explicitly indicated, you can consider these to be supplementary, rather than required reading.

There is one chapter per week, containing material of relevance to that week's teaching activities and assignment.

Course Principles

Although the Course Description contains a rather dry list of learning goals, you may also benefit from keeping the following *principles* in mind when reading these notes. They reflect the philosophy behind the course, and our rationale for picking the material and constructing the assignments.

In AP, we study programming methodologies based on the following principles.

- **Precision**. Code should clearly specify what it does and does not do; largely accomplished through the use of advanced type systems and type-directed design. This is why we use Haskell as our language in the course.
- **Separation of concerns**. It is a fairly mainstream point of view that modular programs, separated into independent units with minimal functional overlap and interdependence, are easier to write and maintain.

By making use of programming techniques that provide precision, we can ensure and verify that our programs are structured thus. Monadic programming is one particularly clear example of this principle, and the one that is our principal object of study, which separates the *use* of effects from the *definitions* of effects.

- **Principled design**. By structuring programs along rigorously defined abstractions, such as monads or similar effect boundaries, we can develop principles for systems design that are both simple and effective, and elegantly support features such as resilience, that can often become quite messy.

We demonstrate these principles through code written in the Haskell programming language, but they are language-agnostic, and can be applied in any language (although often in more awkward forms, for languages that do not provide the requisite flexibility of expression).

While AP is a course focused on practical programming, the time constraints of a seven week course means that we cannot directly study the large programs where these techniques are the most valuable. Instead our examples, exercises, and assignments will be small "toy programs" that cut any unnecessary detail or functionality in order to focus on the essential principles. This does not mean that the techniques we teach in AP to not scale up to large programs; merely that we do not have time for you to observe it for yourselves. You will just have to use your imagination.

Why so many interpreters?

Many of the examples, exercises, and assignments in AP will be in the context of writing interpreters, type checkers, or parsers for programming languages. This is not *solely* because the teachers happen to enjoy this aspect of computer science, but rather because these domains contain the essence of problems that occur frequently in real-world programming.

- **Interpretation** is about making operational decisions and changing state based on program input.

- **Type checking** is input validation.
- **Parsing** is recovering structured information from unstructured input.

For didactical reasons, AP mostly focuses on problems that *exclusively* contains these problems (and little "business logic"), but the ideas we study are applicable even outside the rather narrow niche of implementing programming languages.

Testing

In the assignments you will be required to write tests. You must use the Haskell package Tasty to write our tests. Tasty is a meta-framework that allows different testing frameworks to be combined - in particular, `hunit` for unit tests and `QuickCheck` for property-based testing. Although you will initially only be writing unit tests, later in the course you will be taught property-based testing. In order to avoid having to switch testing framework halfway through the course, we use Tasty from the start.

Generally the code handout will already contain dependencies on the relevant packages. The ones we will use in the following are `tasty` and `tasty-hunit`, which you can add to the `build-depends` field of your `.cabal` file.

The Tasty documentation is fairly good and you are encouraged to read it. However, this section will list everything you need to know for the first week of AP.

Structuring tests

There are many ways of structuring test suites. Because the programs you will write in AP are so small, we will use a particularly simple scheme. For any Haskell module `Foo`, the tests will be in a corresponding *test module* called `Foo_Tests`. Each test module defines a test suite named `test`, which in Tasty is of type `TestTree`. We will see below how to define these.

To run the tests for an entire project, we write a *test runner*, which we will normally call `runtests.hs`. This test runner will import the various `TestTree`s, combine them if necessary, and pass them to the `defaultMain` function provided by Tasty. When the program is run, Tasty will run the test suite and report any errors. If the test runner is registered as a test suite in the `.cabal` file, we can use the `cabal test` command to run the tests. This will be the case for all code handouts that come with a `.cabal` file.

A test runner can look like this, where we assume the tests are defined in a module `Foo_Tests`:

```
import qualified Foo_Tests
import Test.Tasty (defaultMain)

main :: IO ()
main = defaultMain Foo_Tests.tests
```

If we load this module into `ghci`, we can also simply execute `main` to run the test suite. This makes interactive development easy.

Writing tests

To write a unit test, we import the module `Test.Tasty.HUnit`. This gives us access to a variety of functions that produce `TestTree`s. For example, `testCase`:

```
testCase :: TestName -> Assertion -> TestTree
```

The `TestName` type is a synonym for `String` and is used to label failures. The `Assertion` type is a synonym for `IO ()`, but in practice it is constructed using a variety of constructor functions. One of the simplest is `assertBool`:

```
assertBool :: String -> Bool -> Assertion
```

We give it a `String` that is shown when the test fails, and then a `Bool` that is `True` when the test succeeds. This is how we can write a test that fails:

```
failingTest :: TestTree
failingTest = testCase "should not work" $ assertBool "1 is not 2" $
1==2
```

And here is one that succeeds:

```
successfulTest :: TestTree
successfulTest = testCase "should work" $ assertBool "1 is 1" $ 1==1
```

We can combine multiple `TestTree`s with `testGroup`:

```
tests :: TestTree
tests =
  testGroup
    "unit test suite"
    [ successfulTest,
      failingTest
    ]
```

The first argument is a descriptive string, and the second is a list of `TestTree`s. We can use this to define test suites with arbitrarily complicated nesting (and support for running only parts of the entire test suite), but this is not needed for the comparatively simple test suites we write in AP.

The `Test.Tasty.HUnit` module also provides other handy operators for certain common cases. For example, `@?=` can be used for testing equality in a more concise way than using `assertBool`:

```
failingTest2 :: TestTree
failingTest2 = testCase "should not work 2" $ 1 @?= 2
```

When run, this will produce output such as the following:

```
unit test suite
  should work:       OK
  should not work:   FAIL
    .../Week1/Tests.hs:10:
    1 is not 2
    Use -p '/should not work/' to rerun this test only.
  should not work 2: FAIL
    .../Week1/Tests.hs:13:
    expected: 2
     but got: 1
    Use -p '$0=="unit test suite.should not work 2"' to rerun this
test only.
```

Timeouts

Tasty does not know how long a test is supposed to run for, but sometimes we do. We can ask Tasty to fail tests after a specified period via the `mkTimeout` and `localOption` functions, which are imported from `Test.Tasty`. For example, if we want to apply a one second timeout to all tests contained in a given `testTree`, we could write:

```
tests :: TestTree
tests =
  localOption (mkTimeout 1000000) $ testGroup
    "unit test suite"
    [ successfulTest,
      failingTest
    ]
```

The timeout applies to each individual test in the tree passed to `localOption`, not to the entire test suite. Such timeouts will generally be present in the test suite scaffolding of code handouts, as infinite loops are a rather common mistake.

Useful types

This section discusses various useful Haskell types that are available in the base library, as well as common functions on those types. We will make use of some of these types in the later chapters.

`Maybe`

The `Maybe` type is available in the Prelude (the module that is implicitly imported in every Haskell program), but has the following definition:

```
data Maybe a = Nothing
             | Just a
```

A value of type `Maybe a` is either `Nothing`, representing the absence of a value, or `Just x` for some value `x` of type `a`. It is often called the *option type*, and serves roughly the same role as the `null` value in poorly designed languages, but in contrast to `null` is visible in the type system.

It is often used to represent an operation that can fail. For example, we can imagine a function of type

```
integerFromString :: String -> Maybe Int
```

that tries to turn a `String` into an `Integer`, and either returns `Nothing`, indicating that the `String` is malformed, or `Just x`, where `x` is the corresponding integer. We will return to this in Week 3.

One useful function for operating on `Maybe` values is the `maybe` function:

```
maybe :: b -> (a -> b) -> Maybe a -> b
```

It accepts a value that is returned in the `Nothing` case, and otherwise a function that is applied to the value in the `Just` case. It is equivalent to pattern matching, but is more concise.

Another function, which we must import from the Data.Maybe module, is `fromMaybe`:

```
fromMaybe :: a -> Maybe a -> a
```

It turns a `Maybe a` into an `a` by providing a default value:

```
> fromMaybe 0 Nothing
0
> fromMaybe 0 (Just 1)
1
```

Again this is nothing we could not write ourselves using `case`, but using these functions can result in neater code.

Either

The `Either` type is available in the Prelude and has this definition:

```
data Either a b = Left a
                | Right b
```

It is used to represent two different possibilities, with different types. In practice, it is often used to represent functions that can fail, with the `Left` constructor used to represent information about the failure, and the `Right` constructor used to represent success.

A useful function for manipulating `Either` values is `either`:

```
either :: (a -> c) -> (b -> c) -> Either a b -> c
```

Void

The `Void` type must be imported from `Data.Void` and has the following definition:

```
data Void
```

This odd-looking type has *no constructors*, meaning there are no values of type `Void`. This is admittedly somewhat exotic, but it has some uses. For example, if we have a function of type `Int -> Void`, we know that this function cannot possibly return, as no value of type `Void` can be constructed. This is not really useful for a pure function, but if we have an impure function with side-effects, such as the infinite loops that are used in servers for reading incoming requests (later in the course), then it may be sensible to use a `Void` return type to clarify that the function will never terminate.

Another use of `Void` is to eliminate cases in polymorphic types. For example, if we have a type `Either Void a`, then we know that the `Left` case can never occur. This means we do not need to handle it when pattern matching the `Either` type.

> ⚠ **Warning**
>
> This is strictly not true. Haskell is a lazy language, so every value is inhabited by the special value ⊥ ("bottom"), which represents a diverging computation. Example:
>
> ```
> > Left undefined :: Either Void Integer
> Left *** Exception: Prelude.undefined
> ```
>
> We will return to laziness later in the course, but it is standard to reason about the type-level guarantees of Haskell code as if it were eager.

Type classes

Type classes are Haskell's main way of specifying abstract interfaces that can be implemented by concrete types. For example, the predefined `Eq` type class is the interface for any type `a` that support equality:

```haskell
class Eq a where
  (==) :: a -> a -> Bool
  (/=) :: a -> a -> Bool
```

This interface defines two *methods*, `(==)` and `(/=)`, of the specified type, which all type classes must implement. The enclosing parentheses denote that these are actually the infix operators `==` and `/=`.

We can write polymorphic functions that require that the polymorphic types implement a type class. This is done by adding a type class constraint to the type of the function. For example:

```haskell
contains :: (Eq a) => a -> [a] -> Bool
contains _ [] = False
contains x (y : ys) = x == y || contains x ys
```

In the definition of `contains`, we are able to use the `==` method on values of type `a`. If we removed the `(Eq a)` constraint from the type signature, we would get a type error.

Implementing an instance

When implementing an *instance* for a type class, we must implement all the methods described in the interface.

> **Note**
>
> Despite the similarity in nomenclature (class, instance, method), type classes are completely unrelated to classes in object oriented programming, except that both concepts are related to specifying interfaces.

For example, if we define our own type for representing a collection of programming languages:

```
data PL
  = Haskell
  | FSharp
  | Futhark
  | SML
  | OCaml
```

Then we can define an `Eq` instance for `PL` as follows:

```
instance Eq PL where
  Haskell == Haskell = True
  FSharp == FSharp = True
  Futhark == Futhark = True
  SML == SML = True
  OCaml == OCaml = True
  _ == _ = False

  x /= y = not $ x == y
```

In fact, the `Eq` class has a *default method* for `/=` expressed in terms of `==`, so we can elide the definition of `/=` in our instance.

Haskell has a handful of built-in type classes. For us, the most important of these are `Eq` (equality), `Ord` (ordering), and `Show` (converting to text). The Haskell compiler is able to automatically *derive* instances for these when defining a datatype:

```
data PL
  = Haskell
  | FSharp
  | Futhark
  | SML
  | OCaml
  deriving (Eq, Ord, Show)
```

This is very convenient, as the definitions of such instances are usually very formulaic. You should add `deriving (Eq, Ord, Show)` to all datatypes you define.

Type class laws

Type classes are often associated with a set of *laws* that must hold for any instance of the type class. For example, instances of `Eq` must follow the usual laws we would expect for an equality relation:

- **Reflexivity**: `x == x = True`
- **Symmetry**: `x == y = y == x`
- **Transitivity**: if `x == y && y == z = True`, then `x == z = True`
- **Extensionality**: if `x == y = True` and `f` is a function whose return type is an instance of `Eq`, then `f x == f y = True`
- **Negation**: `x /= y = not (x == y)`.

Unfortunately, these laws are not checked by the type system, so we must be careful verify that our instances behave correctly. An instance that follows the proscribed laws is called *lawful*. The instances that are automatically derived by the compiler will always be lawful (unless they depend on hand-written instances that are not lawful).

`Functor`

One of the important standard type classes is `Functor`, which abstracts the notion of a "container of values", where we can apply a function to transform the contained values. We should not carry this metaphor too far, however: some of the types that are instances of `Functor` are only "containers" in the loosest of senses. The somewhat exotic name `Functor` is inspired by a branch of mathematics called *category theory* (as are many other Haskell terms), but we do not need to understand category theory in order to understand the `Functor` type class:

```
class Functor f where
  fmap :: (a -> b) -> f a -> f b
```

The `fmap` method specified by `Functor` is essentially a generalisation of the `map` we are used to for lists. One interesting detail is that `Functor` instances are not defined for *types*, but for *type constructors*. See how the `fmap` method

turns an `f a` into an `f b`, intuitively changing the `a` values to `b` values. That means `f` by itself is not a type - it must be *applied* to a type, and hence is a type constructor.

This is perhaps a bit easier to understand if we first define our own type of linked lists.

```haskell
data List a
  = Nil
  | Cons a (List a)
  deriving (Eq, Ord, Show)
```

Our `List` type is equivalent to Haskell's built-in list type (which is already an instance of `Functor`), but without the syntactic sugar. We can define a `Functor` instance for `List` as follows:

```haskell
instance Functor List where
  fmap _ Nil = Nil
  fmap f (Cons x xs) = Cons (f x) (fmap f xs)
```

Functor laws

Any `Functor` instance must obey these laws:

- **Identity**: `fmap id == id`.
- **Composition**: `fmap (f . g) == fmap f . fmap g`.

Intuitively, they say that `fmap` is not allowed to do anything beyond applying the provided function. For example, we cannot store a count of how many times `fmap` has been applied, or otherwise tweak the observable structure of the container that is being `fmap`ed (e.g. by reversing the list or some such).

`Foldable`

Type classes allow us to write functions that are generic and reusable in varied contexts. An example of this is the standard class `Foldable`, which allows us to

iterate across all elements of a "container". Its true definition looks more complicated than it really is, due to a large number of optional methods. The following is an abbreviated (but still correct) description of `Foldable`:

```
class Foldable t where
  foldr :: (a -> b -> b) -> b -> t a -> b
```

That is, we must provide a method `foldr` for iterating across the elements of type `a`, while updating an accumulator of type `b`, which yields a final accumulator `b`. An instance of `Foldable` for out `List` type looks like this:

```
instance Foldable List where
  foldr _ acc Nil = acc
  foldr f acc (Cons x xs) = f x (foldr f acc xs)
```

Once a type is an instance of `Foldable`, we can define a remarkable number of interesting functions in termss of `foldr` (all of which are already defined for you in the Prelude).

For example, we can turn any `Foldable` into a list:

```
toList :: (Foldable f) => f a -> [a]
toList = foldr op []
  where
    op x acc = x : acc


> toList (Cons 1 (Cons 2 (Cons 3 Nil)))
[1,2,3]
```

Or we can see whether a given element is contained in the collection:

```
elem :: (Eq a, Foldable f) => a -> f a -> Bool
elem needle = foldr op False
  where
    op x acc =
      acc || x == needle
```

```
> elem 1 (Cons 1 (Cons 2 (Cons 3 Nil)))
True
> elem 4 (Cons 1 (Cons 2 (Cons 3 Nil)))
False
```

Monoids

Monoids are algebraic structures comprising a set with an associated binary operation with an identity element. If there is no identity element, we call this structure a *semigroup*. In Haskell, we can model these structures using type classes as follows.

```haskell
class Semigroup a where
  (<>) :: a -> a -> a

class Semigroup a => Monoid a where
  mempty :: a
```

We will generally deal only with monoids (which are also semigroups), although there are some cases where it is useful to have only semigroups. Whenever we talk about some type being a monoid, we are always concerned with its semigroup instance as well.

Monoids generalise the notion of "combining two things", and turn out to be a surprisingly general and useful tool for structuring programs, as it turns out many operations can be generalised as "extracting things from a structure" and then "combining those things".

A valid instance of `Monoid` and `Semigroup` must satisfy certain laws:

- **Identity**: `a <> mempty == mempty <> a == a`.
- **Associativity**: `(a <> b) <> c == a <> (b <> c)`

Monoids are an example of a monoid, with the following instance:

```haskell
instance Semigroup [a] where
  xs <> ys = xs ++ ys

instance Monoid [a] where
  mempty = []
```

Haskell already defines these classes for us, as well as law-abiding predefined instances for all builtin types. However, in some cases multiple instances are possible. For example, we could define a monoid instance `Int` where the operation is addition and the identity element is zero, but we could also define the operation as multiplication and the identity element as unit. Since both are valid, none are defined. If we desire a monoid instance for numbers, we can always define our own type that wraps a number:

```haskell
data Sum = Sum Int

instance Semigroup Sum where
  Sum x <> Sum y = Sum (x+y)

instance Monoid Sum where
  mempty = Sum 0
```

foldMap

One of the prominent abstractions in which monoids appear is the `foldMap` function, which has the following type:

```haskell
foldMap :: (Foldable t, Monoid m) => (a -> m) -> t a -> m
```

Given any `Foldable` structure `x`, `foldMap f x` first uses `f` to transform every element of the structure to an `m`, then uses the monoidal operator `<>` to combine them to a final result. For example, given our `Sum` instance above, we could sum a list like thus:

```haskell
> foldMap Sum [1,2,3,4]
Sum 10
```

This nicely decomposes the notion of "how to fold across a structure" from "how to combine intermediate results". The `foldMap` function itself can be expressed in terms of `foldr`:

```
foldMap f xs =
  foldr (\x acc -> f x <> acc) mempty xs
```

And conversely, `foldr` can be expressed in terms of `foldMap`. The encoding is slightly intricate, and depends on defining a type `Endo` representing functions with the same domain and codomain ("endomorphisms"), which are monoids under composition:

```
data Endo a = Endo (a -> a)

appEndo :: Endo a -> a -> a
appEndo (Endo f) = f

instance Semigroup (Endo a) where
  Endo f <> Endo g = Endo (f <> g)

instance Monoid (Endo a) where
  mempty = Endo id
```

We can now define a `foldr` as a `foldMap` on the `Endo` monoid.

```
foldr op acc xs =
  appEndo (foldMap (Endo . op) xs) acc
```

The trick is to partially apply each operator `op :: a -> b -> b` to each element of type `a`, which can be seen as giving us a collection of `b -> b` "accumulator transformer" functions, which we wrap in `Endo` surely to use its `Monoid` instances. We then simply have to apply the final `Endo` to the initial accumulator.

In fact, `foldMap` is not a normal function, but actually a method of the `Foldable` class, and an instance can be defined in terms of *either* `foldMap` or `foldr`. There is a somewhat complicated law for `Foldable` that specifies their equivalence, but in practice you only need to be concerned with it if you

manually specify *both* `foldMap` and `foldr`, which you would generally only do for a type that allows efficient specialised definitions.

Phantom Types

In this section we will briefly look at a programming technique that uses types to associate extra information with values and expressions, without any run-time overhead. As our example, we will consider writing a function that implements the kinetic energy formula:

$$E = \frac{1}{2}mv^2$$

Here *m* is the mass of the object, *v* is its velocity, and *E* is the resulting kinetic energy. We can represent this easily as a Haskell function:

```
energy :: Double -> Double -> Double
energy m v = 0.5 * m * (v ** 2)
```

However, it is quite easy to mix up which of the two arguments is the mass and which is the velocity, given that they have the same types (`Double`). To clarify matters, we can define some type synonyms:

```
type Mass = Double

type Velocity = Double

type Energy = Double

energy :: Mass -> Velocity -> Energy
energy m v = 0.5 * m * (v ** 2)
```

However, type synonyms are just that: *synonyms*. We can use a `Mass` whenever a `Velocity` is expected (or an `Energy` for that matter), so while the above makes the type of the `energy` function easier to read for a human, the type checker will not catch mistakes for us.

Instead, let us try to define *actual* types for representing the physical quantities of interest.

```haskell
data Mass = Mass Double
  deriving (Show)

data Velocity = Velocity Double
  deriving (Show)

data Energy = Energy Double
  deriving (Show)

energy :: Mass -> Velocity -> Energy
energy (Mass m) (Velocity v) = Energy (0.5 * m * (v ** 2))
```

Now we are certainly prevented from screwing up.

```
> energy (Mass 1) (Velocity 2)
Energy 2.0
> energy (Velocity 2) (Mass 1)

<interactive>:54:9-18: error: [GHC-83865]
    • Couldn't match expected type ‘Mass’ with actual type
‘Velocity’
    • In the first argument of ‘energy’, namely ‘(Velocity 2)’
      In the expression: energy (Velocity 2) (Mass 1)
      In an equation for ‘it’: it = energy (Velocity 2) (Mass 1)

<interactive>:54:22-27: error: [GHC-83865]
    • Couldn't match expected type ‘Velocity’ with actual type
‘Mass’
    • In the second argument of ‘energy’, namely ‘(Mass 1)’
      In the expression: energy (Velocity 2) (Mass 1)
      In an equation for ‘it’: it = energy (Velocity 2) (Mass 1)
```

However, this solution is somewhat heavyweight if we want to also support operations on the `Mass`, `Velocity`, and `Energy` types. After all, from a mathematical (or physical) standpoint, these are all *numbers* with units, and we may want to support number-like operations on them. Unfortunately, we end up having to implement duplicate functions for every type:

```haskell
doubleMass :: Mass -> Mass
doubleMass (Mass x) = Mass (2 * x)

doubleVelocity :: Velocity -> Velocity
doubleVelocity (Velocity x) = Velocity (2 * x)

doubleEnergy :: Energy -> Energy
doubleEnergy (Energy x) = Energy (2 * x)
```

This is not great, and becomes quite messy once we have many functions and types. Instead, let us take a step back and consider how physicists work with their formulae. Each number is associated with a *unit* (joules, kilograms, etc), and the units are tracked across calculations to ensure that the operations make sense (you cannot add joules and kilograms). This is very much like a type system, but they don't do it by inventing new kinds of numbers (well, they do that sometimes), but rather by adding a kind of *unit tag* to the numbers.

Haskell allows us to do a very similar thing. First, we define some types that *do not have constructors*:

```haskell
data Joule

data Kilogram

data MetrePerSecond
```

Similar to `Void`, we cannot ever have a value of type `Kilogram`, but we can use it at the type level. Specifically, we now define a type constructor `Q` for representing a quantity of some unit:

```haskell
data Q unit = Q Double
  deriving (Eq, Ord, Show)
```

Note that we do not actually use the type parameter `unit` in the right hand side of the definition. It is a *phantom type*, that exists only at compile-time, in order to constrain how `Q`s can be used. When constructing a value of type `Q`, we can instantiate that `unit` with anything we want. For example:

```haskell
weightOfUnladenSwallow :: Q Kilogram
weightOfUnladenSwallow = Q 0.020
```

We can still make mistakes when we create these values from scratch, by providing a nonsense unit:

```haskell
speedOfUnladenSwallow :: Q Joule
speedOfUnladenSwallow = Q 9
```

But at least we are prevented from mixing unrelated quantities:

```
> speedOfUnladenSwallow == weightOfUnladenSwallow

<interactive>:69:26-47: error: [GHC-83865]
    • Couldn't match type ‘Kilogram’ with ‘Joule’
      Expected: Q Joule
        Actual: Q Kilogram
```

Now we can define a safe `energy` function:

```haskell
energy :: Q Kilogram -> Q MetrePerSecond -> Q Joule
energy (Q m) (Q v) = Q (0.5 * m * (v ** 2))
```

And in contrast to before, we can define unit-preserving utility functions that apply to any `Q`:

```haskell
double :: Q unit -> Q unit
double (Q x) = Q (2 * x)
```

Phantom types is a convenient technique that requires only a small amount of boilerplate, and can be used to prevent incorrect use of APIs. The type errors are usually fairly simple as well. While phantom types do not guarantee the absence of errors - that requires techniques outside the scope of our course - they are a very practical programming technique, and one of our first examples of fancy use of types. We will return to these ideas later in the course.

> **Note**
>
> A full expression of the SI system of measures in a type system, including a proper handling of compound units such as m/s, requires type-level programming beyond simply using phantom types, and are beyond the scope of this course.

Using newtype

Instead of using `data`, it is best practice to define `Q` with `newtype`:

```
newtype Q unit = Q Double
```

We can use `newtype` whenever we define a datatype with a single constructor that has a single value - intuitively, whenever we simply "wrap" an underlying type. The difference between `data` and `newtype` are semantically almost nil (and the edge case does not matter for this course), but `newtype` is slightly more efficient, as the constructor does not exist when the program executes, meaning our use of `newtype` carries no performance overhead whatsoever. In contrast, a type declared with `data` must store the constructor.

Monads

Monads are a way to describe effectful computation in a functional setting. In Haskell, they are unavoidable as they are used to support true side effects (writing to files etc) with the built-in IO monad. However, monads can also be used as a powerful program structuring technique, even for programs that do not directly interact with the outside world.

Applicative Functors

To motivate the value of monads (and some of the supporting machinery), consider if we have a value `x :: Maybe Int`. We can see such a value as a computation that is either an `Int` or `Nothing`, with the latter case representing some kind of failure. We can interpret failure as a kind of *effect*, that is separate from the functional value (`Int`), although of course they are all just normal Haskell types.

We are often in a situation where we want to perform a computation on the result (if it exists), or otherwise propagate the failure. We can do this with explicit pattern matching:

```
case x of
  Nothing -> Nothing
  Just x' -> Just (x'+1)
```

To make this more concise, we can use the `Functor` instance for `Maybe` that we saw in the previous chapter:

```
fmap (+1) x
```

The above works because we are applying a pure function of type `Int -> Int` to the value in the `Maybe`. But what if the function is also produced from some potentially failing computation, e.g. what if `f :: Maybe (Int -> Int)`? Then `fmap f x` will be ill-typed, because `f` is not a function - it is a function

contained in an *effectful computation* (or less abstractly, stored in a `Maybe` container).

We can write it using pattern matching, of course:

```
case f of
  Just f' ->
    case x of
      Just x' -> Just (f' x')
      Nothing -> Nothing
  Nothing -> Nothing
```

But all this checking for `Failure` becomes quite verbose. Our salvation comes in the form of another typeclass, `Applicative`, which describes *applicative functors*. Any applicative functor must also be an ordinary functor, which we can add as a *superclass constraint*:

```
class Functor f => Applicative f where
  pure :: a -> f a
  (<*>) :: f (a -> b) -> f a -> f b
```

The `pure` method injects a pure value into an effectful computation. The `(<*>)` method applies a function stored in an applicative functor to a value stored in an applicative functor, yielding an applicative functor. Essentially, it is an extension of the notion of `Functor` to also allow the function to be in a "container".

This sounds (and is) abstract, and is perhaps best understood by looking at an example instance:

```
instance Applicative Maybe where
  pure x = Just x
  f <*> x = case f of
              Just f' ->
                case x of
                  Just x' -> Just (f' x')
                  Nothing -> Nothing
              Nothing -> Nothing
```

Or equivalently:

```haskell
instance Applicative Maybe where
  pure x = Just x
  Just f <*> Just x = Just (f x)
  _ <*> _ = Nothing
```

Now we can write `f <*> x` rather than writing out the `case` by hand. Even better, this will work not just when `f` and `x` make use of `Maybe` specifically, but *any type* that is an applicative functor - and as we shall see, any monad is also an applicative functor.

Applicative functors are subject to laws, just like normal functors. As always, they spell out fairly natural properties, even if the notation looks complicated:

- **Identity**: `pure id <*> v = v`. Similar to the identity law for `Functor`, this means that using `<*>` may not itself have any effect.
- **Composition**: `pure (.) <*> u <*> v <*> w = u <*> (v <*> w)`. This is essentially an associativity property, written in a slightly convoluted way to make the types line up.
- **Homomorphism**: `pure f <*> pure x = pure (f x)`. This property states that "unnecessary" uses if `<*>` have no effect.
- **Interchange**: `u <*> pure y = pure (\x -> x y) <*> u`. If only one of the operands to `<*>` has a real effect, then we can flip them around.

Together, these laws allow us to refactor code in natural ways. Usually we do not think consciously about them, as they are specified to fit our intuition about how programs execute.

Monads

Consider now the case where we have a value `x :: Maybe Int` and a function `f :: Int -> Maybe Int`. That is, the function now also has an effect - in this case, it can fail.

If we use `fmap f x`, we get something that is well-typed, but the result has type `Maybe (Maybe Int)`, which is unlikely to be what we desire. What we need is this function:

```
maybeBind :: Maybe a -> (a -> Maybe b) -> Maybe b
maybeBind Nothing _ = Nothing
maybeBind (Just x) f = f x
```

The `maybeBind` function passes a value of type `a` to a provided function that operates on `a`, but returns a potentially failing computation (`Maybe b`). If the original value is `Nothing`, then the final result is `Nothing`. We can now write our application as

```
maybeBind x f
```

or using backticks to make the operator infix:

```
x `maybeBind` f
```

The intuition here is "first execute `x` to produce some pure value, then pass this pure value to the function `f`".

It turns out that functions with the same "shape" as `maybeBind` are pretty common. For example, we can define a similar one for `Either`, where in the type we simply replace `Maybe a` with `Either e a`:

```
eitherBind :: Either e a -> (a -> Either e b) -> Either e b
eitherBind (Left e) _ = (Left e)
eitherBind (Right x) f = f x
```

Note that we operate only on the `Right` part of the `Either` value - the `Left` part, which usually represents some kind of error case, is undisturbed.

We can even define such a function for linked lists:

```
listBind :: [a] -> (a -> [b]) -> [b]
listBind [] _ = []
listBind (x : xs) f = f x ++ listBind xs f
```

The type looks a bit different than `maybeBind` and `eitherBind`, but that is just because of the syntactic sugar that lets us write `[a]` instead of `List a`.

It seems that when we have things that behave a bit like "containers" (in a general sense), we can define these "bind" functions that all have some similar behaviour. When we observe such similarities, we can put them into a typeclass. In this case, the typeclass is `Monad`, and the "bind" function is named the much more intuitive and easy-to-pronounce `>>=`:

```
class Applicative m => Monad m where
  (>>=) :: m a -> (a -> m b) -> m b
```

Haskell already provides a `Monad` instance for `Maybe`, but if it did not, we would define it as follows:

```
instance Monad Maybe where
  Nothing >>= _ = Nothing
  Just x >>= f = f x
```

Note that this definition is equivalent to the `maybeBind` above. Indeed, we could also write this definition like so:

```
instance Monad Maybe where
  (>>=) = maybeBind
```

An instance definition also exists for `Either`, equivalent to `eitherBind` above. The `Maybe` and `Either` monads are heavily used for tracking errors in Haskell code, similar to how we would use exceptions in other languages.

Intuition, nomenclature, and slang

Monads when viewed as an abstract interface can be quite tricky to get a grasp of. They are so abstract and general that it can be difficult to understand what the general concept *means*.

One good approach to learning is to essentially disregard the general concept, and focus only on specific monads. It is not so difficult to understand operationally what the `Monad` instances for `Maybe` and `Either` do, and in this we will mostly be working with *specific* monads.

Another approach is to focus on their form in Haskell. A monad is something that implements the `Monad` type class, which just means a we have access to `>>=` and `pure`, and can do anything these functions allow.

From a pedantic viewpoint, a function `f` of type `a -> m b`, where `m` is some monad (such as `Maybe`), returns a *monadic value* of type `m b`. We also sometimes say that it returns a *command in the monad* `m` which can produce a value of type `b` when "executed" (this term will make a bit more sense for the monads discussed below). We often also say that `f` is a *monadic function*. This is technically an abuse of nomenclature, but functions that "run within a specific monad" are such a common concept in Haskell that terse nomenclature is useful.

Monad laws

Any instance of `Monad` must satisfy the following laws:

- **Left identity**: `pure a >>= k = k a`. This states that "administrative" uses of `pure` and `>>=` can be removed, or said another way, that they must have no observable effect on their own. Intuitively, this is quite similar to the identity laws for `Functor` and `Applicative`.
- **Right identity**: `m >>= pure = m`. Same intuition as left identity.
- **Associativity**: `m >>= (\x -> k x >>= h) = (m >>= k) >>= h`. This states that the grouping of `>>=` must have no observable effect, in the same way that function composition is associative.

The associativity law is slightly easier to specify if instead of bind, we use the so-called "Kleisli arrow", that denotes composition of monadic functions:

```
(>=>) :: Monad m => (a -> m b) -> (b -> m c) -> a -> m c
f >=> g = \x -> f x >>= g
```

Then the statement of associativity is simply:

```
(f >=> g) >=> h = f >=> (g >=> h)
```

Deriving `fmap` and `<*>`

It turns out that when some type is an instance of `Monad`, the `fmap` and `<*>` methods from `Functor` and `Applicative` can be expressed in terms of `>>=` and `pure`. This means that when we implement this trifecta of instances, we only really have to think about `>>=` and `pure`. Specifically, we can define a function `liftM` that behaves like `fmap` for any monad:

```
liftM :: Monad m => (a -> b) -> m a -> m b
liftM f x = x >>= \x' -> pure (f x')
```

And similarly a function `ap` that behaves like `<*>`:

```
ap :: Monad m => m (a -> b) -> m a -> m b
ap f x = f >>= \f' ->
         x >>= \x' ->
           pure (f' x')
```

It can be shown that these are actually the only law-abiding definitions for these functions. Further, these functions are available from the builtin `Control.Monad` module. This means that when defining the instances for `Maybe`, we can take the following shortcuts:

```haskell
import Control.Monad (liftM, ap)

instance Applicative Maybe where
  (<*>) = ap

instance Functor Maybe where
  fmap = liftM
```

do-notation

Monads are particularly ergonomic to use in Haskell, and the main reason for this is a bit of syntactic sugar called `do`-notation, which allows us to imitate imperative programming.

Roughly speaking, the keyword `do` begins a block wherein every line corresponds to a monadic action, with the actions combined with `>>=`, and each statement after the first beginning a lambda. As an example,

```haskell
do x <- foo
   y <- bar
   baz x y
```

is syntactic sugar for

```haskell
foo >>= \x ->
bar >>= \y ->
baz x y
```

We can break a single statement over multiple lines if we are careful about how we indent them: the continuation lines must be indented more deeply than the first:

```haskell
do x <- foo one_argument
          more arguments...
   y <- x
   ...
```

Examples of defining and using monads

In the following we will look at examples of how to define and use our own monads. Most of these correspond to monads that are available and commonly used in standard Haskell libraries.

The Reader Monad

The Reader monad is a commonly used pattern for implicitly passing an extra argument to functions. It is often used to maintain configuration data or similar context that would be annoyingly verbose to handle manually, perhaps because it is not used in all cases of a function. We will call this implicit value an *environment*.

Operationally, a Reader monad with environment `env` and producing a value of type `a` is represented as a function accepting an `env` and returning `a`.

```
newtype Reader env a = Reader (env -> a)
```

Coming up with a definition of the Reader type itself requires actual creativity. On the other hand, the `Functor` / `Applicative` / `Monad` instances can almost be derived mechanically. I recommend starting with the the following template code:

```
instance Functor (Reader env) where
  fmap = liftM

instance Applicative (Reader env) where
  (<*>) = ap
  pure x = undefined

instance Monad (Reader env) where
  m >>= f = undefined
```

The `fmap` and `(<*>)` definitions are done, as discussed above. All we have to do is fill out the definitions of `pure` and `>>=`. I usually start with `pure`, because it is simpler. We start with this:

```
pure x = undefined
```

The `Applicative` class requires that the `pure` instance for `Reader env` has type `a -> Reader env a`. This means we know two things:

1. `x :: a`.
2. The right-hand side (currently `undefined`) must have type `Reader env a`.

Looking at the type definition of `Reader` above, we see that *any* value of type `Reader` takes the form of a `Reader` constructor followed by a payload. So we write this:

```
pure x = Reader undefined
```

Looking at the definition of `Reader` again, we see that this `undefined` must have type `env -> a`.

> **Hint**
>
> Instead of using `undefined`, we can also use `_`. This is a so-called *hole*, and will cause the compiler to emit an error message containing the type of the expression that is supposed to be located at the hole.

How do we construct a value of type `env -> a`? Well, we have a variable of type `a` (namely `x`), so we can simply write an anonymous function that ignores its argument (of type `env`) and returns `x`:

```
pure x = Reader $ \_env -> x
```

This concludes the definition of `pure`. We now turn our attention to `>>=`. The line of thinking is the same, where we systematically consider the types values we have available to us, and the types of values we are supposed to construct. This is our starting point:

```
m >>= f = undefined
```

We know:

1. `m :: Reader env a`
2. `f :: a -> Reader env b`
3. `undefined :: Reader env b`

We don't have anything of type `a`, so we cannot apply the function `f`. One we can do is deconstruct the value `m`, since we know this is a single-constructor datatype:

```
Reader x >>= f = undefined
```

Now we have the following information:

1. `x :: env -> a`
2. `f :: a -> Reader env b`
3. `undefined :: Reader env b`

The values `x` and `f` are functions for which we do not have values of the required argument type, so we cannot do anything to. But we can still start adding a constructor to the right-hand side, just as we did above for `pure`:

```
Reader x >>= f = Reader undefined
```

And again, we know that the `undefined` here must be a function taking an `env` as argument:

```
Reader x >>= f = Reader $ \env -> undefined
```

So far we have not used any creativity. We have simply done the only things possible given the structure of the types we have available. We now have this:

1. `x :: env -> a`
2. `f :: a -> Reader env b`
3. `env :: env`
4. `undefined :: b`

Since we now have a variable of type `env`, we can apply the function `x`. We do so:

```
Reader x >>= f = Reader $ \env ->
                   let x' = x env
                    in undefined
```

Now we have `x' :: a`, which allows us to apply the function `f`:

```
Reader x >>= f = Reader $ \env ->
                   let x' = x env
                       f' = f x'
                    in undefined
```

We have `f' :: Reader env b`, so we can pattern match on `f'` to extract the payload. When a type has only a single constructor, we can do this directly in `let`, without using `case`:

```
Reader x >>= f = Reader $ \env ->
                   let x' = x env
                       Reader f' = f x'
                    in undefined
```

Now we have `f' :: env -> b`, which is exactly what we need to finish the definition:

```
Reader x >>= f = Reader $ \env ->
                   let x' = x env
                       Reader f' = f x'
                    in f' env
```

This finishes the `Monad` instance for `Reader`. However, we still need to define the programming interface for the monad. Some monads (such as `Maybe`, `Either`, or lists) directly expose their type definition. But for more effect-oriented monads like `Reader`, we usually want to hide their definition and instead provide an abstract interface. This usually takes the form of a function for executing a monadic computation, as well as various functions for constructing monadic computations. For `Reader`, we will implement the following interface:

```haskell
runReader :: env -> Reader env a -> a

ask :: Reader env env

local :: (env -> env) -> Reader env a -> Reader env a
```

The `runReader` function is used to execute a `Reader` computation, given an initial environment. It has the following definition:

```haskell
runReader env (Reader f) = f env
```

The `ask` command is used to retrieve the environment. It has the following definition:

```haskell
ask = Reader $ \env -> env
```

The `local` function executes a given `Reader` command in a modified environment. This does not allow stateful mutation, as the environment is only modified while executing the provided command, not any subsequent ones:

```haskell
local f (Reader g) = Reader $ \env -> g (f env)
```

When using the `Reader` monad, we will exclusively make use of `runReader`, `ask`, and `local` (and functions defined in terms of these), and never directly construct `Reader` values.

Using the Reader Monad

The `Reader` monad is mostly useful when writing functions with many cases, where only some need to make use of the environment. This means compelling examples are relatively verbose. You will see such examples in the course exercises, but for now, we will use a somewhat contrived example of modifying a binary tree of integers, such that every node is incremented with its distance from the root.

First we define the datatype.

```haskell
data Tree
  = Leaf Int
  | Inner Tree Tree
  deriving (Show)
```

```
> (Leaf 0 `Inner` Leaf 0) `Inner` Leaf 0
Inner (Inner (Leaf 0) (Leaf 0)) (Leaf 0)
```

Then we can define a monadic recursive function over `Tree`:

```haskell
incLeaves :: Tree -> Reader Int Tree
incLeaves (Leaf x) = do
  depth <- ask
  pure $ Leaf $ x + depth
incLeaves (Inner l r) = do
  l' <- local (+ 1) $ incLeaves l
  r' <- local (+ 1) $ incLeaves r
  pure $ Inner l' r'
```

And then we can run our contrived function on some provided tree:

```
> runReader 0 $ incLeaves $ (Leaf 0 `Inner` Leaf 0) `Inner` Leaf 0
Inner (Inner (Leaf 2) (Leaf 2)) (Leaf 1)
```

The State Monad

The State monad is similar to the Reader monad, except now we allow subsequent commands to modify the state. We represent a stateful computation as a function that accepts a state (of some abstract type `s`) and returns a new state, along with a value.

```haskell
newtype State s a = State (s -> (a, s))
```

The definitions of the type class instances follow similarly to the ones for Reader, and can be derived largely through the same technique of considering the types of values we have available to us.

```haskell
instance Monad (State s) where
  State m >>= f = State $ \state ->
    let (x, state') = m state
        State f' = f x
     in f' state'
```

Note that the have the opportunity to make a mistake here, by using the original `state` instead of the modified `state'` produced by `m`.

```haskell
instance Functor (State s) where
  fmap = liftM

instance Applicative (State s) where
  pure x = State $ \state -> (x, state)
  (<*>) = ap
```

We also provide the following API for executing and interacting with stateful computations. First, computation requires that we provide an initial state and returns the final state:

```haskell
runState :: s -> State s a -> (a, s)
runState s (State f) = f s
```

The `put` and `get` functions are for reading and writing the state.

```haskell
get :: State s s
get = State $ \s -> (s, s)

put :: s -> State s ()
put s = State $ \_ -> ((), s)
```

If we want to store more than a single value as state, we simply store a tuple, record, or some other compound structure.

Typically we also define a utility function `modify` for modifying the state with a function. This does not need to access the innards of the state monad, but can be defined in terms of `get` and `put`:

```haskell
modify :: (s -> s) -> State s ()
modify f = do x <- get
              put $ f x
```

Using the State monad

We will use the State monad to implement a function that renumbers the leaves of a tree with their left-to-right traversal ordering.

```haskell
numberLeaves :: Tree -> State Int Tree
numberLeaves (Leaf _) = do
  i <- get
  put (i + i)
  pure $ Leaf $ i + 1
numberLeaves (Inner l r) = do
  l' <- numberLeaves l
  r' <- numberLeaves r
  pure $ Inner l' r'
```

We may now use it as follows:

```haskell
> runState 0 $ numberLeaves $ (Leaf 0 `Inner` Leaf 0) `Inner` Leaf 0
(Inner (Inner (Leaf 0) (Leaf 1)) (Leaf 2),3)
```

Combining Reader and State

One limitation of monads - in particular the simple form we study in AP - is that they do not compose well. We cannot in general take two monads, such as `Reader` and `State`, and combine them into a single monad that supports both of their functionalities. There are techniques that allow for this, such as *monad transformers*, but they are somewhat complex and outside the scope of AP. Instead, if we wish to have a monad that supports both a read-only environment (such as with `Reader`) and a mutable store (such as with `State`), then we must write a monad from scratch, such as the following `RS` monad.

```haskell
newtype RS env s a = RS (env -> s -> (a, s))
```

See how the function we use to represent the monad takes two arguments, `env` and `s`, corresponding to the environment and store respectively, but returns only a new store.

The `Monad` instance itself is a little intricate, but it just combines the dataflow that we also saw for the `Reader` and `State` monads above:

```
instance Monad (RS env s) where
  RS m >>= f = RS $ \env state ->
    let (x, state') = m env state
        RS f' = f x
     in f' env state'
```

The `Functor` and `Applicative` instances are then just the usual boilerplate.

```
instance Functor (RS env s) where
  fmap = liftM

instance Applicative (RS env s) where
  pure x = RS $ \_env state -> (x, state)
  (<*>) = ap
```

We can then define the following API for `RS`, providing both State and Reader-like operations:

```
runRS :: env -> s -> RS env s a -> (a, s)
runRS env state (RS f) = f env state

rsGet :: RS env s s
rsGet = RS $ \_env state -> (state, state)

rsPut :: s -> RS env s ()
rsPut state = RS $ \_env _ -> ((), state)

rsAsk :: RS env s env
rsAsk = RS $ \env state -> (env, state)

rsLocal :: (env -> env) -> RS env s env -> RS env s env
rsLocal f (RS g) = RS $ \env state -> g (f env) state
```

Monadic and Applicative Parsing

It is sometimes the case that we have to operate on data that is not already in the form of a richly typed Haskell value, but is instead stored in a file or transmitted across the network in some serialised format - usually in the form of a sequence of bytes or characters. Although such situations are always regrettable, in this chapter we shall see a flexible technique for making sense out of unstructured data: *parser combinators*.

Parsing Integers Robustly

To start with, consider turning a `String` of digit characters into the corresponding `Integer`. That is, we wish to construct the following function:

```
readInteger :: String -> Integer
```

The function `ord :: Char -> Int` from `Data.Char` can convert a character into its corresponding numeric code. Exploiting the fact that the integers have consecutive codes, we can write a function for converting a digit character into its corresponding `Integer`. Note that we have to convert the `Int` produced by `ord` into an `Integer`:

```
import Data.Char (ord)

charInteger :: Char -> Integer
charInteger c = toInteger $ ord c - ord '0'
```

Exploiting the property that the numeric characters are consecutively encoded, we can implement `readInt` with a straightforward recursive loop over the characters of the string, from right to left:

```haskell
readInteger :: String -> Integer
readInteger s = loop 1 $ reverse s
  where
    loop _ [] = 0
    loop w (c : cs) = charInteger c * w + loop (w * 10) cs
```

Example use:

```
> readInteger "123"
123
```

However, see what happens if we pass in invalid input:

```
λ> readInteger "xyz"
8004
```

Silently producing garbage on invalid input is usually considered poor engineering. Instead, our function should return a `Maybe` type, indicating invalid input by returning `Nothing`. This can be done by using `isDigit` from `Data.Char` to check whether each character is a proper digit:

```haskell
readIntegerMaybe :: String -> Maybe Integer
readIntegerMaybe s = loop 1 $ reverse s
  where
    loop _ [] = Just 0
    loop w (c : cs)
      | isDigit c = do
          x <- loop (w * 10) cs
          pure $ charInteger c * w + x
      | otherwise =
          Nothing
```

Note how we are using the fact that `Maybe` is a monad to avoid explicitly checking whether the recursive call to `loop` fails.

We now obtain the results we would expect:

```
> readIntegerMaybe "123"
Just 123
> readIntegerMaybe "xyz"
Nothing
```

Composing Parsers

Now suppose we extend the problem: we must now parse an integer, followed by a space, followed by another integer. We can of course write a function that does this from scratch, but it would be better if we could reuse our function that parses a single integer. Unfortunately, this is not possible with `readIntegerMaybe`, as it requires that the input string consists solely of digits. We could split the string by spaces in advance, but this is rather ad-hoc. Instead, let us construct a function that reads a *leading* integer from a string, and then returns a *remainder string*.

```
readLeadingInteger :: String -> Maybe (Integer, String)
readLeadingInteger s =
  case span isDigit s of
    ("", _) -> Nothing
    (digits, s') -> Just (loop 1 $ reverse digits, s')
  where
    loop _ [] = 0
    loop w (c : cs) =
      charInteger c * w + loop (w * 10) cs
```

The `span` function breaks a string into two parts: the prefix of characters that satisfy the predicate (here `isDigit`), and a *remainder string* with the prefix removed. If the first part is empty, we return `Nothing`, as an integer requires at least a single digit. Otherwise we convert the digits into an `Integer` and return it along with the remaining string.

```
> readLeadingInteger "123"
Just (123,"")
> readLeadingInteger "123 456"
Just (123," 456")
```

Let us also write two more helper functions: one that reads a single leading space from a string (and returns the remainder), and one that asserts that the string is empty.

```
readSpace :: String -> Maybe (Char, String)
readSpace (' ' : s) = Just (' ', s)
readSpace _ = Nothing

readEOF :: String -> Maybe ((), String)
readEOF "" = Just ((), "")
readEOF _ = Nothing
```

Note `readLeadingInteger`, `readSpace`, and `readEOF` all have types of the same form: `String -> Maybe (a, String)` for some `a`. This strongly suggests that there is some kind of commonality that we can exploit to construct ways of composing them. But first, let us try to compose them manually, to solve the problem of reading two space-separated integers:

```
readTwoIntegers :: String -> Maybe ((Integer, Integer), String)
readTwoIntegers s =
  case readLeadingInteger s of
    Nothing -> Nothing
    Just (x, s') -> case readSpace s' of
      Nothing -> Nothing
      Just (_, s'') ->
        case readLeadingInteger s'' of
          Nothing -> Nothing
          Just (y, s''') -> pure ((x, y), s''')
```

```
> readTwoIntegers "123 456"
Just ((123,456),"")
```

While it works, it is quite verbose with all that explicit pattern-matching of `Nothing` / `Just`. We can exploit the fact that `Maybe` is a monad to write it a bit more concisely:

```haskell
readTwoIntegers2 :: String -> Maybe ((Integer, Integer), String)
readTwoIntegers2 s = do
  (x, s') <- readLeadingInteger s
  (_, s'') <- readSpace s'
  (y, s''') <- readLeadingInteger s''
  Just ((x, y), s''')
```

However, it is still quite annoying that we manually have to thread the `String` around. It also means we can screw up, and use the wrong one, since they all have the same type. It would be better if this kind of book-keeping was done automatically behind the scenes. And indeed that is possible, by creating a *parser monad*, whose structure is essentially the same as the functions above, and which hides away the book-keeping behind the monadic interface.

A simple parser monad

As stated above, all our parser functions are of the form `String -> Maybe (a, String)`. We will make that the definition of our parser type:

```haskell
newtype Parser a = Parser {runParser :: String -> Maybe (a, String)}
```

Now `runParser :: Parser a -> String -> Maybe (a, String)` is the function for running a parser on some input.

This type can be made a monad. Its definition looks like this:

```haskell
instance Monad Parser where
  f >>= g = Parser $ \s ->
    case runParser f s of
      Nothing -> Nothing
      Just (x, s') -> runParser (g x) s'
```

The idea is the following. We have `f :: Parser a` and `g :: a -> Parser b`. We can run the parser `f` to get either a parse error or a value `x :: a` and a remainder string `s'`. We can then pass `x` to `g` in order to obtain a `Parser b`,

which we can then run on `s'`. It is quite similar to a state monad combined the `Maybe` monad.

We also need to define `Applicative` and `Functor` instances. As always, the only case that is not purely mechanical is `pure`, which is used for a "parser" that consumes no input and always succeeds.

```
import Control.Monad (ap)

instance Functor Parser where
  fmap f x = do
    x' <- x
    pure $ f x'

instance Applicative Parser where
  (<*>) = ap
  pure x = Parser $ \s -> Just (x, s)
```

Now that we have defined the fundamental machinery, we can define some very simple primitive parsers. We start with one that parses a single character that satisfies some predicate:

```
satisfy :: (Char -> Bool) -> Parser Char
satisfy p = Parser $ \s -> case s of
  c : cs ->
    if p c
      then Just (c, cs)
      else Nothing
  _ -> Nothing


> runParser (satisfy isDigit) "123"
Just ('1',"23")
```

And a parser that succeeds only if there is no more input left:

```
eof :: Parser ()
eof = Parser $ \s ->
  case s of
    "" -> Just ((), "")
    _ -> Nothing
```

While `eof` may seem a bit odd, we often use it as the very last step of parsing a complete file or data format, as usually we do not want to allow trailing garbage.

A parser *combinator* is a function on parsers. As our first parser combinator, we will construct one that accepts a list of parsers, and tries them all in turn. This is useful in the very common case where multiple inputs can be valid.

```
choice :: [Parser a] -> Parser a
choice [] = Parser $ \_ -> Nothing
choice (p : ps) = Parser $ \s ->
  case runParser p s of
    Nothing -> runParser (choice ps) s
    Just (x, s') -> Just (x, s')
```

The parsers and combinators above directly manipulate the parser state and use the `Parser` constructor. We say they are *primitive parsers*. However, the vast majority of parsers we will write will not be primitive, but will instead be built in terms of the primitives, using the monadic interface. For example, we can define a derived parser that parses an expected string and returns it:

```
import Control.Monad (void)

chunk :: String -> Parser String
chunk [] = pure ""
chunk (c : cs) = do
  void $ satisfy (== c)
  void $ chunk cs
  pure $ c : cs


> runParser (choice [chunk "foo", chunk "bar"]) "foo"
Just ("foo","")
```

Using Parser Combinators

Let us try to rewrite our integer parser using the `Parser` type. First, we define a function for parsing a single digit, including decoding.

```haskell
parseDigit :: Parser Integer
parseDigit = do
  c <- satisfy isDigit
  pure $ toInteger $ ord c - ord '0'
```

Here is how we would use it to write a function that parses two digits:

```haskell
parseTwoDigits :: Parser (Integer, Integer)
parseTwoDigits = do
  x <- parseDigit
  y <- parseDigit
  pure (x, y)
```

We also construct a combinator that repeatedly applies a given parser until it fails:

```haskell
many :: Parser a -> Parser [a]
many p =
  choice
    [ do
        x <- p
        xs <- many p
        pure $ x : xs,
      pure []
    ]
```

```
> runParser (many parseDigit) "123"
Just ([1,2,3],"")
```

(Ponder what happens if we flipped the elements in the list we pass to `choice` in the definition of `many`.)

We have to be careful when using `many`: if it is given a parser that can succeed without consuming any input (such as `eof`), it will loop forever.

Also, `many` is not quite what we need, as it will succeed even if the given parser succeeds zero times. The variant `some` requires that the given parser succeeds at least once:

```haskell
some :: Parser a -> Parser [a]
some p = do
  x <- p
  xs <- many p
  pure $ x : xs
```

Now we can write our function for parsing integers:

```haskell
parseInteger = do
  digits <- some parseDigit
  pure $ loop 1 $ reverse digits
  where
    loop _ [] = 0
    loop w (d : ds) =
      d * w + loop (w * 10) ds
```

Or more concisely:

```haskell
parseInteger :: Parser Integer
parseInteger = loop 1 . reverse <$> some parseDigit
  where
    loop _ [] = 0
    loop w (d : ds) =
      d * w + loop (w * 10) ds
```

And finally we can use it to build a parser for two space-separated integers:

```haskell
parseTwoIntegers :: Parser (Integer, Integer)
parseTwoIntegers = do
  x <- parseInteger
  _ <- satisfy (== ' ')
  y <- parseInteger
  pure (x, y)
```

See how easy it is to understand what input it parses (once you understand parser combinators, mind you), compared to the original `readTwoIntegers` function, which was littered with book-keeping.

```
> runParser parseTwoIntegers "123 456"
Just ((123,456),"")
```

We would likely also want to use the `eof` parser to assert that no garbage follows the second integer.

Tokenisation

Syntaxes for programming languages and data formats are usually structured in the form of rules for how to combine *tokens* (sometimes also called *lexemes*), along with rules for how the tokens are formed.

Following the example above, we could consider integers (in the form of a nonzero number of decimal digits) to be a token. In many languages, tokens can be separated by any number of whitespace characters. In the traditional view of parsing, the syntactic analysis is preceded by a *lexical analysis* that splits the input into tokens, using for example regular expressions. With parser combinators, lexical and syntactic analysis is done simultaneously. Yet to correctly handle the lexical rules of a language, we must exercise significant discipline.

> **Info**
>
> Not all languages have straightforward lexical rules. Some (such as Python or Haskell) use indentation to structure their syntactical rules, others have context-sensitive tokenisation, and others allow extensible or user-defined syntaxes. Parser combinators can cope with all of these, but for simplicity we stick to more well-behaved syntaxes in these notes.

Whitespace

One core concern is how to handle whitespace. The most common convention is that each parser must consume any *trailing* whitespace, but can assume that there is no *leading* whitespace. If we systematically follow this rule, then we can ensure that we never fail to handle whitespace. The exception is the top parser, where we cannot assume that the initial input does not have leading

whitespace. Still, systematically doing anything without mistakes is difficult for humans. In the following, we define some simple (but rigid) rules that are easy to follow in our code.

First we write a parser that skips any number of whitespace characters.

```
import Data.Char (isSpace)

space :: Parser ()
space = do
  _ <- many $ satisfy isSpace
  pure ()
```

Then we write a parser combinator that takes any parser and consumes subsequent whitespace:

```
lexeme :: Parser a -> Parser a
lexeme p = do x <- p
              space
              pure x
```

Now we institute a rule: whenever we write a parser that operates at the lexical level, it must be of the form

```
lFoo = lexeme $ ...
```

and *no other parser than those of this form* is allowed to use `lexeme` or `space` directly. The `l` prefix is a mnemonic for *lexical* - similarly we will begin prefixing our syntax-level parsers with `p`.

For example, this would now be a parser for decimal integers that consumes trailing whitespace:

```
lDecimal :: Parser Integer
lDecimal = lexeme $ loop 1 . reverse <$> some parseDigit
  where
    loop _ [] = 0
    loop w (d : ds) =
      d * w + loop (w * 10) ds
```

Note that we will use `parseDigit` (which does not handle whitespace) as a low level building block. We must only use these from within `l`-prefixed functions.

Now we can easily a function that parses any number of whitespace-separated decimal numbers:

```
pDecimals :: Parser [Integer]
pDecimals = many lDecimal
```

```
> runParser pDecimals "123  456   789   "
Just ([123,456,789],"")
```

However, we will fail to parse a string with leading whitespace:

```
> runParser pDecimals " 123"
Just ([]," 123")
```

The solution is to explicitly skip leading whitespace:

```
> runParser (space >> pDecimals) " 123"
Just ([123],"")
```

We do this *only* in the top level parser - usually in the one that is passed immediately to the function that executed the `Parser` monad itself.

Longest match

When lexing the string `"123"`, we see it as a single token `123` rather than three tokens `1`, `2`, `3`? The reason for this is that most grammars follow the *longest match* (or *maximum munch*) rule: each token extend as far as possible. This principle has actually been baked into the definition of the `many` combinator above, as it tries the recursive case before the terminal case. If we instead defined `many` like this:

```haskell
many :: Parser a -> Parser [a]
many p =
  choice
    [ pure [],
      do
        x <- p
        xs <- many p
        pure $ x : xs
    ]
```

Then we would observe the following behaviour (because `lDecimal` uses `some`, which uses `many`):

```
> runParser lDecimal "123"
Just (1,"23")
```

Thus, simply by defining `many` the way we originally did, we obtain conventional behaviour.

A larger example

Let us study a larger example of parsing, including proper tokenisation, handling of keywords, and transforming a given grammar to make it amenable to parsing.

Consider parsing a language of Boolean expressions, represented by the following Haskell datatype.

```haskell
data BExp
  = Lit Bool
  | Var String
  | Not BExp
  | And BExp BExp
  | Or BExp BExp
  deriving (Eq, Show)
```

The concrete syntax will be strings such as `"x"`, `"not x"`, `"x and true"`, `"true and y or z"`. We write down a grammar in EBNF:

```
var ::= ? one or more alphabetic characters ? ;
BExp ::= "true"
       | "false"
       | "not" BExp
       | BExp "and" BExp
       | BExp "or" BExp ;
```

The words enclosed in double quotes are *terminals* (tokens). The lowercase `var` is also a token, but is defined informally using an EBNF comment. We adopt the convention that tokens can be separated by whitespace, and that we follow the longest match rule. This is strictly speaking an abuse of convention, as the handling of whitespace ought also be explicit in the grammar, but it is common to leave it out for simplicity.

Note that the grammar does not *exactly* match the Haskell abstract syntax tree (AST) definition - in particular, the `"true"` and `"false"` cases are combined into a single `Lit` constructor. This is quite common, and we will see many cases where superfluous details of the form of the concrete syntax are simplified away in the AST.

Our first attempt at parsing Boolean expressions looks like this:

```haskell
import Data.Char (isAlpha)

lVar :: Parser String
lVar = lexeme $ some $ satisfy isAlpha

lTrue :: Parser ()
lTrue = lexeme $ void $ chunk "true"

lFalse :: Parser ()
lFalse = lexeme $ void $ chunk "false"

lAnd :: Parser ()
lAnd = lexeme $ void $ chunk "and"

lOr :: Parser ()
lOr = lexeme $ void $ chunk "or"

lNot :: Parser ()
lNot = lexeme $ void $ chunk "not"

pBool :: Parser Bool
pBool =
  choice
    [ do
        lTrue
        pure True,
      do
        lFalse
        pure False
    ]

pBExp :: Parser BExp
pBExp =
  choice
    [ Lit <$> pBool,
      Var <$> lVar,
      do
        lNot
        e <- pBExp
        pure $ Not e,
      do
        x <- pBExp
        lAnd
        y <- pBExp
        pure $ And x y,
      do
        x <- pBExp
```

```
        lOr
        y <- pBExp
        pure $ Or x y
    ]
```

Note now the structure of the code fairly closely matches the structure of the grammar. This is is always something we seek to aspire to.

Keywords

Unfortunately, despite looking pretty, it fails to work properly for any but trivial cases:

```
> runParser pBExp "x"
Just (Var "x","")
> runParser pBExp "true"
Just (Lit True,"")
> runParser pBExp "not x"
Just (Var "not","x")
```

The third input (`"not x"`) doesn't work. How can that be? The reason it goes wrong can be seen by trying to parse a variable name:

```
> runParser lVar "not"
Just ("not","")
```

Words such as `not` , `and` , `or` , `true` , and `false` are also valid variable names according to our parser. While we forgot to state it explicitly in the grammar, our intention is for these words to be *keywords* (or *reserved words*), which are not valid as variables. So now we add another side condition to the grammar: a `var` must not be one of `not` , `and` , `or` , `true` , and `false` . How do we implement this in the parser? After all, in `lVar` we cannot know whether we will end up reading a keyword until after we are done. We actually need to add a new primitive operation to our parser: *explicit failure*. We do this by implementing the `MonadFail` type class, which requires a single method, `fail :: String -> Parser a` :

```
instance MonadFail Parser where
  fail _ = Parser $ \_ -> Nothing
```

The argument to `fail` is for an error message, which is not supported by our parser definition, so we just throw it away. The result of `fail` is a parser that always fails. We can use this to fix our definition of `lVar` to explicitly not allow keywords:

```
keywords :: [String]
keywords = ["not", "true", "false", "and", "or"]

lVar :: Parser String
lVar = lexeme $ do
  v <- some $ satisfy isAlpha
  if v `elem` keywords
    then fail "keyword"
    else pure v
```

This shows some of the strength (and danger) of monadic parsing: we can intermingle arbitrary Haskell-level decision making with the purely syntactical analysis. This allows parser combinators to support heinously context-sensitive grammars when necessary, but as mentioned above, we will stick to more well-behaved ones in this course.

Now we get the desired behaviour:

```
> runParser pBExp "not x"
Just (Not (Var "x"),"")
```

But another case still behaves oddly:

```
> runParser pBExp "truexx"
Just (Lit True,"xx")
```

We don't really want to parse `"truexx"` as the Boolean literal `true` followed by some garbage - this is in violation of the longest match rule. We can fix this by requiring that a keyword is *not* followed by an alphabetic character. This does require us to add a new primitive parser to our parsing library (but this is the last one):

```haskell
notFollowedBy :: Parser a -> Parser ()
notFollowedBy (Parser f) = Parser $ \s ->
  case f s of
    Nothing -> Just ((), s)
    _ -> Nothing
```

The `notFollowedBy` combinator succeeds *only* if the provided parser fails (and if so, consumes no input). We can then use this to define a combinator for parsing keywords:

```haskell
lKeyword :: String -> Parser ()
lKeyword s = lexeme $ do
  void $ chunk s
  notFollowedBy $ satisfy isAlpha
```

Using `lKeyword`, there is no need for dedicated functions for parsing the individual keywords, although you can still use them if you like. I prefer using `lKeyword` directly:

```haskell
lKeyword :: String -> Parser ()
lKeyword s = lexeme $ do
  void $ chunk s
  notFollowedBy $ satisfy isAlpha

pBool :: Parser Bool
pBool =
  choice
    [ lKeyword "true" >> pure True,
      lKeyword "false" >> pure False
    ]

pBExp :: Parser BExp
pBExp =
  choice
    [ Lit <$> pBool,
      Var <$> lVar,
      do
        lKeyword "not"
        Not <$> pBExp,
      do
        x <- pBExp
        lKeyword "and"
        y <- pBExp
        pure $ And x y,
      do
        x <- pBExp
        lKeyword "or"
        y <- pBExp
        pure $ Or x y
    ]
```

```
> runParser pBExp "truexx"
Just (Var "truexx","")
```

Left recursion

We have now implemented tokenisation properly. Unfortunately, our parser still does not work:

```
> runParser pBExp "x and y"
Just (Var "x","and y")
```

The reason is the `choice` in `pBExp`. Our definition of `choice` takes the *first* parser that succeeds, which is the one that produces `Var`, and so it never proceeds to the one for `And`.

> **Info**
>
> There are ways of implementing parser combinators such that the ordering does not matter, which is largely by using a list instead of a `Maybe` in the definition of `Parser`. However, this will not solve the nontermination problem discussed below.

We can try to fix our parser by changing the order of parsers provided to `choice`:

```
pBExp :: Parser BExp
pBExp =
  choice
    [ do
        x <- pBExp
        lKeyword "and"
        y <- pBExp
        pure $ And x y,
      do
        x <- pBExp
        lKeyword "or"
        y <- pBExp
        pure $ Or x y,
      Lit <$> pBool,
      Var <$> lVar,
      do
        lKeyword "not"
        Not <$> pBExp
    ]
```

Unfortunately, now the parser goes into an infinite loop:

```
> runParser pBExp "x"
... waiting for a long time ...
```

The operational reason is that underneath all the monadic syntax sugar, our parsers are just recursive Haskell functions. Looking at `pBExp`, we see that the very first thing it does is recursively invoke `pBExp`. If we look at the EBNF grammar for Boolean expressions, we also see that some of the production rules for `BExp` start with `BExp`. In the nomenclature of parser theory, this is called *left recursion*. The style of parser combinator library we are studying here is equivalent to so-called *recursive descent parsers with arbitrary lookahead* (or *LL(k)*), which are known to not support left recursion. The solution to this problem is to rewrite the grammar to eliminate left-recursion. If you need a refresher on how to do this, see Grammars and parsing with Haskell using parser combinators, but the idea is to split the non-recursive cases into a separate nonterminal (often called `Atom`).

Transforming the grammar (note that we do not modify the Haskell AST definition) provides us with the following:

```
var ::= ? one or more alphabetic characters ? ;
Atom ::= "true"
        | "false"
        | var
        | "not" BExp ;
BExp' ::= "and" Atom BExp'
        | "or" Atom BExp'
        | ;
BExp ::= Atom BExp' ;
```

Note that we have decided that the `and` operator is left-associative - meaning that `"x and y and z"` is parsed as `"(x and y) and x"` (or would be if our syntax supported parentheses).

A grammar without left-recursion can be implemented fairly straightforwardly. The idea is that parsing a `BExp` consists of initially parsing a `BExp2`, followed by a chain of zero or more `and` / `or` clauses.

```haskell
pAtom :: Parser BExp
pAtom =
  choice
    [ Lit <$> pBool,
      Var <$> lVar,
      do
        lKeyword "not"
        Not <$> pBExp
    ]

pBExp :: Parser BExp
pBExp = do
  x <- pAtom
  chain x
  where
    chain x =
      choice
        [ do
            lKeyword "and"
            y <- pBExp
            chain $ And x y,
          do
            lKeyword "or"
            y <- pBExp
            chain $ Or x y,
          pure x
        ]
```

```
> runParser pBExp "x and y"
Just (And (Var "x") (Var "y"),"")
> runParser pBExp "x and y and z"
Just (And (And (Var "x") (Var "y")) (Var "z"),"")
```

Usually when constructing a parser, we do not expose the raw parser functions (such as `pBExp`), but instead define a convenient wrapper function, such as the following:

```haskell
parseBExp :: String -> Maybe BExp
parseBExp s = fst <$> runParser p s
  where
    p = do
      space
      x <- pBExp
      eof
      pure x
```

Note how this "top level parser" also takes care to skip *leading* whitespace (in contrast to lexer functions that skip *trailing* whitespace as their principle), and asserts that no input must remain unconsumed.

Operator precedence

We still have a final problem we must address. Consider parsing the input `"x or y and z"`:

```
> parseBExp "x or y and z"
Just (And (Or (Var "x") (Var "y")) (Var "z"))
```

Whether this is correct or not of course depends on how the grammar is specified, but the usual convention in logical formulae is that conjunction (`and`) binds tighter than disjunction (`or`). This is similar to how mathematical notation assigns higher priority to multiplication than addition. Generally, a grammar specification will come with a set of side conditions specifyint an operator priority (and associativity). The way to handle operator priority in a parser build with combinators is to perform yet another a grammar transformation. The idea is to split the grammar rules into multiple *levels*, with one level per priority. For our Boolean expressions, the transformed grammar looks like this:

```
var ::= ? one or more alphabetic characters ? ;

Atom ::= "true"
        | "false"
        | var
        | "not" BExp ;

BExp1' ::= "and" Atom BExp1'
        | ;
BExp1 ::= Atom BExp1' ;

BExp0' ::= "or" BExp1 BExp0'
        | ;
BExp0 ::= BExp1 BExp0' ;

BExp ::= BExp0 ;
```

And performing the corresponding transformation on our parser (or simply rewriting it from scratch, given this new grammar) produces this:

```haskell
pBExp1 :: Parser BExp
pBExp1 = do
  x <- pAtom
  chain x
  where
    chain x =
      choice
        [ do
            lKeyword "and"
            y <- pAtom
            chain $ And x y,
          pure x
        ]

pBExp0 :: Parser BExp
pBExp0 = do
  x <- pBExp1
  chain x
  where
    chain x =
      choice
        [ do
            lKeyword "or"
            y <- pBExp1
            chain $ Or x y,
          pure x
        ]

pBExp :: Parser BExp
pBExp = pBExp0
```

And now we observe the parser result that we desire:

```
> parseBExp "x or y and z"
Just (Or (Var "x") (And (Var "y") (Var "z")))
```

Megaparsec

While the parser library implemented above is fully operational, it has serious flaws that leave it unsuitable for production use:

1. It is rather inefficient. This is partially because of the use of `String` as the fundamental type, but mostly because of how `choice` is implemented, which has to keep track of the original input essentially forever, even if backtracking will never become relevant.

2. It produces no error messages, instead simply returning `Nothing`.

While point 1 does not matter much for AP, point 2 makes it very difficult to debug your parsers - which is certainly going to have an impact. For the exercises and assignments, you will therefore be using a state-of-the-art industrial-strength parser combinator library: Megaparsec.

The downside of using an industrial parser library such as Megaparsec is that is is complicated. It has a rich programming interface and more complicated types than what we need in AP. However, a subset of Megaparsec's interface is identical to the interface presented above (this is not a coincidence), and this is what we will be using in AP.

The facilities we will need are from the `Text.Megaparsec` module. Megaparsec is quite well documented, so it may be worth your time to skim the documentation, although the information provided here is intended to be sufficient for our needs. In Megaparsec, parsers are monadic functions in the `Parsec e s` monad, which is itself a specialisation of the `ParsecT` monad *transformer* - a concept that lies outside of the AP curriculum. The point of this flexibilty is to be generic in the underlying *stream type* (e.g. the kind of "string" we are parsing), the form that errors can take, and so on. We do not need such flexibility, and the first thing we need to do when using Megaparsec is to define the following type synonym:

```
import Data.Void (Void)

type Parser a = Parsec Void String a
```

This states that our `Parser`s will have no special error component, and the input will be a `String`.

To run such a `Parser`, we make use of the `runParser` function, which in simplified form has this type:

```
runParser :: Parser a
          -> String
          -> String
          -> Either (ParseErrorBundle String Void) a
```

The first `String` is the *filename* of the input, which is used for error messages. The second `String` is the actual input. The result is either a special error value, or the result of parsing. Note that in contrast to our `runParser`, no remainder string is returned. The error value can be turned into a human-readable string with the function `errorBundlePretty`.

For example, this is how we would define `parseBExp` using Megaparsec. The rest of the parser code is (for now) completely unchanged:

```
parseBExp :: FilePath -> String -> Either String BExp
parseBExp fname s = case runParser p fname s of
  Left err -> Left $ errorBundlePretty err
  Right x -> Right x
  where
    p = do
      space
      x <- pBExp
      eof
      pure x
```

We are using the `FilePath = String` type synonym in the function type to make it clearer which is the filename and which is the input string.

It mostly works just as before:

```
> parseBExp "input" "x and y and z"
Right (And (And (Var "x") (Var "y")) (Var "z"))
```

But it can now also produce quite nice error messages:

```
> either putStrLn print $ parseBExp "input" "x and y and"
input:1:12:
  |
1 | x and y and
  |            ^
unexpected end of input
expecting "false", "not", or "true"
> either putStrLn print $ parseBExp "input" "x true"
input:1:3:
  |
1 | x true
  |   ^
unexpected 't'
expecting "and", "or", or end of input
```

However, some inputs now produce a rather unexpected error:

```
> either putStrLn print $ parseBExp "input" "truex"
input:1:5:
  |
1 | truex
  |     ^
unexpected 'x'
```

The reason for this is that Megaparsec, for efficiency reasons, does not automatically backtrack when a parser fails. Due to the way we have ordered our `choice` in `pAtom`, we will initially try to parse the literal `true` with `lKeyword` in `pBool`, which will read the input `true`, and then fail due to `notFollowedBy`. However, *the input remains read*, which means Megaparsec's implementation of `choice` won't even try to the other possibilities in `choice`. The way to fix this is to use the `try` combinator, which has this type (specialising to our `Parser` monad):

```
try :: Parser a -> Parser a
```

A parser `try p` behaves like `p`, but ensures the the parser state is unmodified if `p` fails. We must use it whenever a parser can fail *after* successfully consuming input. In this case, we must use it in `lVar` and `lKeyword`:

```haskell
lVar :: Parser String
lVar = lexeme $ try $ do
  v <- some $ satisfy isAlpha
  if False && v `elem` keywords
    then fail "keyword"
    else pure v

lKeyword :: String -> Parser ()
lKeyword s = lexeme $ try $ do
  void $ chunk s
  notFollowedBy $ satisfy isAlpha
```

When to use `try` is certainly rather un-intuitive at first, and remains fairly subtle for ever. One possibility is to *always* use it in the cases we pass to `choice` - this will work, but is inefficient, as it makes every `choice` a potential backtracking point. Most grammars are designed such that backtracking is only needed for the lexical functions.

Applicative parsing

As you may remember, all `Monad` s are also `Applicative` s. For parsing, we can exploit this to write our parsers in a particularly concise form called *applicative style*. The technique inolves to use the `<$>` operator (from `Functor`) and the `<*>` operator (from `Applicative`) to directly intermix the constructors, that put together data, with the parsers that produce it. For example, recall our definition of `many` :

```haskell
many :: Parser a -> Parser [a]
many p =
  choice
    [ do
        x <- p
        xs <- many p
        pure $ x : xs,
      pure []
    ]
```

In the first `choice` case, we are are essentially running the computation `p`, then `many p`, then combining their results with the list constructor. Using applicative style and the prefix form of the list instructor `(:)`, we can instead write `many` as:

```
many :: Parser a -> Parser [a]
many p =
  choice
    [ (:) <$> p <*> many p,
      pure []
    ]
```

Another example is `parseTwoDigits`, which we can rewrite as follows:

```
parseTwoDigits :: Parser (Integer, Integer)
parseTwoDigits = (,) <$> parseDigit <*> parseDigit
```

In fact, we can write many monadic computations in applicative style, but parsing benefits significantly. Two other useful applicative operators are `(<*)` and `(*>)`:

```
(<*) :: Applicative f => f a -> f b -> f a
(*>) :: Applicative f => f a -> f b -> f b
```

They accept two applicative (or monadic) computations as arguments, and then return the value of respectively the first or the second operand, discarding the other. This is quite useful for handling grammars where syntactical constructs are surrounded by keywords, which must be parsed, but do not produce any interesting values. For example, recall our definition of `pBool`:

```
pBool :: Parser Bool
pBool =
  choice
    [ do
        lKeyword "true"
        pure True,
      do
        lKeyword "false"
        pure False
    ]
```

Using applicative style we might write this as:

```
pBool :: Parser Bool
pBool =
  choice
    [ lKeyword "true" *> pure True,
      lKeyword "false" *> pure False
    ]
```

As another example, we might add support for parenthetical grouping like so:

```
parens :: Parser a -> Parser a
parens p = lexeme (chunk "(") *> p <* lexeme (chunk ")")

pAtom :: Parser BExp
pAtom =
  choice
    [ Lit <$> pBool,
      Var <$> lVar,
      do
        lKeyword "not"
        Not <$> pBExp,
      parens pBExp
    ]
```

> **Info**
>
> Applicative style goes beyond merely notational convenience. It is possible to construct parser combinator libraries that are *solely* applicative, and not monadic, which allows the parser to be inspected and transformed in a deeper way, because there is no impenetrable `>>=` operator. Applicative parsers are however fundamentally less powerful than monadic ones - specifically, they can handle only context-free languages.

Parsing Comments

Most grammars allow some form of comment notation that is ignored when parsing, but hopefully provides useful information to human readers. In this section we will extend the parser described above to support classic Unix-style *line comments*, where the character `#` begins a comment that continues to the next linebreak. For example, we want the following string to parse:

```
x   # here goes a comment
and # and
y   # another
    # one
```

It turns out that comments are quite easy to support in a combinator-based parser. Specifically, we simply treat comments as a kind of whitespace and implement them in our `space` combinator. Recall that it currently looks like this:

```
space :: Parser ()
space = do
  _ <- many $ satisfy isSpace
  pure ()
```

First we write a parser that parses a line comment.

```
comment :: Parser ()
comment = void $ do
  void $ chunk "#"
  many $ satisfy (/= '\n')
```

```
> parseTest comment "#foo"
()
> parseTest comment "#foo\n"
()
> parseTest comment "foo"
1:1:
  |
1 | foo
  | ^
unexpected 'f'
expecting '#'
```

As desired, it succeeds only when we parse a comment. Now we modify the `space` parser to parse whitespace, followed by a comment, and *if* we parse a comment, then we recursively invoke `space`. This ensures `space` will onsume arbitrary combinations of spaces and comments.

```
> parseTest (space *> eof) "\n#foo\n#bar\n"
()
```

Because of the principled design of our parser, that is actually all we need to in order to support comments:

```
> putStrLn "x # comment\nand y"
x # comment
and y
λ> parseBExp "" "x # comment\nand y"
Right (And (Var "x") (Var "y"))
```

More Monads

Before we build new monads, it is worth recalling what a monad is. We then show how monad parameterisation and free monads can be used to write code that generates computations once and then executes computations using different concrete monads. Finally, we provide examples of applying free monads to a selection of programming problems, ranging from various implementations of the state monad to memoisation, exception handling and a form of asynchronous programming.

Monads Revisited

Recall that an *abstract* monad is an *abstract data type*. It has the *interface* consisting of the operation signatures

```
-- type M a
fmap   :: (a -> b) -> M a -> M b
pure   :: a -> M a
(>>=)  :: M a -> (a -> M b) -> M b
```

and certain properties these operations must satisfy. The operation `(>>=)` is called *bind*.
We can optionally use the name `return` instead of `pure` in monads, which emphasises that the operation does nothing else but return its argument. We will stick to `pure` in the code in these notes, however.

A generally useful function is *Kleisli composition* `(>=>)`. It is defined using bind by

```
(>=>)  :: (a -> M b) -> (b -> M c) -> (a -> M c)
(f >=> g) x = f x >>= g
```

The monad laws are then most easily expressed in terms of Kleisli composition:

```
(f >=> g) >=> h   =  f >=> (g >=> h)
pure >=> g        =  g
f >=> pure        =  f
```

The laws say that Kleisli composition is associative with `pure` as the neutral element, just like ordinary function composition `.` is associative with `id` as its neutral element.

Another useful function is `join`. It is defined by using `(>>=)` by

```
join :: M (M a) -> M a
join m = m >>= id
```

Recall that once we have defined `pure` and `>>=`, the definition of `fmap` is essentially forced to satisfy

```
fmap f m   =  m >>= pure . f
```

Similarly, `(<*>)` is typically defined using `(>>=)` by

```
mf <*> ma = mf >>= \f -> ma >>= pure . f
```

The upshot is that defining a monad instance in Haskell can be done by using the template below, filling only the `...` in the definitions of `pure` and `>>=`.

```
instance Functor M where
   fmap f m   =  m >>= pure . f

instance Applicative M where
   pure =  ...    -- definition of pure
   mf <*> ma = mf >>= \f -> ma >>= pure . f

instance Monad M where
   m >>= f = ...    -- definition of bind
```

Note that `(>=>)` and `join` are available as monad-parameterised functions in `Control.Monad`.

Subclasses of monads

Subclasses of monads have operations beyond this interface, with additional properties. The abstract state monad `M s` over states of type `s` has operations `get` and `put` with the following types.

```
class (Monad m) => StateMonad m s | m -> s where
  get :: m s
  put :: s -> m ()
```

and properties

```
put s >> get      ==  put s >> pure s      -- put-get
put s >> put s'   ==  put s'               -- put-put
get >>= put       ==  pure ()              -- get-put
```

(The functional dependency `m -> s` says: the state monad `m` determines the type `s` of state it depends on. Since Haskell insists on passing a class instance implicitly to a function---the programmer *must not* write it as an explicit argument---Haskell has to make sure there is a unique instance whenever such an argument
is needed. Using `newtype` declarations and functional dependencies helps in disambiguating when multiple instances might fit the bill. Since instance inference is Haskell-specific trickery rather than a general programming technique we will usually try to stay away from having to use them and getting into the details of how they aid instance inference.)

The properties express that storing a state and then reading it is the same as storing it and returning it; storing a state and then another state is the same as just storing the second state; and reading the state and then storing it again is the same as doing nothing at all.

Note that `StateMonad` is an abstract data type: It does not contain an implementation, but it tells us what is required of any implementation of a state monad--we will see several below--and what a user of *any* implementation can rely on. As such, an abstract data type, including its properties, is a contract. It obliges the implementor of a particular implementation to provide the operations and ensure they satisfy the requisite properties, and it obliges

the user to write code that works with *any* implementation of the abstract data type, not only a particular one.

Abstracting over monads

Suppose we want to write code that uses state, without committing to how the state is implemented.

We can write functions that are parameterised by the state monad class. Here is a counter, and a small stack machine:

```
tick :: (StateMonad m Int) => m Int
tick = do
  n <- get
  put (n + 1)
  pure n



push :: (StateMonad m [a]) => a -> m ()
push x = modify (x :)

pop :: (StateMonad m [a]) => m (Maybe a)
pop = do
  xs <- get
  case xs of
    [] -> pure Nothing
    (y : ys) -> do put ys; pure $ Just y

stackExample :: (StateMonad m [Int]) => m (Maybe Int)
stackExample = do
  push 3
  push 5
  a <- pop
  b <- pop
  pure $ addMaybe a b
  where
    addMaybe mx my = do
      x <- mx
      y <- my
      pure $ x + y
```

The purely functional implementation `State` from chapter 2 is one instance of `StateMonad`.

```haskell
-- A newtype is an isomorphism; record syntax names both directions
of it,
-- the constructor one way and the selector the other.
newtype State s a = State (s -> (a, s))

runState :: s -> State s a -> (a, s)
runState s (State f) = f s



instance Monad (State s) where
  c >>= f = State $ \s ->
    let (a, s') = runState s c
     in runState s' (f a)

-- Haskell requires Functor and Applicative to be DECLARED, but for
a monad
-- they need not be INVENTED: the two definitions below say nothing
about
-- State, and recur verbatim for every monad here.  pure is the one
-- operation (>>=) cannot supply.
instance Applicative (State s) where
  pure a = State $ \s -> (a, s)
  mf <*> ma = mf >>= \f -> fmap f ma

instance Functor (State s) where
  fmap f m = m >>= pure . f

instance StateMonad (State s) s where
  get = State $ \s -> (s, s)
  put s = State $ \_ -> ((), s)
```

We will meet more implementations in the next section. All of them run the *same* `tick` and `stackExample`, with no change to their source text:

```haskell
tickF :: State Int Int
tickF = tick

tickI :: IState Int Int
tickI = tick

tickO :: OState Int Int
tickO = tick

tickFree :: FreeState Int Int
tickFree = tick
```

```
> runState 0 tickF
(0,1)
> runFreeStateF tickFree 0
(0,1)
```

Both return the pair of the value `tick` returned and the final state.

This technique — write against an interface, pick the implementation at the call site — is the first of two ways of decoupling the use of an effect from its interpretation. It is simple and it is what most Haskell libraries do.

Once we pick a state monad instance `m`, the only way of observing an element of `m a` is by *running* it — that is, by passing it to a `run...` function. Nothing happens until something kicks off execution.
The monad operations `pure`, `(>>=)` and any primitive operations producing a computation such as `get` and `put s` only compose computations; they don't execute any of the effects in them.

Consider `tickF` above. Its type is `State Int Int`, a monad instance, and the value is a `newtype` around a function `Int -> (Int, Int)`. The `run...` function is `runState s m`. It executes the entire computation `m` in an initial state `s`.
The pair it returns is everything about `tickF` one can observe. Which operations it performed on the way, and in which order, is opaque.

Free Monads

A *free monad* is a tree of constructors that *name* `pure` and the primitive operations of a monad without binding them to any particular implementation. An element of a free monad is essentially data. We can provide multiple *interpretations* of the named operations. An *interpretation* that maps the constructors to operations in a monad can be automatically extended to mapping entire computations into that monad.
This way we achieve the same effect as monad-abstracted code.

Additionally, free monad elements are *data*, which we can analyse and manipulate at run time by pattern matching on the constructors used to name the primitive operations. A half-run computation is a value that can be stored and resumed, rewriting a computation yields another computation, and an operation can carry a subcomputation as data.

The Free State Monad, Directly

Before giving the general construction of a free monad for any set of primitive operations, let us build one instance of it by hand, for state. The construction is short and natural, and the general construction will then be a matter of separating the steps into what is common and what needs to be done for each set of primitive operations separately.

Consider a state monad, which has primitive operations `get` and `put`. Give each operation its continuation as an extra argument to arrive at their *continuation-passing style* versions:

```
putk :: (StateMonad m s) => s -> (() -> m a) -> m a
putk s = (put s >>=)

getk :: (StateMonad m s) => (s -> m a) -> m a
getk = (get >>=)
```

Both directions relate the continuation-passing forms to the originals:

```
put s = putk s pure          put s >>= m  =  putk s m
get   = getk pure            get   >>= m  =  getk m
```

The left-hand column is the right unit law `m >>= pure == m`, so this uses no property of the implementation beyond the monad laws. The point of the exercise is the right-hand column: in a computation built from `getk` and `putk`, it can be shown that bind is no longer needed. Every occurrence of bind can be transformed into one that has a `get` or `put` or `pure` on the left. In the first two cases we can use `getk` and `putk`, and in the case of `pure` we can eliminate it using the monad law `pure x >>= f  =  f x`. The bind is no longer needed as a separate operation; it has been pushed into the continuation arguments of primitive operations themselves.

Now we make `getk` and `putk` *uninterpreted* named functions, that is, constructors `Getk` and `Putk`. Their types are

```
Return :: a -> m a
Getk   :: (s -> m a) -> m a
Putk   :: s -> m a -> m a
```

where `() -> m a` has been replaced by `m a` since `() -> m a` and `m a` are essentially isomorphic.
We now build a particular `m` as the data type built by these constructors. Let us call it `FSM s`:

```
data FSM s a =
    Return a
  | Getk (s -> FSM s a)
  | Putk s (FSM s a)
```

We now show that `FSM s` can be made into a state monad instance by providing definitions of `pure` and `(>>=)`. `pure` is just the constructor `Return`. Because each primitive operation carries a continuation, `(>>=)` can be defined by composing the continuation attached to a primitive operation with the continuation serving as the second argument of bind using Kleisli composition. Specifically, `Getk k >>= f  =  Getk (k >=> f)`.
Since we have `FSM s a` as the continuation argument type to `Putk` instead of

`() -> FSM s a`, the corresponding definition becomes `Putk s m >>= f = Putk s (m >>= f)`. Finally, for `Pure` we get `Pure x >>= f = f x`.

We arrive at the following definition of `FSM s` as a `StateMonad` instance.

```
instance Monad (FSM s) where
  Return x >>= f = f x
  Getk k >>= f = Getk $ k >=> f
  Putk s m >>= f = Putk s $ m >>= f

instance Functor (FSM s) where
  fmap f m = m >>= pure . f

instance Applicative (FSM s) where
  pure = Return
  mf <*> ma = mf >>= \f -> fmap f ma
```

We can check that the continuation-passing equalities we started from hold by calculation:

```
get >>= f    =  Getk Return >>= f
             =  Getk (\s -> Return s >>= f)
             =  Getk f

put s >>= f  =  Putk s (Return ()) >>= f
             =  Putk s (Return () >>= f)
             =  Putk s (f ())
```

We can now provide a `run` function that interprets the constructor functions as the corresponding functions in `State`.

```
instance StateMonad (FSM s) s where
  get = Getk Return
  put s = Putk s $ Return ()

runFSM :: s -> FSM s a -> (a, s)
runFSM s (Return x) = (x, s)
runFSM s (Getk k) = runFSM s (k s)
runFSM _ (Putk s m) = runFSM s m
```

The point of `FSM s` is that this interpretation is in its `run` function, in contrast to `State`, where it is hardwired into the definitions of `get` and `put`. In particular, after having built an `FSM s a` element we can run it with another and indeed multiple implementations of the primitive operations.

Abstracting the Primitive Operations

`FSM s` is the free state monad. To get the general construction, we separate what is specific to the state monad, the operations `Getk` and `Putk`, from what all monads have in common, `Return`.

To do this we abstract over `FSM s a` by replacing it with a type variable `r` in the constructor arguments of `Getk` and `Putk`. The point is that instantiation of `r` to `FSM s a` is deferred to the point where we add `Return`. We get a data type

```
data StateOp s r = StateGet (s -> r) | StatePut s r
```

so that, as a type isomorphism, we have

```
X  =  a +  StateOp s X
```

where `+` is the sum type constructor. In Haskell this means we want `X` to be a type `Free (StateOp s) a` that is isomorphic to `Pure a | Free (StateOp s (Free (StateOp s) a))`.
Replacing `StateOp s` by any type function `e` yields the definition for the free monad type:

```
data Free e a
  = Pure a
  | Free (e (Free e a))
```

To define `(>>=)` for `Free e a` we need to equip `e` with a way of performing Kleisli composition. Here it is useful that

```
k >=> f  = (>>= f) . k
```

That is, we can take the current continuation `k` of a primitive operation and *functionally* compose it with `(>>= f)`. This is useful since in the definition of `e r` the `r` is a type variable, not the eventual free monad it is instantiated to. It turns out that making `e` a functor by providing an `fmap` is exactly what is needed to make `Free e a` a monad. For `StateOp` we arrive at

```
instance Functor (StateOp s) where
  fmap h (StateGet k) = StateGet $ h . k
  fmap h (StatePut s a) = StatePut s $ h a
```

We are now ready to provide the definition of `Free e a` as a monad:

```
instance Functor e => Functor (Free e) where
  fmap f m = m >>= pure . f                -- standard

instance Functor e => Applicative (Free e) where
  pure = Pure
  mf <*> ma = mf >>= \f -> ma >>= pure . f  -- standard

instance Functor e => Monad (Free e) where
  Pure x >>= f = f x
  Free g >>= f = Free $ fmap (>>= f) g     -- only interesting line
```

Note that the definitions of `fmap` and `(>>=)` are *not* mutually recursive. The occurrence of `fmap` in the definition of `(>>=)` is the `fmap` of functor `e`, not of functor `Free e`.

By inlining the definition of `(>>=)` into `fmap` we arrive at its recursive form, which is commonly seen:

```
instance (Functor e) => Functor (Free e) where
  fmap f (Pure x) = Pure $ f x
  fmap f (Free g) = Free $ fmap (fmap f) g
```

This duplicates the code for `(>>=)` and camouflages that `fmap` can be defined the standard way, though.

Example: The Free Reader Monad

To construct a free monad for a given set of primitive operations we need to provide a data type of constructors corresponding to the operations in continuation-passing style and equip it with `fmap` by systematically composing the continuation argument of each constructor with the argument to `fmap`.

The reader monad has the primitive operation `ask :: Reader r r`. Its continuation-passing form is `askk :: (r -> Reader r a) -> Reader r a`. Abstracting over `Reader r a` in the argument we get the constructor `ReadOp` with argument of type `r -> a`:

```
data ReadOp r a = ReadOp (r -> a)
```

with `fmap` forced to be the composition:

```
instance Functor (ReadOp r) where
  fmap f (ReadOp g) = ReadOp $ f . g
```

We can use this to construct the free reader monad using `Free`:

```
type Reader r a = Free (ReadOp r) a
```

A run function provides an interpretation of the `ReadOp` effect:

```
runReader :: r -> Reader r a -> a
runReader _ (Pure x) = x
runReader r (Free (ReadOp g)) = runReader r (g r)
```

We can now define `ask` to be the constructor `ReadOp` with `pure` as its continuation, as we have seen before:

```haskell
ask :: Reader r r
ask = Free $ ReadOp Pure
```

While it is perhaps not terribly interesting to define other interpretations of the `Reader` monad, it is possible to do so; for example by storing the environment in a global variable or in a database, and defining an interpretation function that runs in `IO` and fetches the environment from there. This is made possible by decoupling the name of an effect from its interpretation.

Instantiating `e` to `StateOp s` recovers the type we built by hand at the start of this section. The correspondence is constructor by constructor:

```haskell
toFree :: FSM s a -> FreeState s a
toFree (Return x) = Pure x
toFree (Getk k) = Free $ StateGet $ toFree . k
toFree (Putk s m) = Free $ StatePut s $ toFree m

fromFree :: FreeState s a -> FSM s a
fromFree (Pure x) = Return x
fromFree (Free (StateGet k)) = Getk $ fromFree . k
fromFree (Free (StatePut s m)) = Putk s $ fromFree m
```

The Recipe for Other Effects

The two steps above — pass the continuation, then abstract the recursive occurrence — work for any operation, not just `get` and `put`. Given a monadic operation

```haskell
op  :: T1 -> ... -> Tn -> M R
```

its continuation-passing version has type

```haskell
opk :: T1 -> ... -> Tn -> (R -> M a) -> M a
```

Abstracting `M a` to a parameter gives the constructor

```
Op  :: T1 -> ... -> Tn -> (R -> a) -> Ops a
```

When `R` is `()`, the continuation `() -> a` collapses to a plain `a`. That is how `Putk` lost its unit argument, and why `StatePut` carries a plain `a` while `StateGet` carries a function.

> **Tip**
>
> The `Functor` instance is then forced: `a` occurs only in the continuation, so `fmap` can only compose onto it. This is why the `Functor` instances in this chapter are so mechanical — they are not making a choice, they are the only lawful thing to write.

Note what the recipe does *not* cover: an operation whose argument is itself a computation, such as `catch` below. We return to that in the section on error handling below. It is the shape that most of the interesting effects turn out to have.

Example: The Free State Monad

We already have `StateOp` from the development at the start of this section. All that remains is to make it a `Functor` — which, by the recipe above, is forced — and to apply `Free` to it. We call the result `FreeState`, to keep it apart from the `State` of chapter 2.

```
instance Functor (StateOp s) where
  fmap h (StateGet k) = StateGet $ h . k
  fmap h (StatePut s a) = StatePut s $ h a
```

Now we can define the monad `FreeState` simply as `Free` applied to the state effects, and make it an instance of the `StateMonad` class from the previous section:

```haskell
type FreeState s a = Free (StateOp s) a


instance StateMonad (Free (StateOp s)) s where
  get = Free $ StateGet Pure
  put s = Free $ StatePut s $ Pure ()
```

Evaluation of a `FreeState` computation is similar to the case for `Reader`, and takes the form of a recursive function that interprets the `StateOp` effects. When we encounter a `StatePut`, we discard the current state and use the provided one.

```haskell
runState :: s -> FreeState s a -> (a, s)
runState s (Pure x) = (x, s)
runState s (Free (StateGet k)) = runState s (k s)
runState _ (Free (StatePut s' m)) = runState s' m
```

Finally, we can define the usual `put` and `get` operations as the corresponding constructors with `pure` as the continuation argument.

```haskell
put :: s -> FreeState s ()
put s = Free $ StatePut s $ Pure ()

get :: FreeState s s
get = Free $ StateGet Pure
```

On top of these, we can define the usual helper functions, such as `modify`, in the way discussed in chapter 2 — or simply reuse the `modify` we already wrote against the `StateMonad` interface, since `FreeState` is an instance of it.

Interpreting into a Monad

`runState` above interprets a `FreeState` computation directly into a Haskell function. Notice, however, how little of it was about *state*: the `Pure` case and

the recursion are the same in every interpreter we have written. Only the treatment of the named primitive operations differs.

We can make that precise. An *interpretation* of an effect functor `e` in a monad `m` is a function

```
h :: forall x. e x -> m x
```

It says what a single operation means, and says nothing whatsoever about sequencing using bind. The `forall x` matters: `h` must work for every result type, so it cannot inspect the continuation — it can only hand it back. (Such a uniform family of functions is called a *natural transformation*.)

From any such `h` we get an interpreter for whole computations mechanically:

```
interpret ::
  (Monad m, Functor e) =>
  (forall x. e x -> m x) ->
  Free e a ->
  m a
interpret _ (Pure x) = pure x
interpret h (Free g) = join $ fmap (interpret h) $ h g
```

The `Free` case is where the `join` presentation of a monad is illustrated. We have `g :: e (Free e a)`. Applying `h` gives `m (Free e a)`: the operation, now performed in `m`, yielding the rest of the computation. Interpreting that rest with `fmap` gives `m (m a)`, and `join` flattens it.

For state, the interpretation of a single operation is three lines:

```
stateOps :: (StateMonad m s) => StateOp s x -> m x
stateOps (StateGet k) = fmap k get
stateOps (StatePut s x) = put s >> pure x

runFreeState :: (StateMonad m s) => FreeState s a -> m a
runFreeState = interpret stateOps
```

and instantiating `m` picks the implementation:

```
runFreeStateF :: FreeState s a -> s -> (a, s)
runFreeStateF m s = runState s (runFreeState m)

runFreeStateI :: FreeState s a -> IORef s -> IO a
runFreeStateI m = runIState (runFreeState m)
```

```
> runFreeStateF stackExampleFree []
(Just 8,[])
> newIORef [] >>= runFreeStateI stackExampleFree >>= print
Just 8
```

Going through `interpret` and then through `State` builds an intermediate closure for every operation. Nothing stops us from writing the composite directly — and in fact we already did, several pages ago. Compare

```
runState :: s -> FreeState s a -> (a, s)
runState s (Pure x) = (x, s)
runState s (Free (StateGet k)) = runState s (k s)
runState _ (Free (StatePut s' m)) = runState s' m
```

with `runFreeStateF` : up to the order of the arguments they are the same function, the second being the first with `interpret` and the `State` instance inlined and the intermediate closures fused away. The hand-written interpreter was never a different technique; it was the general one, specialised.

Being able to write either — the compositional one for clarity, the fused one for speed, and to check them against each other — is one of the practical benefits of having computations as data.

Computations as Data

Interpreting is not the only thing we can do with an element of `Free e a` . It is an ordinary value, so we can also *inspect* it and *rewrite* it.

Inspecting means walking the structure without running it. How far we get depends on the effects. `StatePut s m` carries its continuation as a

computation, so we can walk past it; `StateGet k` carries a *function*, and to get past it we would have to choose an `s` and apply `k` to it — and different choices give different computations. So a static count of the writes a computation performs is available only until the first read:

```
countPuts :: FreeState s a -> Maybe Int
countPuts (Pure _) = Just 0
countPuts (Free (StatePut _ m)) = fmap (1 +) $ countPuts m
countPuts (Free (StateGet _)) = Nothing
```

This is the general limit on inspecting free-monad computations. What you can see depends on how much of the computation is *first-order*. A monadic operation that returns a value of type `T` is represented by a constructor with a continuation of type `T -> M a` as an argument.
Inspecting the computation `M a` requires choosing a value of type `T`.

Rewriting means transforming the operations while leaving the sequencing by bind alone:

```
modifyEffects ::
  (Functor e, Functor h) =>
  (e (Free e a) -> h (Free e a)) ->
  Free e a ->
  Free h a
modifyEffects _ (Pure x) = Pure x
modifyEffects g (Free e) = Free $ fmap (modifyEffects g) $ g e
```

> ⚠ **Warning**
>
> `modifyEffects` walks the *spine* of the computation. If an operation's argument is itself a `Free` value — as `ErrorCatch` will be in the next section, and as most of the interesting effects in assignment 4 are — then the rewriting function must rebuild that argument explicitly. Leave it out and the rewrite does nothing to it.

What Makes `Free` Free

We can now say what "free" means.

`Free e` is the monad generated by `e` in such a way that only those equalities between its elements hold that are forced by the monad laws. Specifically, for every monad `m` and every interpretation `h :: forall x. e x -> m x`, there is exactly one function `interpret h :: Free e a -> m a` that respects `pure` and `(>>=)`. Existence means you never have to invent sequencing behaviour — writing the interpreter for the primitive operations is the *whole* job. Uniqueness means that once you have fixed what the primitive operations do, the meaning of every computation built from them is determined.

We noted that a state monad must satisfy `put s >> put s' = put s'`. The free state monad does not satisfy it because the monad laws do not force the two terms to be equal. The equation only becomes true after interpretation. So `Free (StateOp s)` is *not* a state monad. Keeping the names of primitive operations is what lets us interpret them in multiple ways.

> ⚠ **The cost of freedom**
>
> The representation we use for free monads has a performance trap. In `m >>= f` with `m = Free g`, the bind is pushed *through* the whole of `m` by `fmap`. So a left-nested chain `((m1 >>= f1) >>= f2) >>= ...` re-traverses its prefix at every step, and building an `n`-operation computation that way takes time quadratic in `n`. Right-nested chains — what `do` notation produces from straight-line code — are fine, which is why this rarely bites in practice.
>
> Fixes are known: continuation-passing (Church) encodings, as in `Control.Monad.Free.Church`, make `>>=` constant-time but destroy the very inspectability we get from using constructors; representing the continuation as a type-aligned queue keeps both. Neither is needed for this course.

Implementing an Error Monad

As another example, consider a free monad with error handling, similar to that provided by `Either`. The main thing we will find is that it is easy to define a free effect that is law-abiding and intuitively appears correct, but exhibits unexpected behaviour. We will need to introduce a new language construct to fix it.

We define the `ErrorOp` effect as follows.

```
-- Beware: subtly wrong!
data ErrorOp e a
  = ErrorThrow e
  | ErrorCatch a (e -> a)

instance Functor (ErrorOp e) where
  fmap _ (ErrorThrow e) = ErrorThrow e
  fmap f (ErrorCatch a c) = ErrorCatch (f a) $ \e -> f (c e)

type Error e a = Free (ErrorOp e) a
```

The interpretation function `runError` is a little more sophisticated than the ones we saw before, as some control flow is now required to handle the error cases. However, it is fundamentally very similar to the bind method we have seen previously for the `Either` monad.

```
runError :: Error e a -> Either e a
runError (Pure x) = Right x
runError (Free (ErrorThrow e)) = Left e
runError (Free (ErrorCatch x c)) =
  case runError x of
    Left e -> runError $ c e
    Right x' -> Right x'
```

Finally, we can define the usual boilerplate accessor functions for using the effects:

```
throw :: e -> Error e a
throw e = Free $ ErrorThrow e

catch :: Error e a -> (e -> Error e a) -> Error e a
catch x c = Free $ ErrorCatch x c
```

Although these definitions look right, and the `Functor` instance for `ErrorOp` is law-abiding, our definition of `catch` behaves unexpectedly. The problem is that for `Free`, the `>>=` operator distributes the continuation into every branch of the continuation type, leading to the following equality:

```
(m `catch` h) >>= k   ==   (m >>= k) `catch` (\e -> h e >>= k)
```

Essentially, the continuation `k` ends up running *inside* the handler. If the `catch` succeeds and `k` afterwards throws, the handler catches an exception raised after the `catch` had already finished. With a concrete example:

```
k 1 = throw "E2"
k v = pure v

> runError ((pure 1) `catch` (\_ -> pure 99) >>= k)
Right 99
```

We could expect this to always throw an error, since that is what `k` does, but the exception handler `(\_ -> pure 99)` is also applied to the continuation `k`. The behaviour we have here is not wrong from a type-class perspective, but it is certainly not like most `catch` constructs in real programming languages. It behaves more like installing an exception handler that is dynamically active *for the entire rest of the computation*, which is certainly a feature that may be meaningful, but is not really what we intended.

To fix this, we need to define an `ErrorOp e a` where the "action to perform in case of error" is *not* a continuation as far as `Free` is concerned. This means it must have a different type than `a`, or else we cannot define the `Functor` instance, as that must apply the given function to *every* `a` in order to be lawful.

The solution is to use *existential quantification* to bind a type parameter `x` that is known *only* inside the `ErrorCatch` constructor, and then express the

`ErrorCatch` constructor as containing three values:

- The immediate action to perform, of type `ErrorM e x`.
- The action to perform in case of error, of type `e -> ErrorM e x`.
- The continuation, of type `x -> a`.

In Haskell it is written in this way:

```haskell
data ErrorOp e a
  = ErrorThrow e
  | forall x. ErrorCatch (ErrorM e x) (e -> ErrorM e x) (x -> a)

instance Functor (ErrorOp e) where
  fmap _ (ErrorThrow e) = ErrorThrow e
  fmap f (ErrorCatch m h c) = ErrorCatch m h (f . c)

type ErrorM e a = Free (ErrorOp e) a
```

The type variable `x` occurs only in the arguments of the constructor, not in its result, so it is existentially quantified.

Despite the added complexity of the type, the run function is fairly straightforward:

```haskell
runError :: ErrorM e a -> Either e a
runError (Pure x) = Right x
runError (Free (ErrorThrow e)) = Left e
runError (Free (ErrorCatch m h c)) =
  case runError m of
    Right x -> runError (c x)
    Left err -> runError (h err >>= c)
```

And similarly, the accessor functions are also almost identical:

```haskell
throw :: e -> ErrorM e a
throw e = Free $ ErrorThrow e

catch :: ErrorM e a -> (e -> ErrorM e a) -> ErrorM e a
catch m h = Free $ ErrorCatch m h Pure
```

As a rule of thumb, it is generally a mistake to have more than a single real continuation (of type `a`) in an effect definition. At least we must give careful thought to the consequences when it occurs. This issue occurs for many interesting effects: transactions, retries, timeouts, resource scoping, loops you can break out of. Using existential quantification is a common solution to this problem, although it can also be done by requiring the "intermediate result" `x` to have a *specific* type, rather than be existentially quantified.

The IO Monad

As we have seen in previous chapters, monads are not a special language construct in Haskell. While the `Monad` typeclass is part of the standard library, we could have defined it ourselves if not, and indeed that is how it was originally done. The only special affordance that Haskell provides is syntactic sugar in the form of `do` notation, and while it is certainly very convenient, it does not let us do anything we could not otherwise do. Ultimately, the monads you have seen have merely been convenient and abstract ways of doing things that could also be done in non-monadic Haskell, and they are indeed all ultimately expressed in terms of non-monadic code.

There is one exception, however: the `IO` monad is truly built into the language, and cannot be expressed using normal Haskell. It is the ultimate mechanism by which Haskell programs interact with the surrounding world. This is evident in the type of `main`, the canonical entry point for Haskell programs:

```haskell
main :: IO ()
```

We can imagine that the Haskell runtime system has some kind of interpreter for `IO`, the same way we write interpreters for other monads, but there is no

way to express this interpretation in pure Haskell.

In the assignments and exercises you have worked with, the `main` function is in the `runtests.hs` file, and serves as the entry point to the unit tests. We can also simply type in `IO` actions at the `ghci` prompt, after which `ghci` will execute them.

There are various metaphors for how to understand `IO`. One is that it is a kind of state monad that passes around the entire state of the universe, with functions like `putStr` and `readFile` modifying the state, the same way `put` and `get` modify the state of the `State` monad. This interpretation is useful to an extent, but breaks down when considering concurrency, which we will look at later in the course. Ultimately, it is most useful to simply consider `>>=` for the IO monad as straight-up impure and executing side effects.

Programming with IO

Programming with the `IO` monad in Haskell is very similar to programming in a conventional imperative language, and the same as with programming with any other monad. However, the fact that it is so similar to other languages means that our intuition can sometimes betray us. For example, consider the function `putStrLn`, which prints a given string to stdout, and has the following type:

```
putStrLn :: String -> IO ()
```

Simply putting `putStrLn` somewhere in our program will not cause anything to be printed, even when it is evaluated. For example, evaluating this expression will produce no output:

```
let x = putStrLn "hello world"
in ()
```

There is nothing here you haven't seen before. All this does is create a binding `x` of type `IO ()`, which is not used for anything, and then returns the unit value. In fact, this expression just has type `()` - it is not monadic at all. In order to actually *execute* an effect, we must pass it to `>>=` somehow, putting together

an even larger `IO` operation, which must ultimately be the definition of the program's `main` function:

```haskell
main :: IO ()
main = putStrLn "hello world" >>= \_ -> pure ()

-- or equivalently

main :: IO ()
main = do putStrLn "hello world"
          pure ()
```

The fact that IO operations are normal Haskell values, that just happen to be executable, means we can manipulate them in various ways. As a particularly trivial example:

```haskell
main :: IO ()
main = do let x = putStrLn "hello world"
          x
          x
```

This will print twice, because we are executing the action twice.

Exceptions and IO

We have previously seen how to use monads to encode and propagate errors, such as with `Maybe` and `Either`. The advantage of this approach is that the potential errors are directly visible in the types of functions. However, not all errors are represented, or *can* be represented, in this way. For example, the `div` function has the following type:

```haskell
div :: Integral a => a -> a -> a
```

Yet if we try to divide by zero, we will get an error:

```
> div 1 0
*** Exception: divide by zero
```

This is an *exception*. Despite Haskell being a pure language, it is indeed the case that some ostensibly pure functions, such as `div`, can raise exceptions. One common cause of exceptions is the function `error`, which we often use to indicate program bugs, or `undefined`, which we often use during development. Many prelude functions such as `head` or `tail` are also *partial*, meaning they are not defined on their entire domain. Taking the `head` of an empty list will also raise an exception:

```
> head []
*** Exception: Prelude.head: empty list
```

Exceptions raised by pure code are often called *imprecise exceptions*, because they are not evident in the type.

Today, partial functions are largely considered a bad idea by most Haskell programmers, because they make the types unreliable. Instead `head` should perhaps return a `Maybe` value. Yet even adherents of this approach may hesitate to make functions such as `div` return `Maybe`, due to the sheer amount of boilerplate this would require (even when using monads to propagate the error situation).

Further, other exceptions are harder to avoid: they are also raised for out-of-memory situations or various asynchronous signals. Most functions that perform IO, such as opening or writing to a file, will also use exceptions to report errors. As a result, we need a way to *handle* exceptions.

> **Info**
>
> *Exception* is an overloaded term. In this section we discuss solely the kinds of exceptions that are thrown by functions like `error`. It is common to use the word "exception" to refer to the kind of error handling done with `Either` or similar monads.

Handling Exceptions

Exceptions are undeniably an effect, and while they can be thrown in pure code, it would be a violation of referential transparency to also *handle* them in pure code. As a result, exceptions can only be caught in the IO monad. The facilities for working with exceptions are found in Control.Exception. This is a rather rich and complicated module, and we will not need much of what it provides. The main thing we will need is the `catch` function:

```
catch :: Exception e => IO a -> (e -> IO a) -> IO a
```

The Haskell exception handling machinery is fairly similar to that which you might be familiar with from other languages. The `catch` function takes two arguments. It tries to run the provided `IO` action, and if an exception is thrown during that action, it calls the provided handler function with the exception. The wrinkle is that `catch` is *polymorphic*: since an exception is any value that implements the `Exception` typeclass, any use of `catch` must somehow specify *exactly* which type of exception is caught by this specific `catch`. This may sound unclear, so here is an example where we try to handle a division by zero:

```
handleDivByZero :: IO ()
handleDivByZero =
  print (div 1 0) `catch` (\e -> putStrLn "I am a handler")
```

This will give us a rather long (here abbreviated) error message:

```
    • Ambiguous type variable ‘e0’ arising from a use of ‘catch’
      prevents the constraint ‘(Exception e0)’ from being solved.
      Probable fix: use a type annotation to specify what ‘e0’
should be.
```

The problem is that `catch` can handle *any* exception, so how is Haskell to know which one we mean? We need to put in a type annotation to specify the one we are interested in. We will mainly use the type `SomeException`, which acts as a "root type" for all other kinds of exceptions. In general, we will not discriminate between different types of exceptions, although Haskell provides facilities for doing so. The easiest way to indicate that this is the exception we want to catch is to make the handler a local function with an explicit type ascription:

```
handleDivByZero :: IO ()
handleDivByZero = do
  let handler :: SomeException -> IO ()
      handler e = putStrLn "I am a handler"
  print (div 1 0) `catch` handler


> handleDivByZero
I am a handler
```

We can turn a `SomeException` into a (hopefully) human-readable string by using its `Show` instance:

```
handleDivByZero :: IO ()
handleDivByZero = do
  let handler :: SomeException -> IO ()
      handler e = putStrLn $ "It went wrong: " ++ show e
  print (div 1 0) `catch` handler
```

Example: Safely Reading Files

The Haskell prelude provides the function `readFile`. Given a `FilePath` (a synonym for `String`), it returns the contents of a file:

```
readFile :: FilePath -> IO String
```

If the file cannot be read, it is reported with an exception:

```
> readFile "doesnotexist"
*** Exception: doesnotexist: openFile: does not exist (No such file
or directory)
```

Since the exception handling machinery in Haskell is somewhat cumbersome, we may want to write a wrapper for `readFile` that returns a proper sum type with error conditions instead. Reading a file can go wrong in many ways (perhaps someone cuts a disk cable at an inopportune time), but there are often some common errors we want to handle specially, such as the file not existing. We define a sum type that captures the result of attempting to read a file:

```haskell
data FileContents
  = FileNotFound
  | CouldNotRead String
  | FileContents String
  deriving (Show)
```

The `CouldNotRead` constructor is used as a catch-all for all errors except for file-not-found, and `FileContents` represents success.

When `readFile` fails, it throws an exception of type `IOError`. Using the function `isDoesNotExistError` from `System.IO.Error`, we can detect whether such an `IOError` corresponds to the case where a file of the given name does not exist. We can put all this together into a function for reading a file safely (in this case, "safely" means "does not throw exceptions"):

```haskell
import System.IO.Error (isDoesNotExistError)

readFileSafely :: FilePath -> IO FileContents
readFileSafely f = (FileContents <$> readFile f) `catch` onException
  where
    onException :: IOError -> IO FileContents
    onException e =
      if isDoesNotExistError e
        then pure FileNotFound
        else pure $ CouldNotRead $ show e
```

And observe how well it works:

```
> readFileSafely "doesnotexist"
FileNotFound
> readFileSafely "/root/secrets"
CouldNotRead "/root/secrets: openFile: permission denied (Permission
denied)"
> readFileSafely "ap-exam-solution.hs"
FileContents "module Solution where[...]"
```

Laziness and Exceptions

Haskell's laziness can sometimes make it difficult to handle exceptions in pure code. The reason is that exceptions are not thrown until the associated

computation is forced, which may not be when you expect. For example, we may write code like this, with the intent of replacing a division-by-zero error with an appropriate dummy value:

```
doesNotWork :: IO Int
doesNotWork = do
  let handler :: SomeException -> IO Int
      handler e = do
        putStrLn $ "It went wrong: " ++ show e
        pure 42
  pure (div 1 0) `catch` handler
```

But we receive an unpleasant surprise:

```
> doesNotWork
*** Exception: divide by zero
```

The reason is that the expression `div 1 0` is not actually fully evaluated inside the computation protected by `catch` - instead it is simply returned un-evaluated, and not until `ghci` tries to print the result of the computation (after `catch` is done) will the division actually be attempted and the exception thrown.

One solution is to use the `evaluate` function, also from `Control.Exception`, which has this signature:

```
evaluate :: a -> IO a
```

An expression `evaluate x` is much like `pure x`, but evaluates its argument to *weak head normal form* (*WHNF*) before injecting it into the monad. Intuitively, it will evaluate the provided expression up to the *first* constructor, hopefully uncovering any exceptions immediately. For `Int`, that will be the entire value, but for a list, it will only be up to the first cons cell. However, this is enough to make this simple example work:

```haskell
doesWork :: IO Int
doesWork = do
  let handler :: SomeException -> IO Int
      handler e = do
        putStrLn $ "It went wrong: " ++ show e
        pure 42
  evaluate (div 1 0) `catch` handler
```

```
> doesWork
It went wrong: divide by zero
42
```

IO References

One of the basic facilities provided by the IO monad is a form of mutable references, called `IORef`s, which can be found in the module Data.IORef. The type `IORef a` denotes a mutable reference to a value of type `a`. We can create an `IORef` using the function `newIORef`:

```haskell
newIORef :: a -> IO (IORef a)
```

When creating an `IORef`, we must provide an initial value. Reading and writing is done through the following functions:

```haskell
readIORef :: IORef a -> IO a

writeIORef :: IORef a -> a -> IO ()
```

While other utility functions exist, this interface is all we need in order to interact with `IORef`s.

```
> r <- newIORef True
> readIORef r
True
> writeIORef r False
> readIORef r
False
```

> ⚠ **Warning**
>
> `IORef`s are *not* thread safe. When we discuss concurrent programming later in the course we must be careful not to access them in unstructured ways from multiple concurrent threads. Programming with `IORef`s in Haskell is vulnerable to all the usual tragedies of concurrent programming with mutable state.

Example: Imperative State Monad

We promised earlier that we would provide another implementation of the `StateMonad` interface. We keep the state in a mutable cell of type `IORef ...` and let a computation be a function from that cell to an `IO` action.

```
-- Like State, a newtype whose selector is the true inverse of the
-- constructor: the caller supplies the reference.
newtype IState s a = IState {runIState :: IORef s -> IO a}

-- Convenience: create the reference from an initial state, then
run.
runIStateFrom :: s -> IState s a -> IO a
runIStateFrom s c = do
  ref <- newIORef s
  runIState c ref
```

The instance definitions are unremarkable:

```haskell
instance Monad (IState s) where
  c >>= f = IState $ \ref -> do
    a <- runIState c ref
    runIState (f a) ref

instance Applicative (IState s) where
  pure a = IState $ \_ -> pure a
  mf <*> ma = mf >>= \f -> fmap f ma

instance Functor (IState s) where
  fmap f m = m >>= pure . f

instance StateMonad (IState s) s where
  get = IState readIORef
  put s = IState $ \ref -> writeIORef ref s
```

This is the implementation an imperative programmer would have written first, and it is genuinely different from `State`: it mutates one cell rather than threading a value. Yet it satisfies the same laws, and

```haskell
tickI :: IState Int Int
tickI = tick
```

is the same `tick` we ran with `State`, at a different type.

> **Note**
>
> `IState` is not more efficient than `State` in any deep sense — GHC is perfectly good at compiling state threading — but it is the shape you are forced into when the state lives outside the program: a file, a database, a device.

Example: Object-based State Monad

`IState` passes the reference holding the state to every operation. It is worth asking whether the state could be kept as the private field of an object that has

`get` and `put` as its methods. It can, and the result is a fourth implementation of the `StateMonad` interface.

An object is a record of operations closed over a cell that nothing else can reach:

```
data StateObj s = StateObj
  { objGet :: IO s,
    objPut :: s -> IO ()
  }

newStateObj :: s -> IO (StateObj s)
newStateObj s0 = do
  ref <- newIORef s0
  pure $ StateObj {objGet = readIORef ref, objPut = writeIORef ref}
```

`newStateObj` creates the cell and captures it in the two closures. After that, nothing passes the cell to `objGet` or `objPut` — they already have it.

A computation is then a function from such an object to an `IO` action, and running one allocates a fresh object:

```
newtype OState s a = OState {runOStateOn :: StateObj s -> IO a}

runOState :: OState s a -> s -> IO a
runOState m s0 = do
  o <- newStateObj s0
  runOStateOn m o
```

```haskell
instance Monad (OState s) where
  OState g >>= f = OState $ \o -> do
    a <- g o
    runOStateOn (f a) o

instance Functor (OState s) where
  fmap f m = m >>= pure . f

instance Applicative (OState s) where
  -- The object is discarded: pure performs no side effects.
  pure a = OState $ \_ -> pure a
  mf <*> ma = mf >>= \f -> fmap f ma

instance StateMonad (OState s) s where
  get = OState objGet
  put s = OState $ \o -> objPut o s
```

Reading the three definitions in order, `pure a` *ignores* the object, which is exactly why it performs no side effects; `m >>= f` hands the *same* object to both halves, which is why the second sees what the first did. `get` and `put` are the object's own methods, lifted into the monad. As always, `fmap` and `<*>` follow from `>>=`.

`OState` satisfies the same three laws as the others, and the same `tick` runs over it:

```
> runOState tick 0
0
> runOState (tick >> tick >> tick) 0
2
```

This is `IState`, with the reference abstracted

Put the two side by side:

```
newtype IState s a = IState (IORef s -> IO a)
newtype OState s a = OState (StateObj s -> IO a)
```

They have the same shape, and the reference is still threaded in both — it has merely moved from an explicit argument into the object you must be holding in order to call a method at all. That is not a defect of the encoding. It is what makes two runs independent: `runOState` allocates a fresh object each time, so the only thing distinguishing two runs is *which* object they were given.

Making the reference genuinely global — a top-level `IORef` created with `unsafePerformIO` — does remove the argument, and removes that independence with it: one cell for the whole program, no way to scope or reset it, and, by the functional dependency on `StateMonad`, exactly one state type for the monad `IO` forever after.

The gain over `IState` is that `IORef s` is a concrete representation while `StateObj s` is an interface, so `OState` accepts implementations that `IState` cannot express. Any function on objects gives one; here is a wrapper that reports every operation:

```
tracing :: (Show s) => String -> StateObj s -> StateObj s
tracing name o =
  StateObj
    { objGet = do
        s <- objGet o
        putStrLn (name ++ ": get -> " ++ show s)
        pure s,
      objPut = \s -> do
        putStrLn (name ++ ": put " ++ show s)
        objPut o s
    }
```

```
> o <- newStateObj (0 :: Int)
> runOStateOn (tick >> tick) (tracing "counter" o)
counter: get -> 0
counter: put 1
counter: get -> 1
counter: put 2
1
```

The same `tick`, the same monad, a different object — and state in a file, in an `MVar`, or behind a network call would be three more, none of them requiring a change to `tick` or to `OState`. This is the same separation the free monad buys, obtained by abstracting the *representation* rather than the *program*, and it has the same limit as the rest of this section: you can vary what `get` and `put` do, but a computation is still a function, so there is nothing to inspect, rewrite or suspend.

Free Monads with IO

The examples of free monads we saw above are perhaps a bit contrived, as they merely involved replicating existing monads. In practice, we often use free monads to abstract over complicated effects, typically those in IO. Let us look at some use cases. Given the constraints of these notes, they will still be somewhat contrived (we can't fit an actual production system here), but they will be more interesting than spelling `State` in a new way.

One Computation, Four Implementations

Before moving on, it is worth collecting what we now have. The single definition

```haskell
tick :: (StateMonad m Int) => m Int
tick = do
  n <- get
  put (n + 1)
  pure n
```

mentions no implementation of state at all, and we can run it five ways:

```
> runState 0 tickF                  -- pure state threading
(0,1)
> newIORef 0 >>= runIState tickI    -- a mutable IORef
0
> runOState tickO 0                 -- an object holding the
reference
0
> runFreeStateF tickFree 0          -- free monad, interpreted purely
(0,1)
> newIORef 0 >>= runFreeStateI tickFree   -- free, interpreted in IO
0
```

The first three use the type-class method: `tick` is a polymorphic value, and choosing its type chooses the implementation. The last two use the free monad: `tickFree` is a concrete *data structure*, and choosing the interpreter chooses the implementation.

The difference between the two methods only becomes visible when we want to do something with the computation other than run it. `tickF` is a function: the only thing to be done with it is to pass it to `runState`, and the pair that comes back is everything observable — which operations ran, and in what order, cannot be recovered from it. `tickFree` is a value you can pattern match on, store in a list, count the operations of, interpret twice and compare, or — as we are about to do — interpret in a way that *no* implementation of the `StateMonad` interface could have provided, because it does something the interface never mentions.

An Uncontrived Real World Example

To start out with, let us consider one of the most interesting and useful functions, the recursive Fibonacci function:

```haskell
fib :: Int -> Int
fib 0 = 1
fib 1 = 1
fib n = fib (n - 1) + fib (n - 2)
```

One common requirement when writing software is *logging*, yet we do not wish every function to depend on some specific implementation of a logging system. In particular, logging typically requires IO, and we don't want every single function to live in the `IO` monad. Free monads are a handy way to abstract out the notion of logging. Let us define a type `FibOp` that encapsulates the effects that we need in our `fib` function, currently restricted to merely logging.

```
data FibOp a = FibLog String a

instance Functor FibOp where
  fmap f (FibLog s c) = FibLog s $ f c
```

Now we can define a `FibM` monad that supports `FibOp` effects, with a function `fibLog` that performs a single logging step:

```
type FibM a = Free FibOp a

fibLog :: String -> FibM ()
fibLog s = Free $ FibLog s $ Pure ()
```

Finally we use it in a monadic definition of `fib`:

```
fib :: Int -> FibM Int
fib 0 = pure 1
fib 1 = pure 1
fib n = do
  fibLog ("fib(" ++ show n ++ ")")
  x <- fib (n - 1)
  y <- fib (n - 2)
  pure $ x + y
```

One of the interesting parts of the `FibM` monad is that there are many legitimate and interesting ways to interpret it (in contrast to `Reader` or `State`, which have only one sensible interpretation). One obvious one is to interpret it in the `IO` monad, where the logging messages are printed as lines:

```haskell
ioFibM :: FibM a -> IO a
ioFibM (Pure x) = pure x
ioFibM (Free (FibLog s x)) = do
  putStrLn s
  ioFibM x
```

Example use:

```
> ioFibM $ fib 5
fib(5)
fib(4)
fib(3)
fib(2)
fib(2)
fib(3)
fib(2)
8
```

In a real application, we might log to the file system or some dedicated logging daemon (and probably, we would be computing something more interesting than Fibonacci numbers), but this is decoupled from the *users* of the `fibLog` effect.

But another useful interpretation function is one that just discards the logging messages - and is pure:

```haskell
pureFibM :: FibM a -> a
pureFibM (Pure x) = x
pureFibM (Free (FibLog _ c)) = pureFibM c
```

```
> pureFibM $ fib 5
8
```

And yet another useful interpretation is a pure one that accumulates the log messages in a list.

```haskell
logFibM :: FibM a -> (a, [String])
logFibM (Pure x) = (x, [])
logFibM (Free (FibLog s c)) =
  let (x', msgs) = logFibM c
   in (x', s : msgs)
```

```
> logFibM $ fib 5
(8,["fib(5)","fib(4)","fib(3)","fib(2)","fib(2)","fib(3)","fib(2)"])
```

Note that the list is in the same order as the lines `ioFibM` printed. That is not an accident we should be relaxed about: two interpreters of the same computation disagreeing about the order of the effects they both report would be a bug in one of them, and it is precisely the kind of bug that having two interpreters makes easy to catch.

It is easy to imagine how this could be useful for testing the pure logic for logging, without actually interacting with a complicated logging infrastructure.

> ⚠ **Warning**
>
> Be careful to prepend with `s : msgs` rather than append with `msgs ++ [s]`. Appending would both reverse the order and make logging cost quadratic time in the number of messages. If you do need to build a list from the far end, either accumulate in reverse and `reverse` once at the end, or use a data structure with cheap appending. This is orthogonal to the issue of free monads, but it is an easy mistake to make in an interpreter.

Adding Another Effect

Above we saw how we could interpret the same effectful function (`fib`) in three different ways, without modifying `fib` at all. Let us now add another effect. One problem with the recursive Fibonacci function is that it is very slow, as it redundantly recomputes the same recursive invocations over and over again. If you execute `pureFibM $ fib 25` at the `ghci` prompt, you will likely wait several seconds before you get a response.

One way to improve the performance of recursive computations with many shared subresults is *memoisation*, where we maintain a cache mapping function arguments to results. Then, whenever we encounter an argument we have seen before, we merely retrieve the result that was computed last time.

Memoisation is notoriously inconvenient to implement in pure languages, because of the need to maintain a state. The idea behind memoisation is that the effect of the cache is not *observable*, but merely speeds up the computation, but Haskell does not know that. Instead, we have to manually manage the cache of previous results, which raises additional questions, such as when to expire cache entries in order to avoid space leaks. This is a rather complex design space, and intermingling memoisation logic with algorithmic logic is likely to result in a mess.

Instead, let us augment the `FibM` monad to handle memoisation. First, we add a new kind of effect to `FibOp`, namely `FibMemo`:

```
data FibOp a
  = FibLog String a
  | FibMemo Int (FibM Int) (Int -> a)
```

The `FibMemo` constructor has three components:

1. An integer `n` denoting that this effect refers to the result of computing `fib n`.
2. A `FibM Int` computation that computes `fib(n)` if executed.
3. A continuation `Int -> a` that should be invoked with the result of the computation stored in the `FibM Int` - or a memoised version if available.

The instance definition for `FibOp` as a functor is straightforward, resembling those we have seen before.

```haskell
instance Functor FibOp where
  fmap f (FibLog s c)    = FibLog s $ f c
  fmap f (FibMemo n m c) = FibMemo n m $ f . c

fibMemo :: Int -> FibM Int -> FibM Int
fibMemo n m = Free $ FibMemo n m Pure
```

The idea behind `fibMemo` is that we use it to wrap a computation.

```haskell
fib :: Int -> FibM Int
fib 0 = pure 1
fib 1 = pure 1
fib n = fibMemo n $ do
  fibLog ("fib(" ++ show n ++ ")")
  x <- fib (n - 1)
  y <- fib (n - 2)
  pure $ x + y
```

The operational idea is that whenever the `fib n` case is reached, we want to check whether an existing result for `n` has already been computed. If so, we return it. If not, we compute the result using the provided computation. Of course, whether that is *actually* what happens depends on how we write our interpreter function for `FibM`. For example, we can add support for `FibMemo` to `pureFibM` in a way that does not actually perform any memoisation:

```haskell
pureFibM :: FibM a -> a
pureFibM (Pure x) = x
pureFibM (Free (FibLog _ c)) = pureFibM c
pureFibM (Free (FibMemo _ fn c)) = pureFibM $ c $ pureFibM fn
```

But of course, that rather defeats the purpose of `FibMemo`. Instead, we can write an interpretation function `memoFibM` that carries around a cache of type `[(Int,Int)]`, in which memoisation results are stored. We implement this by using a local helper function that accepts and returns the state, and then we discard the final state at the end.

```haskell
memoFibM :: FibM a -> a
memoFibM m = fst $ run [] m
  where
    run :: [(Int, Int)] -> FibM b -> (b, [(Int, Int)])
    run cache (Pure x) = (x, cache)
    run cache (Free (FibLog _ c)) = run cache c
    run cache (Free (FibMemo n fn c)) =
      case lookup n cache of
        Just x -> run cache $ c x
        Nothing ->
          let (x, cache') = run cache fn
           in run ((n, x) : cache') (c x)
```

In the `FibMemo` case, we check whether a result for `n` is already known, and if so, we return it. Otherwise we compute it by executing `fn`. Note that `run` could also be implemented using the `State` monad, as the way we handle the cache is identical to how `State` handles state.

Now even large applications of `fib` finish almost instantaneously, as memoisation transforms the exponential number of recursive calls into a linear number of recursive calls:

```
> memoFibM $ fib 1000
9079565065540428013
```

Although `memoFibM` is a rather simplistic way of handling memoisation, in that the cache is not persistent across multiple invocations, it is not difficult to conceive of a variant that uses `IO` to store memoisation results in a database or on the file system, perhaps with limitations on the maximum size of stored results, or some expiry policy that removes entries after a time. All of these changes can be made without modifying `fib` itself.

Another useful change is to allow memoisation of arguments and results that are not exclusively of type `Int` (or some other fixed type), or to use a more efficient data structure than a list to store the cache. This is not particularly difficult, although somewhat more verbose, and so we have left it out of the exposition here.

Asynchronous Programming with Free Monads

Asynchronous programming styles have become common in languages targeted towards network programming, often supported by language features such as async/await facilities. The purpose of such language features is to *hide* the somewhat contorted control flow otherwise required by asynchronous programming. For example, a program often needs to *wait* for an event to happen. Operationally, this happens by suspending the current computation by recording its state in a data structure somewhere. Whenever an event comes in, it is checked whether any suspended computations depend on it, and if so, they are resumed until the next time they need to be suspended. However, exposing this directly leads to a very awkward programming experience. Instead, we would like to have a seemingly normal function that *blocks* until the requested event arrives - but of course without suspending the entire system. We shall now see how this can be accomplished using a design based on free monads.

We start by defining a *very* simple model of events. An event is a name paired with a value, and the value is always an integer.

```
type EventName = String

type EventValue = Int

type Event = (EventName, EventValue)
```

Events come from the outside world, in unpredictable order and with uncertain timing, and are in practice often the result of communication with other programs. For testing purposes, we can model sequences of events as Haskell lists, but baking such an assumption into our systems would make them useless in practice. At any given time, many computations may be suspended waiting for events to happen; some of them perhaps waiting for the same events.

To support this style of programming, we define an effect type for our asynchronous programming model, with support for two effects:

1. Waiting for an event of a given name.

2. Logging a message. This is solely so we can observe execution of our asynchronous programs through their side effects (printing to the console), and is in principle unrelated to events.

The definition, along with its `Functor` instance, is as follows:

```
data EventOp a
  = WaitFor EventName (EventValue -> a)
  | LogMsg String a

instance Functor EventOp where
  fmap f (WaitFor s c) = WaitFor s $ f . c
  fmap f (LogMsg s c)  = LogMsg s $ f c

type EventM a = Free EventOp a
```

And it is all packed together under the name `EventM` with two functions `waitFor` and `logMsg`:

```
waitFor :: EventName -> EventM EventValue
waitFor s = Free (WaitFor s Pure)

logMsg :: String -> EventM ()
logMsg s = Free $ LogMsg s $ Pure ()
```

Here are three examples of how to use the monad. All of these functions listen for events and do something fairly simple with the result. The purpose of the `divider` example is solely to illustrate that control flow and looping are possible.

```
adder, multiplier :: EventM ()
adder = do
  logMsg "starting adder"
  x <- waitFor "add"; y <- waitFor "add"
  logMsg $ unwords [show x, "+", show y, "=", show $ x+y]
multiplier = do
  logMsg "starting multiplier"
  x <- waitFor "mul"; y <- waitFor "mul"
  logMsg $ unwords [show x, "*", show y, "=", show $ x*y]

divider :: EventM ()
divider = do
  logMsg "starting divider"
  x <- waitFor "div"
  y <- waitForDivisor
  logMsg $ unwords [show x, "/", show y, "=", show $ div x y]
  where
    waitForDivisor = do
      y <- waitFor "div"
      if y == 0
        then do
          logMsg "Cannot divide by zero"
          waitForDivisor
        else pure y
```

A definition such as `adder` represents a *process*. It runs as far as possible until the value of an event is needed, at which point it is suspended. We can write an interpretation function that does just that; evaluating as many of the effects as possible until reaching a `WaitFor`:

```
stepUntilWait :: EventM a -> IO (EventM a)
stepUntilWait (Pure x) = pure $ Pure x
stepUntilWait (Free (LogMsg s c)) = do
  putStrLn s
  stepUntilWait c
stepUntilWait w@(Free (WaitFor _ _)) = pure w
```

It is possible to use `stepUntilWait` directly in `ghci`, but the result is not terribly interesting:

```
> a <- stepUntilWait adder
starting adder
```

Now we have a name `a` representing a suspended execution.

```
:t a
a :: EventM ()
```

Unfortunately we cannot inspect its *structure*, because `EventOp` is not an instance of `Show`, but we can be sure it is stuck on a `WaitFor` effect.

At some point, an event may arrive. We can then check whether the event name matches what the suspended execution is waiting for, and if so, call the continuation with the value. If the event name does not match, we do nothing. We can encapsulate this in a function:

```
deliver :: Event -> EventM () -> IO (EventM ())
deliver (name, val) (Free (WaitFor wanted c))
  | wanted == name = stepUntilWait (c val)
deliver _ p = pure p
```

After invoking the continuation, we use `stepUntilWait` to evaluate any subsequent non-`WaitFor` effects, but we do *not* recursively call `deliver`. This is because events are distinguishable: we want the `adder` process to process two *distinct* events, not have the same event with name `"add"` provide values for both `waitFor`s.

Here is how we can use `deliver`, continuing execution of the `a` above:

```
> b <- deliver ("add", 1) a
> c <- deliver ("add", 2) b
1 + 2 = 3
```

Note how execution returns to our control after every invocation of `deliver`. This allows us to use arbitrary logic to retrieve events (such as reading them from the network or a file), without the process definitions (`adder`, `multiplier`, `divider`) having to care about the details.

This is often called an *event pump*, by analogy to old-fashioned water pumps. We continue cranking the handle (calling `deliver`), which lets the process continue through its execution. Note that the suspended computations, the `a`, `b`, and `c` values above, are ordinary Haskell values, which we can manipulate like any other Haskell values. One potentially dubious thing we can do is to keep reusing the same suspended computation multiple times:

```
> deliver ("add", 2) b
1 + 2 = 3
> deliver ("add", 2) b
1 + 2 = 3
> deliver ("add", 2) b
1 + 2 = 3
```

A useful extension is keeping *multiple* suspended processes in a list. Whenever an event arrives, we deliver it to each of them, where every process that is *not* waiting for this particular event just ignores it. So the whole pump is a recursive function over the list of events:

```
runEventM :: [EventM ()] -> [Event] -> IO [EventM ()]
runEventM ps [] = mapM stepUntilWait ps
runEventM ps (e : es) = do
  ps' <- mapM stepUntilWait ps
  ps'' <- mapM (deliver e) ps'
  runEventM ps'' es
```

The first `mapM stepUntilWait` gets every process started, up to its first `WaitFor`; after that, `deliver` does the stepping itself.

This lets us have interleaved execution of asynchronous operations - a programming technique that in most languages is either hopelessly complicated, or requires direct runtime support.

```
> runEventM [adder, multiplier, divider]
            [("add", 1),
             ("mul", 2),
             ("div", 3),
             ("add", 4),
             ("div", 0),
             ("mul", 5),
             ("div", 6)]
starting adder
starting multiplier
starting divider
1 + 4 = 5
Cannot divide by zero
2 * 5 = 10
3 / 6 = 0
```

As an example that truly demonstrates how decoupled the processes are from how events are read, the following function reads events interactively from the console, through the `readLn` function, which reads a value in Haskell syntax.

```haskell
interactivelyRunEventM :: [EventM ()] -> IO ()
interactivelyRunEventM ps = do
  ps' <- mapM stepUntilWait ps
  case filter running ps' of
    [] -> pure ()
    qs -> do
      event <- readLn
      qs' <- mapM (deliver event) qs
      interactivelyRunEventM qs'
  where
    running (Pure ()) = False
    running _ = True
```

Here is an example of using it, where the output from the processes is intermixed with typed input:

```
> interactivelyRunEventM [adder, multiplier, divider]
starting adder
starting multiplier
starting divider
("add", 1)
("add", 2)
1 + 2 = 3
("div", 3)
("mul", 2)
("mul", 4)
2 * 4 = 8
("div", -1)
3 / -1 = -3
```

Property-Based Testing

This chapter is about how we convince ourselves - and others - that a program does what it is supposed to do. The technique we focus on is *property-based testing*: we state properties that the code is expected to satisfy, and then let a machine try very hard to find inputs that violate them. In Haskell the standard tool for this is QuickCheck, which is also the origin of the idea; property-based testing libraries in other languages (Hypothesis for Python, PropEr for Erlang, jqwik for Java, proptest for Rust, ...) are all descendants of it.

Before we get to the mechanics we spend some time on the vocabulary, because much confusion about testing is really confusion about what a test *is*. We then work through properties, generators, shrinking, and test data design, and close with the most valuable industrial application of the technique: testing *stateful* systems against a *model*.

Specification, implementation, verification

Programming consists of three activities that are important to keep apart:

1. **Specification**: *What* is computed? Which functions exist, and which properties do they have?
2. **Implementation** (or *coding*): *How* is it computed? The source code.
3. **Verification**: Which explicit *evidence* is there that the code meets its specification?

Verification always requires *both* code and specification. The same code may be correct with respect to one specification and incorrect with respect to another. This has an immediate practical consequence:

> ⚠ **Avoid saying: my code works**
>
> With respect to *which specification*? Code is trivially correct with respect to whatever it happens to do - there is nothing to test, because it does what it does. The interesting question is what it does that a *user of the code* is entitled to rely on, and whether you have evidence that it always does so, for all valid inputs, and not merely on the handful of inputs you happened to try.

Specifications

A *specification* describes what a collection of functions does, not how it is implemented. The litmus test is that it contains *everything* a user of a module needs to know, and *nothing else*.

Specifications are not automatically perfect. They may be

- *inconsistent*: they contain a contradiction, so no implementation can satisfy them; or
- *incomplete*: they leave room for several implementations with observably different behaviour. This is very common, and often deliberate.

It is useful to distinguish a specification from the *requirements*, which are an informal description of the desired functionality. Requirements are typically vague and open to interpretation; a specification *resolves and documents* that ambiguity. In practice a full specification consists of a formal part (properties that must hold) plus an informal natural-language part.

An interface is a contract

A module interface specification is a *contract* between the programmer who *uses* a module and the programmer who *implements* it:

- User to implementor: "Everything I need to know must be in the module specification. How I use your implementation is none of your business.

Don't make me read your code."
- Implementor to user: "Everything you need to know is in the module specification. How I implement it is none of your business. Don't read my code - I may change it at any time."

Note that *user* and *implementor* are roles; they are frequently the same person on different days.

What happens if specification and implementation are inconsistent, as shown by a failing test? There are three possible repairs:

- Change the code to meet the specification. Requires no negotiation with users.
- Change the specification to meet the code. Requires negotiation, because the change may propagate into code that users have already written.
- Change both. This is a natural part of *exploratory* programming, while the requirements are still being worked out.

The most common problem in practice is that modules are *underspecified*: there is too little information for a user. So the user experiments with the implementation and comes to rely on behaviour that was never specified. The implementor later changes that behaviour, and the change manifests itself as errors in the user's code that had not been observed before.

Localising the cause of such an error is not merely difficult: attributing it to either the implementation or the use of a module is *ill-defined* without a clear module specification. The implementor may have one interface in mind, which may be as useless as "whatever my code does in any given version, user beware", and the user may have another, which may be as useless as "whatever a previous version of the module did, or whatever I think it should do when my code uses it".

What testing is - and is not

Testing is partial verification by

- executing code on inputs and checking whether the result is evidence of a specification violation; and

- *systematically* constructing those inputs so as to *maximise the likelihood of demonstrating a violation*.

It is worth separating three activities that are often all called "testing":

- **Formal verification**: a mathematical proof that there exists *no* valid input leading to a specification violation.
- **Testing**: the systematic discipline of finding "nasty" inputs that maximise the chance of finding a violation *quickly*. Failing to break the code is taken as *evidence* - not proof - that the code may be correct with respect to the given specification.
- **Trying out**, *illustrating*, *exemplifying*: running the code on some inputs to show that it does something useful. Perfectly respectable, but it is not testing.

> **Note**
>
> Do not use the word "test" unless you have *both* a specification and code, and you are *systematically* looking for violations of the former by the latter. Say "illustrate", "exemplify" or "try out" instead.

Finally, a note on terminology for the different scopes of testing.

Unit testing is testing a unit of code against its specification. Here a *unit* typically corresponds to a module with a number of related types and functions. In unit testing the unit's dependencies are stubbed out. This is to ensure that a specification violation observed during unit testing can be attributed to the unit itself, and not to an error in the code it depends on. Suppose, for example, that code in the unit calls a function of an auxiliary module `M` in a way that violates that function's precondition, and that this *does not* lead to an observable error. It *is* an error in the unit nonetheless, and a stubbed implementation of the function should flag it during testing. Why is it an error in the unit? Because the function's *specification* promises correct behaviour only when the precondition is satisfied. On inputs that violate it the function may do anything at all---and the implementor may change what it does at will between versions.

Integration testing tests a composition of modules against the specification of the resulting *component*. The most common and easiest form is *bottom-up integration testing*: testing one or more modules together with all the modules they depend on. This essentially treats the entire component as a single (big) "unit".

System testing tests a deployable system against its system specification. It essentially treats the entire system as a single (huge) "unit" of code and data.

Finally, *acceptance testing* tests a deployed system in its target execution environment against business and user requirements, including usability and integration into the work flow.

This chapter is primarily about property-based testing of small units, and model-based testing of larger (sub)systems.

Properties

A *logical statement* is a statement that is either true or false. A *property* of one or more functions is a logical statement involving those functions that is supposed to be true. Most properties we care about are *universally quantified*:

$$\forall x, y, \ldots, , Q(x, y, \ldots)$$

or, when the property only applies to some inputs,

$$\forall x, y, \ldots, , P(x, y, \ldots) \Rightarrow Q(x, y, \ldots).$$

A *partial formal specification* is typically a conjunction of such properties.

When we discussed type classes in Chapter 1 we met the concept of *laws*, which are properties that must hold for instances of a given type class. For instance, if a type `T` is an instance of `Eq` we expect `x == x` to evaluate to `True` for every value `x :: T`.

Properties are not, however, intrinsically linked to type classes. An example of a property not related to any type class is the interaction between `length` and

`(++)`. This holds in general, but for concreteness consider lists of integers. For any two lists `xs :: [Integer]` and `ys :: [Integer]` we have

```
length (xs ++ ys) = length xs + length ys
```

Another way of stating the same thing is that the function

```
prop_lengthAppend :: [Integer] -> [Integer] -> Bool
prop_lengthAppend xs ys = length (xs ++ ys) == length xs + length ys
```

returns `True` for all possible arguments. We could come up with a number of test cases, e.g.

```
tediousTestCases :: [([Integer], [Integer])]
tediousTestCases = [([], []), ([0], [1, 2]), ([3, 4, 5], [])] --
etc.
```

and test the universal statement with `all (\(xs, ys) -> prop_lengthAppend xs ys) tediousTestCases`. This simple-minded brute-force method does not work when the input domain is infinite (here, all pairs of lists) or merely very large (for example, all 64-bit floating-point numbers).

Property-based testing is a *specification-driven* testing method. Its point of departure is a conjunction of *properties* that the code must satisfy. A property usually has the form of a universally quantified statement

$$\forall x_1, \ldots, x_k, ., P(x_1, \ldots, x_k) \Rightarrow Q(x_1, \ldots, x_k).$$

By inspection of the property, a test engineer then constructs an *enumeration* of values $v_0, v_1, \ldots$, where $v_i = (v_{i1}, \ldots, v_{ik})$, such that

1. $P(v_i)$ is guaranteed to hold;
2. if there exists a v with $P(v)$ and $\neg Q(v)$, then at least one such v occurs in the enumeration, and as early as possible;
3. the v_i are efficient to construct.

These objectives can be combined into one: enumerate candidate values that can falsify the property efficiently, and enumerate them in decreasing order of the likelihood that they do so.

QuickCheck is a programming framework that provides functions for constructing such enumerations, primarily by generating pseudo-random numbers and mapping these to values of the desired type. The workhorse is `quickCheck`. It accepts what QuickCheck calls a *property*: a logical property like the one above, wired together with a *specific enumeration* of the v_i as a lazy stream. Running `quickCheck prop_lengthAppend` evaluates the underlying logical property on the first 100 elements of the enumeration that the property comes with. If we want a different number we can say so explicitly, e.g.

```
quickCheck $ withMaxSuccess 10000 prop_lengthAppend
```

Tuning the number of tests allows us to trade testing time for certainty: more test data increases the chance of finding a specification violation. For an infinite domain, however, no number of tests can prove the *absence* of a violation, and hence none can establish correctness of the code with respect to the property.

Counterexamples

Consider another property, stating that `(++)` is commutative:

```
prop_appendCommutative :: [Integer] -> [Integer] -> Bool
prop_appendCommutative xs ys = xs ++ ys == ys ++ xs
```

Running `quickCheck prop_appendCommutative` quickly falsifies this theory and prints out a counterexample such as `[0]` and `[1]`. QuickCheck is very useful when we are genuinely unsure whether a property holds, since in practice false properties often have easy-to-find counterexamples.

The value of properties

Why do we care whether a property like `prop_lengthAppend` holds? It does, after all, not directly say anything about the correctness of `length`, `(+)` or `(++)`. For instance, given the obviously wrong definitions

```
_ + _ = 0
_ ++ _ = []
```

the property would still hold. The crucial---and, once stated, obvious---observation is that the more of the requirements we capture as *testable properties*, the easier it becomes to demonstrate that a piece of candidate code is buggy.

Where properties come from

Coming up with testable properties is the genuinely difficult part of property-based testing, and it is a skill that improves with practice. The following catalogue of recurring patterns is a good place to start when staring at a fresh module.

Algebraic laws. The type class laws of Chapter 1 and the monad laws of Chapter 2 are properties, and so are associativity, commutativity, idempotence, distributivity, and the presence of neutral or absorbing elements for whichever operations your module provides.

Invariants. Many data structures maintain an invariant that every operation must preserve. Suppose we have a function that inserts an element into a sorted list:

```haskell
sorted :: (Ord a) => [a] -> Bool
sorted xs = and $ zipWith (<=) xs (drop 1 xs)

insertSorted :: (Ord a) => a -> [a] -> [a]
insertSorted x [] = [x]
insertSorted x (y : ys)
  | x <= y = x : y : ys
  | otherwise = y : insertSorted x ys
```

The invariant is that the result is sorted whenever the input is:

```haskell
prop_insertSorted :: Integer -> [Integer] -> Property
prop_insertSorted x xs =
  sorted xs ==> sorted (insertSorted x xs)
```

The `(==>)` operator expresses an *implication*, that is a *conditional property*. Logically, a conditional property holds if the precondition (its left-hand side) is *false*, or the postcondition (its right-hand side) is *true*. Operationally, test cases that do not satisfy the precondition are *discarded*, since there is no need to check the postcondition for them. We shall see below that this innocent-looking property has a serious problem.

Round-trip (inverse) properties. Whenever a module offers two functions that are supposed to undo each other - encode and decode, serialise and deserialise, print and parse - their composition should be the identity:

```haskell
prop_showRead :: [Integer] -> Property
prop_showRead xs = read (show xs) === xs
```

This pattern is very useful. The printer and parser from Chapter 3 are a case in point.

Consistency with a reference implementation. If a simple, obviously correct but perhaps hopelessly inefficient implementation is available, it can serve as an *executable specification*. Here is a merge sort:

```haskell
msort :: (Ord a) => [a] -> [a]
msort [] = []
msort [x] = [x]
msort xs = merge (msort ys) (msort zs)
  where
    (ys, zs) = splitAt (length xs `div` 2) xs
    merge [] bs = bs
    merge as [] = as
    merge (a : as) (b : bs)
      | a <= b = a : merge as (b : bs)
      | otherwise = b : merge (a : as) bs
```

and here is its specification, in terms of the standard library's `sort`:

```haskell
prop_msort :: [Integer] -> Property
prop_msort xs = msort xs === sort xs
```

A simplified or partial executable specification is also called a *model* in software engineering discourse, giving rise to the terms *model-based testing (MBT)* and *software under test (SUT)*. A model is usually easier to come by than a logical specification, requires no background in logic, and remains practical even for large systems. MBT is consequently a very useful---and very common---form of specification-driven testing in industrial practice, and the last part of this chapter is devoted to it.

Metamorphic properties. Sometimes we cannot say what the right answer *is*, but we can say how the answers of *several runs of the same function* must relate to each other. For example, for all `xs` and `ys`,

```haskell
msort (xs ++ ys) = msort (ys ++ xs)
```

says that `msort` ignores the order in which two lists are concatenated, without saying anything about what either result is. It is expressed by `prop_msortAppend`:

```haskell
prop_msortAppend :: [Integer] -> [Integer] -> Property
prop_msortAppend xs ys = msort (xs ++ ys) === msort (ys ++ xs)
```

Metamorphic properties are a special case of *relational* properties, which constrain the results of runs of *one or more* functions on the same or related inputs. When the two functions are different, a relational property compares two implementations. For example, if `qsort` is a second sorting function, then for all `xs`,

```
qsort xs = msort xs
```

says that the two agree everywhere, without saying what either of them computes. This is the essence of the model-based testing described above: the reference implementation is one of the two functions.

Postconditions. A weaker but still useful pattern: state what must be true of the output `f x` ---possibly in relation to the input `x` ---without pinning the output down completely. Such properties are also called *direct* or *oracle-based* properties. For example,

```
prop_msortSorted :: [Integer] -> Bool
prop_msortSorted xs = sorted (msort xs)
```

Note that `prop_msortSorted` alone is satisfied by the function `const []`, which is why a good specification usually combines several properties. A conjunction of weak properties can pin down behaviour surprisingly tightly: a sorting function that produces a sorted list *and* preserves the multiset of elements is fully specified.

The `Testable` type class

The `quickCheck` function works on any type which is an instance of `Testable`. The primary instances and their semantics for testing are as follows:

- `()` is testable and succeeds if it returns `()` (the only possible value of the type `()`) and fails if an exception occurs.
- `Bool` is testable where `True` means success and `False` or an exception means failure.

- `Maybe a` is testable if `a` is. `Nothing` means that the test should be *discarded* and counted neither as a success nor as a failure. `Just result` has the meaning of `result`.
- `a -> b` is testable if `a` is `Arbitrary` (meaning that we have a way of generating values of that type; see the next section) and `b` is `Testable`. The semantics is that `f :: a -> b` succeeds if `f x :: b` succeeds for all `x :: a`. In practice this is tested by generating random values of `a`. Note that this instance applies recursively, so e.g. `Integer -> Integer -> Bool` is `Testable` because `Integer -> Bool` is `Testable`; and `Integer -> Bool` is `Testable` because `Bool` is `Testable`.
- `Property` is testable, and is the type of properties that have been enriched with extra information - preconditions, labels, expected coverage, and so on.

This works great. We can write down properties using familiar Haskell types without even depending on the QuickCheck library. However, what if we want to collect all our properties into a list for test organisation purposes? If we have `p :: Integer -> Bool` and `q :: String -> Bool` then `[p, q]` is not well-typed. The `Testable` type class has a method `property :: a -> Property` which converts any `Testable` value into a `Property`. Think of `Property` as a `Testable` value of unknown type. A list of properties should be of type `[Property]` and constructed like `[property p, property q]`.

Combinators on properties

A handful of combinators turn ordinary boolean functions into more informative properties. The most important are:

```
(==>)        :: Testable prop => Bool -> prop -> Property
(===)        :: (Eq a, Show a) => a -> a -> Property
(=/=)        :: (Eq a, Show a) => a -> a -> Property
forAll       :: (Show a, Testable prop) => Gen a -> (a -> prop) ->
Property
counterexample :: Testable prop => String -> prop -> Property
ioProperty   :: Testable prop => IO prop -> Property
withMaxSuccess :: Testable prop => Int -> prop -> Property
```

`(===)` deserves special mention. It behaves like `(==)`, but when the test fails it prints *both* values, which is almost always what one wants:

```
prop_lengthAppend' :: [Integer] -> [Integer] -> Property
prop_lengthAppend' xs ys = length (xs ++ ys) === length xs + length
ys
```

`(==>)` introduces a precondition, as we saw above. A property whose precondition is false is *discarded*: QuickCheck generates a fresh test case instead. By default QuickCheck gives up after discarding 1000 times as many cases as it needs successes, and reports how many it threw away. As an example of a genuine precondition, division:

```
prop_divMod :: Integer -> Integer -> Property
prop_divMod x y = y /= 0 ==> (x `div` y) * y + (x `mod` y) === x
```

Discarding is wasteful when the precondition is unlikely to hold, and - as we shall see - it silently biases the test data. Where possible it is better to generate only values that satisfy the precondition in the first place. For simple side conditions QuickCheck offers *modifier* newtypes that do exactly that:

```
prop_divMod' :: Integer -> NonZero Integer -> Property
prop_divMod' x (NonZero y) = (x `div` y) * y + (x `mod` y) === x
```

Useful modifiers include `NonZero`, `NonNegative`, `Positive`, `NonEmptyList`, `OrderedList`, `InfiniteList`, and `Fun` (which we return to below). They are all ordinary `newtype`s with a tailored `Arbitrary` instance, so you can easily write your own.

Generators and `Arbitrary`

So far we have relied on QuickCheck to come up with values for our properties automatically. The mechanism behind this is the `Arbitrary` type class alluded

to above, which provides a method `arbitrary :: Gen a` giving the "canonical" generator for the type. Let us first understand generators.

Generator basics

Intuitively, type `Gen a` represents a probability distribution over elements of `a`. It can also be thought of as a computation that can depend on random choices. Concretely it is essentially

```
newtype Gen a = MkGen { unGen :: QCGen -> Int -> a }
```

which is a state-like monad. A generator is a function from the current state of a pseudo-random number generator---confusingly, but conventionally, called a *seed*---and a *size* parameter to a value. The seed is threaded through in the style of a state monad, using operations that produce a new seed, or a pair of new seeds, from a given one; the size is available in the style of a reader monad. Generators are built from primitives and monad operations rather than by manipulating seed and size directly.

The simplest generator is `pure x` which produces the value `x` with probability

1. Given a list of generators `gs :: [Gen a]` the generator `oneof gs` chooses one of the generators with equal probability. For instance `oneof [pure "heads", pure "tails"]` produces `"heads"` and `"tails"` with equal probability, but `oneof [pure "heads", pure "tails", pure "tails"]` is biased in favour of `"tails"`.

It is also possible to explicitly control the bias using `frequency`, which is like `oneof` but allows specifying the weight of each option. The biased example using `oneof` would be written more idiomatically as `frequency [(1, pure "heads"), (2, pure "tails")]`.

The other core primitives are `elements :: [a] -> Gen a`, which picks uniformly from a list of *values* (rather than generators), and `choose :: Random a => (a, a) -> Gen a`, which picks uniformly from an inclusive range. For integers, `chooseInt :: (Int, Int) -> Gen Int` avoids a class constraint.

QuickCheck has a function called `sample` which takes a generator and prints 10 example values. This is quite useful to get a rough sense of what a generator produces and is often sufficient to spot simple biases like in the previous example. Its sibling `sample' :: Gen a -> IO [a]` returns the values instead of printing them, and `generate :: Gen a -> IO a` produces a single one.

> **Tip**
>
> Whenever you write a non-trivial generator, run `sample` on it. It takes a few seconds and it may save you hours of debugging. Later in this chapter we shall see how to automate the same sanity check so that it runs as part of your test suite.

Recursive generators

A *generator combinator* is a function that maps one or more generators to a generator. QuickCheck has a generator combinator called `listOf` that generates `[a]` given a generator for `a`. Let us generate a list of integers using the standard integer generator given by its `Arbitrary` instance. An example output is:

```
> sample $ listOf (arbitrary :: Gen Integer)
[]
[2]
[1,0]
[1,6,0,-4]
[-6,6,6]
[-7,5,2]
[8,-8,-1,12,-4,11,-2,9,5]
[13,-14,2,5,9,8,-7,3,-9,-8,-13,-7,-1,1]
[8,-10,14,15,-9,-3,-5,-8,9,-2,-5,8,-16,-4,-15]
[18,-8,-3,15,13,1,6,-2]
[-12,20,18,10]
```

Note that there is a decent spread both in the length of the list and the individual integer values. How would we go about implementing a combinator like `listOf`? A first attempt might be:

```
list1 :: Gen a -> Gen [a]
list1 g = oneof [pure [], (:) <$> g <*> list1 g]
```

This is a choice between an empty list and a list consisting of an element generated by `g` followed by a recursively generated list. Each choice has equal chance, so 50% of lists will be empty, another 25% will have just a single element and so on.

Alas, the distribution leaves something to be desired:

```
> sample $ list1 (arbitrary :: Gen Integer)
[0]
[]
[]
[]
[1,8,7]
[]
[1,-12]
[]
[-16,7,0]
[4,3]
[]
```

Every other sample is an empty list and long lists are exceedingly unlikely, which makes this generator inefficient for exploring the search space. Re-testing a value we have already tried is wasted effort: the property is a deterministic function of its input, so a repeated test case can never tell us anything new.

A second attempt might be to use `frequency` to introduce a bias towards longer lists:

```
list2 :: Gen a -> Gen [a]
list2 g = frequency [(1, pure []), (9, (:) <$> g <*> list2 g)]
```

The resulting distribution is better, but the length of the list now follows a *geometric* distribution with mean 9 - the generator has no idea how large a test case is currently wanted, so it produces lists of roughly the same length

whether QuickCheck is looking for a small first test case or a large hundredth one.

```
> sample $ list2 (arbitrary :: Gen Integer)
[]
[1,1,1,0,-2,-1,-1,1,2,-2,-1,-1,2,-1,-1,2,-1,1,1]
[4]
[]
[-4,-7,-4]
[-4,10,-6,4,5,-7,9]
[-11,-3,-7,-5,-2,2,-3,6,-1,12,9,-9]
[-7,-8,-8,10,1,-9,-9,8,-9,2,4]
[15,1]
[-13,13,5,14,4,5,-8,2,-13,9,4,0,-15]
[-5,17,6,14,3,4,-18,-1]
```

We could try adjusting the bias, but no matter what value we use the length will follow the same shape of distribution, which is not really what we want.

For our third attempt, we exploit the fact that `Gen` is a monad. First generate a non-negative integer `n`, and then generate a list of length `n`:

```
list3 :: Gen a -> Gen [a]
list3 g = abs <$> (arbitrary :: Gen Int) >>= go
  where
    go 0 = pure []
    go n = (:) <$> g <*> go (n - 1)
```

Now the distribution is similar to QuickCheck's `listOf`.

```
> sample $ list3 (arbitrary :: Gen Integer)
[]
[0,-2]
[0,2,-1,2]
[]
[-8]
[8,-6,0,-10]
[-8,5,10,3]
[-1,-1,-12]
[-16,-1,-11,6,-1,-13,14,-1,8,-1]
[-4,14,-7,-9,9,-13,1,6,-18,10]
[12,-16,11,-3,20]
```

Size-dependent generators

Why did `list3` work? Because `arbitrary :: Gen Int` respects the *size* parameter, and `list3` more or less accidentally inherits that behaviour. It is better to be explicit about it.

When testing a property it is often a good idea to start with small values and then gradually increase the complexity of the test cases. QuickCheck uses this approach by giving generators access to a size parameter which under default settings starts at 0 and increases by 1 every test up to a maximum of 99.

The size can be accessed directly using `getSize :: Gen Int` but usually a neater approach is to use the combinator `sized :: (Int -> Gen a) -> Gen a` which turns a size-dependent generator into an ordinary one. A good generator respects the size parameter, so our list generator is more idiomatically written as:

```
list4 :: Gen a -> Gen [a]
list4 g = sized $ \n -> chooseInt (0, n) >>= go
  where
    go 0 = pure []
    go n = (:) <$> g <*> go (n - 1)
```

This is essentially the definition of `listOf` in QuickCheck (where `go` is known as `vectorOf`). The related combinators `resize :: Int -> Gen a -> Gen a` and `scale :: (Int -> Int) -> Gen a -> Gen a` let you locally override the size, which is the standard trick for making a recursive generator terminate: recurse at a strictly smaller size, and produce a leaf when the size reaches zero.

> ⚠ **Warning**
>
> A recursive generator that does *not* shrink the size on recursive calls may diverge, or produce test cases so enormous that the test suite appears to hang. If your property-based test suite suddenly stops terminating, an unbounded generator is the first thing to suspect.

Function generators

Some properties quantify over *functions*, not just over first-order data. The functor law `fmap (f . g) == fmap f . fmap g` is an example. QuickCheck can generate functions, print them, and even shrink them, via the `Fun` modifier:

```
applyFun :: Fun a b -> (a -> b)
```

A property taking a `Fun a b` argument receives a random function that QuickCheck knows how to display as a finite table of input/output pairs, which makes counterexamples readable. The argument type `a` must be an instance of `CoArbitrary` and `Function`; both can be derived for most types, and we shall not go into how.

Testing an abstract data type against a reference implementation

We now have enough machinery to attack a realistic specification problem. Recall that an *abstract data type* (ADT) is a declaration of a possibly parameterised type, type signatures of operations involving that type, and properties the operations must satisfy. An *implementation* of the ADT consists of definitions of the type and the operations such that the properties are satisfied.

In Haskell an ADT is commonly declared by a class declaration, and an implementation by an instance declaration. The properties belong to the class declaration. They are usually just documentation, because Haskell checks statically that an instance provides the correctly *typed* operations, but has no built-in facility for verifying that an implementation satisfies the stipulated properties. This is where property-based testing plays a key role: the class properties provide the specification against which an instance is tested.

Recall the `StateMonad` class from Chapter 4, an ADT with the standard monad laws and the three state laws put-get, put-put and get-put.

```
class (Monad m) => StateMonad m s | m -> s where
  get :: m s
  put :: s -> m ()
```

Let us consider the two state monads `State` from Chapter 2 and `IState` from Chapter 4.

```
-- A newtype is an isomorphism; record syntax names both directions
of it,
-- the constructor one way and the selector the other.
newtype State s a = State (s -> (a, s))

runState :: s -> State s a -> (a, s)
runState s (State f) = f s
```

```
-- Like State, a newtype whose selector is the true inverse of the
-- constructor: the caller supplies the reference.
newtype IState s a = IState {runIState :: IORef s -> IO a}

-- Convenience: create the reference from an initial state, then
run.
runIStateFrom :: s -> IState s a -> IO a
runIStateFrom s c = do
  ref <- newIORef s
  runIState c ref
```

We have called both of them state monads, in the sense of implementations of the `StateMonad` ADT. Their instance declarations (which we shall not repeat here; see Chapter 4) do provide the correctly typed operations---but do they satisfy the monad and state laws?

There are two common ways of testing an ADT implementation.

Method 1: Testing the laws directly

The first is to state *universal equational properties* that the operations must satisfy. Let us consider the state laws:

```
put s >> get      =  put s >> pure s      -- put-get
put s >> put s'   =  put s'               -- put-put
get >>= put       =  pure ()              -- get-put
```

A fundamental question is what `=` means here. What does it mean for two computations to be equal? The most compelling answer is: they are equal when they are substitutable for each other in any *program context* without changing the *observable behaviour* of the program. In Haskell a program context can be taken to be a program of type `IO ()` with a "hole" into which an expression can be plugged. If plugging the hole with `e` or with `e'` has the same outcome in *every* program context, then `e` and `e'` are *observationally equivalent*. Taking observational equivalence as equality gives us exactly what we want from an equality:

- it licenses replacing `e` by `e'` *wherever* `e` occurs in a program; and
- conversely, if `e` and `e'` are intersubstitutable then they are "equal" in precisely that sense.

This notion of equality is the computational counterpart of what philosophers call *Leibniz equality*. Note that it *depends on the operations available in the contexts*: the fewer operations there are to distinguish two expressions, the more expressions are equal. For example, if there is no `run...` operation for "getting out of" a monad, then any two computations of the same monadic type are observationally equivalent. Conversely, if two computations behave the same when run, then they behave the same in all contexts. For the `State Int` monad we can therefore define observational equivalence as the QuickCheck property `(~=)`:

```
infix 4 ~=

(~=) :: (Eq a, Show a) => State Int a -> State Int a -> Property
c1 ~= c2 = property $ \s -> runState s c1 === runState s c2
```

Note what has happened here: the universal quantification over initial states has turned into a QuickCheck property, because `property` applied to a function generates the argument. We can now turn the state laws into QuickCheck properties:

```
prop_putGet :: Int -> Property
prop_putGet s = (put s >> get) ~= (put s >> pure s)

prop_putPut :: Int -> Int -> Property
prop_putPut s s' = (put s >> put s') ~= put s'

prop_getPut :: Property
prop_getPut = (get >>= put) ~= pure ()
```

> **Note**
>
> `(~=)` is an *approximation* of observational equivalence for testing purposes: it tries only finitely many initial states, and it fixes the state type to `Int`. It is *sound* in the sense that observational equivalence implies `(~=)`; it is *incomplete* in the sense that `(~=)` does not imply observational equivalence.
>
> That is the right way round. Deciding observational equivalence of programs in a Turing-complete language is undecidable, so the point of testing is to find a *sound* approximation of an undecidable property: a failing test case then *proves* that the property does not hold. Being unable to find a failing test case *despite one's best efforts at finding one* is taken as supporting evidence---but never proof---that the property may hold after all.

Method 2: Testing against a reference implementation

The second way is to provide a *reference implementation* (also called an *executable specification* or *model*) and to require that the software under test (SUT) be observationally indistinguishable from it. The basic idea is this: if the reference implementation is correct, then so is the SUT. Sometimes the reference implementation *is* the specification.

Let us take `State Int` as the reference implementation and `IState Int` as the SUT. The two have different types, so we cannot compare them directly. We

instead provide a way of running an `IState` computation that yields the same observation as running a `State` computation, namely an initial state mapped to a value and a final state:

```
-- Run an IState computation the way runState runs a State
computation:
-- from an initial state to a value AND a final state.  (Week 4's
-- runIStateFrom discards the final state, which is exactly the part
we
-- need to observe here.)
runIStateFull :: s -> IState s a -> IO (a, s)
runIStateFull s c = do
  ref <- newIORef s
  a <- runIState c ref
  s' <- readIORef ref
  pure (a, s')
```

Note that Chapter 4's `runIStateFrom` will not do here: it discards the final state, which is precisely one of the two things we need to observe. With `runIStateFull` in hand, the property "the implementation *simulates* the reference" becomes expressible:

```
simulates :: (Eq a, Show a) => IState Int a -> State Int a ->
Property
c `simulates` r = property $ \s ->
  ioProperty $ do
    x <- runIStateFull s c
    pure (x === runState s r)
```

and the specification of the implementation is a handful of one-line properties:

```haskell
prop_simGet :: Property
prop_simGet = get `simulates` get

prop_simPut :: Int -> Property
prop_simPut s = put s `simulates` put s

prop_simPure :: Int -> Property
prop_simPure x = pure x `simulates` pure x

prop_simBind :: Fun Int Int -> Property
prop_simBind fun =
  (get >>= put . f) `simulates` (get >>= put . f)
  where
    f = applyFun fun
```

Observe that each of these properties mentions its program *once*: because `get`, `put` and `pure` are overloaded, the very same expression denotes a computation in `IState Int` on the left of `simulates` and in `State Int` on the right. Testing that an implementation simulates a reference implementation is, in this setting, just running one program under two instances of the same class.

The last property is where `Fun` comes into play: `(>>=)` takes a *function* as its second argument, so to test it at all we must generate one. `applyFun` turns the generated `Fun Int Int` into an ordinary function, and if the property fails QuickCheck prints that function as a readable table rather than as `<function>`.

> **Note**
>
> `prop_simBind` tests `(>>=)` only at continuations of the very restricted shape `put . f`. Testing `(>>=)` for *arbitrary* continuations requires generating random *computations*, not just random functions---which means generating values of a datatype that describes computations, and interpreting that datatype in both implementations. That is exactly the technique we develop in the last part of this chapter.

Test data design: partitioning and coverage

A property-based test is only as good as the data it is run on. Random generation is excellent at producing large numbers of *typical* values, but it is poor at hitting the rare *boundary* values where bugs actually live. The classical technique for finding those is *input partitioning*, and the two approaches combine rather than compete.

Input partitioning

Specification-driven *input partitioning* (also called *equivalence partitioning*) is a design method for constructing test data by inspecting the property to be checked. Given a universally quantified property, partition its valid inputs into a *finite* set of pairwise disjoint subsets whose union is the whole input space. From each partition, choose

- a *typical* element, and
- one or more *boundary* elements;

and determine the expected output for each according to the *specification* - not the code. The boundary elements are where the falsification power lies.

For example, a generally useful partitioning of numeric types is:

- For 32-bit two's-complement integers: negative numbers, `{0}`, positive numbers, with values such as `-2147483648`, `-137`, `-1`, `0`, `1`, `2377`, `2147483647`.
- For 64-bit IEEE 754 floating-point numbers: negative infinity, negative numbers, `{0}`, positive numbers, positive infinity, and NaN - remembering that `0.0` has two representations, and that NaN is not equal to itself.

Note how most of the interesting values are boundary values that a uniform random generator is unlikely to produce.

> ⚠ **Test design is not just the test suite**
>
> Specification-driven testing is the systematic, documented *process* of analysing a specification to arrive at test data with high falsification power. The test suite is the *output* of that process. By itself a test suite says nothing about its own falsification power; only the reasoning that produced it does. This is why your reports are asked to describe *how* you designed your tests, not merely to list them.

Seeing what you are actually testing

The bridge between the two approaches is that QuickCheck can *report* the distribution of the test data it generated, so that we can check whether our partitions are being hit. The relevant combinators are:

```
classify :: Testable prop => Bool -> String -> prop -> Property
label    :: Testable prop => String -> prop -> Property
collect  :: (Show a, Testable prop) => a -> prop -> Property
tabulate :: Testable prop => String -> [String] -> prop -> Property
```

`classify` reports the percentage of tests for which a condition held, `label` and `collect` group test cases by a string, and `tabulate` maintains a named table of frequencies. Let us apply them to the sorted-insertion property from earlier, which used `(==>)`:

```
lengthBucket :: [a] -> String
lengthBucket xs
  | n == 0 = "0"
  | n < 5 = "1-4"
  | n < 20 = "5-19"
  | otherwise = ">=20"
  where
    n = length xs
```

```
prop_insertSorted' :: Integer -> [Integer] -> Property
prop_insertSorted' x xs =
  sorted xs ==>
    classify (length xs < 2) "trivial" $
      tabulate "length of xs" [lengthBucket xs] $
        sorted (insertSorted x xs)
```

The result is sobering:

```
> quickCheck prop_insertSorted'
*** Gave up! Passed only 77 tests; 1000 discarded tests (64%
trivial).

length of xs (77 in total):
74% 1–4
26% 0
```

The property passes, but it has tested almost nothing. Random lists are almost never sorted, so 1000 test cases were thrown away, and the ones that survived were sorted *by accident* - which is to say, they were nearly all of length 0, 1 or 2. A precondition that is rarely satisfied does not merely waste time: it quietly restricts the test data to a tiny and unrepresentative corner of the input space.

The fix is to generate values that satisfy the precondition by construction, and to feed them to the property with `forAll`:

```
genSorted :: Gen [Integer]
genSorted = sort <$> arbitrary
```

```
prop_insertSorted'' :: Integer -> Property
prop_insertSorted'' x =
  forAll genSorted $ \xs ->
    tabulate "length of xs" [lengthBucket xs] $
      sorted (insertSorted x xs)
```

```
> quickCheck prop_insertSorted''
+++ OK, passed 100 tests.

length of xs (100 in total):
48% >=20
35% 5–19
11% 1–4
 6% 0
```

Much better - no discards, and a healthy spread of list lengths.

Demanding coverage

Inspecting a table by eye is fine while developing a generator, but it does not help six months later when someone changes the generator and silently destroys the distribution. QuickCheck can therefore be asked to *require* a given level of coverage, and to fail the test if it is not met:

```
cover         :: Testable prop => Double -> Bool -> String -> prop -
> Property
checkCoverage :: Testable prop => prop -> Property
```

`cover p b s` asserts that at least `p` percent of test cases satisfy `b`, and labels them `s`. On its own this only prints a warning; wrapping the property in `checkCoverage` turns insufficient coverage into a *test failure*. Note that `checkCoverage` does something statistically non-trivial: rather than testing a fixed number of times and comparing percentages, it keeps running more tests until it can conclude, with high confidence, either that the requirement is met or that it is not. This is why the number of tests reported below is not 100.

Let us apply the partitioning idea to insertion into a sorted list. The interesting partitions are: the empty list, insertion before all elements, insertion after all elements, and insertion of a value already present.

```
prop_insertSortedCover :: Integer -> Property
prop_insertSortedCover x =
  forAll genSorted $ \xs ->
    checkCoverage
      . cover 2 (null xs) "empty list"
      . cover 20 (not (null xs) && x < head xs) "insert at front"
      . cover 20 (not (null xs) && x > last xs) "insert at back"
      . cover 20 (x `elem` xs) "insert a duplicate"
      $ sorted (insertSorted x xs)
```

Our carefully improved generator turns out to be badly inadequate:

```
> quickCheck prop_insertSortedCover
*** Failed! Insufficient coverage (after 400 tests):
17.5% insert a duplicate
 7.8% insert at back
 5.8% empty list
 5.8% insert at front

Only 5.8% insert at front, but expected 20.0%
```

Of course: `x` is generated completely independently of `xs`, so in a list of thirty elements it will almost always land somewhere in the middle. The boundaries - which is where an off-by-one error in `insertSorted` would show up - are hardly ever exercised.

The remedy is to let the design of the test data drive the generator: generate `x` *in relation to* `xs`, deliberately producing boundary cases a good fraction of the time.

```haskell
boundaries :: [Integer] -> [(Int, Gen Integer)]
boundaries xs =
  [ (3, pure $ head xs - 1),
    (3, pure $ last xs + 1),
    (3, elements xs)
  ]

genInsertion :: Gen (Integer, [Integer])
genInsertion = do
  xs <- genSorted
  x <-
    frequency $
      (2, arbitrary) : if null xs then [] else boundaries xs
  pure (x, xs)



prop_insertSortedCover' :: Property
prop_insertSortedCover' =
  forAll genInsertion $ \(x, xs) ->
    checkCoverage
      . cover 2 (null xs) "empty list"
      . cover 20 (not (null xs) && x < head xs) "insert at front"
      . cover 20 (not (null xs) && x > last xs) "insert at back"
      . cover 20 (x `elem` xs) "insert a duplicate"
      $ sorted (insertSorted x xs)
```

```
> quickCheck prop_insertSortedCover'
+++ OK, passed 800 tests:
30.0% insert a duplicate
27.8% insert at back
26.8% insert at front
 5.4% empty list
```

This is the practical synthesis of the two techniques: *input partitioning determines what the distribution ought to look like, `cover` states that intention as a machine-checkable requirement, and the generator is written to satisfy it.* The random generator then fills in the typical values within each partition for free.

> **Where implementation knowledge comes in**
>
> Test design, which inputs to generate to falsify a given specification, can also be driven by peeking at the implementation. It consists of deriving a finite input partition such that each block exercises a certain set of control paths in the program, for example the then-branch of a conditional or iterating at least once through a loop. Its purpose is to ensure that the test data exercises each part of a program. If some part is left unexercised an error could lurk in there. It critically important to keep in mind that you *still need a specification*. Just running code on data that exercise all parts of the code proves literally nothing about the code if there is no specification. The code does what it does--which is a tautology. *Smoke testing* has the basic property "does-not-crash*, which is extremely weak, but at least is a property to test against. It is useful in practice because almost all specifications include the "does-not-crash" property.

Shrinking

Suppose we define our own type of pairs with an instance of `Arbitrary`:

```
data Pair a b = Pair a b
  deriving (Show)

instance (Arbitrary a, Arbitrary b) => Arbitrary (Pair a b) where
  arbitrary = Pair <$> arbitrary <*> arbitrary
```

We can now define a version of commutativity for `(++)` that takes the input as a `Pair`:

```
prop_appendCommutative' :: Pair [Integer] [Integer] -> Bool
prop_appendCommutative' (Pair xs ys) = xs ++ ys == ys ++ xs
```

QuickCheck still finds a counterexample. A possible output is:

```
> quickCheck prop_appendCommutative'
*** Failed! Falsified (after 5 tests):
Pair [-2,1,3,-2] [-4,4,-1,2]
```

However, the counterexample is not as simple as the counterexample to our original property. Running `quickCheck` multiple times will reveal that `prop_appendCommutative` consistently produces small counterexamples while `prop_appendCommutative'` produces counterexamples of various sizes.

The secret ingredient is *shrinking*. The `Arbitrary` type class also defines a member `shrink :: a -> [a]` which takes a value and produces a list of *shrinks*, i.e. "slightly smaller" values. The idea is that if `x` is a counterexample to some property then any of the elements in the list `shrink x` could also be counterexamples.

When QuickCheck finds a counterexample `x` it tests the property for each shrink of `x`. If that results in another, by definition simpler, counterexample the process repeats recursively until the (locally) simplest counterexample is reached.

For a value `Pair x y` a natural notion of "slightly smaller" is a pair which is slightly smaller in *either* the first component *or* the second component. The complete `Arbitrary` instance is thus:

```
instance (Arbitrary a, Arbitrary b) => Arbitrary (Pair a b) where
  arbitrary = Pair <$> arbitrary <*> arbitrary

  shrink (Pair x y) = [Pair x' y | x' <- shrink x] ++ [Pair x y' |
y' <- shrink y]
```

Why not shrink both components simultaneously? Well, suppose `x'` is slightly smaller than `x` and `y'` is slightly smaller than `y`. Then `Pair x' y'` is slightly smaller than `Pair x y'` (or `Pair x' y`) which in turn is slightly smaller than `Pair x y`, so assuming that either `Pair x y'` or `Pair x' y` is also a counterexample the process would reach `Pair x' y'` in two steps. In general, there is a trade-off between efficiency (i.e. not producing too many shrinks) and likelihood of finding the very simplest counterexample.

> ⚠ **The rule every shrink function must obey**
>
> Every value in `shrink x` must be *strictly smaller* than `x` under some well-founded ordering. If a shrink can ever produce a value that is not smaller, shrinking may loop forever, and your test suite will appear to hang after finding a bug - which is the worst possible moment for it to do so. `shrink _ = []` is always a legal (if useless) definition, and is a perfectly reasonable starting point.

There is a second pitfall, which we shall meet in earnest in the next section: shrinking is not aware of any preconditions. If your generator produces values satisfying an invariant that `shrink` does not preserve, QuickCheck will happily "simplify" your counterexample into a value that was never a legal input in the first place, and you will spend an afternoon debugging a non-bug. When using `forAll` with a custom generator, use `forAllShrink` to supply a matching shrink function - and make sure it preserves the invariant.

Running properties with Tasty

The Tasty testing framework we have used since Chapter 1 has support for QuickCheck via the package `tasty-quickcheck`. For example, `testProperties "properties" props` is a simple test tree, given `props :: [(String, Property)]`, and `testProperty name prop` produces a single one.

This test tree exposes options for Tasty on the command line. The most useful are:

```
$ cabal test --test-option --quickcheck-tests=10000
$ cabal test --test-option --quickcheck-replay=42
$ cabal test --test-option --quickcheck-verbose
```

The first controls the number of tests, and is the standard way of trading time for confidence. The second fixes the random seed, which makes a failing run *reproducible* - indispensable when you are chasing a bug that only shows up

once in a thousand runs. The same effects can be had programmatically with `withMaxSuccess` and `quickCheckWith stdArgs { replay = ... }`.

> **Tip**
>
> A property that fails intermittently in CI and passes on your machine is a property whose seed you have not recorded. Note the seed from the failing run and replay it.

Testing stateful systems

The examples above assume that we are testing a pure function where properties involve a relationship between the inputs and outputs of the function. Regrettably however, many computer systems are *stateful*: they encapsulate some internal data which is modified in response to commands, and partially revealed in response to requests. A conventional mutable data structure, such as a resizable array, is a simple example of a stateful system, and is the one we shall study below. The definition covers much larger systems too - a network service, or the concurrent servers of Chapter 6.

Large stateful systems are often complicated and error-prone, and hence it is very desirable to be able to test them effectively. It turns out that property-based testing is also an effective approach for such systems (and in industry, this may even be the most impactful use case), although it requires us to build some additional infrastructure.

The basic idea is a generalisation of the reference-implementation technique we applied to the state monad above. We take the so-called *software under test* (SUT) and construct a *model* that captures the most important properties of the SUT. In many cases a stateful system has implementation details that are important to its operation (such as caching, optimisations, persistence in a database, integration with other systems, etc), but which are not part of its external interface. A model is a program that imitates some subset of the behaviour of the SUT, and is typically much simpler than the SUT. We then

randomly generate *commands* that interact with the SUT and the model, and test that the observable behaviour is the same. Essentially, the model is an executable specification that we use to validate the behaviour of the SUT.

One very important detail is that there is no requirement that the SUT is implemented in the same language as the property-based testing framework (e.g., Haskell in our case). Indeed, it is a common approach to construct a model in Haskell and use QuickCheck to verify that some other network-based system matches the behaviour of the model. This is particularly useful when the choice of technology for the SUT is constrained due to efficiency or integration concerns.

A sample stateful system

The SUT we will test in the following happens to be implemented in Haskell for simplicity, although we stress that this is not required for the approach to work. Specifically, our SUT is a datatype `DynamicArray a` that implements mutable arrays of elements of type `a` with efficient support for appending elements. Since the array is mutable, all operations on it will be in `IO`. The implementation details of the array are not important, and so will be covered somewhat briefly.

First we will need to import some library functions.

```
import Control.Monad (forM_)
import Data.Array.IO (IOArray)
import Data.Array.MArray (newArray_, readArray, writeArray)
import Data.IORef (IORef, newIORef, readIORef, writeIORef)
```

Our representation of `DynamicArray` will be quite similar to what you may have seen in systems programming classes, or implemented yourself. The idea is to have an underlying array with room for more elements than have actually been inserted yet, with the inserted elements all being at the front of the array, such that there is room to grow at the end. We call the size of this array the *capacity*. When the number of elements inserted by the user exceeds the capacity, then we bump the capacity by some factor, allocate a new array of that size, then copy the old elements to the new array. If we always double the capacity, then

it can be shown that appending an element can be done in amortised constant time.

```
data DynamicArray a
  = DynamicArray
  { -- Number of elements inserted.
    daUsed :: IORef Int,
    -- Capacity.
    daCap :: IORef Int,
    -- Underlying array.
    daArr :: IORef (IOArray Int a)
  }
```

Because we need to modify both the number of used elements and the capacity, we represent these as mutable `IORef`s. The underlying array is an `IOArray Int a`, which is a mutable (but non-resizable) array type provided by Haskell. The `Int` type argument is the index type, which can be used to represent multidimensional arrays, but we will not make use of this.

> **Note**
>
> The interface to `IOArray` is through various class-polymorphic functions defined by `Data.Array.MArray`. For simplicity of exposition, the types in the following have been monomorphised to be specific to `IOArray`.

An `IOArray` is produced by the function `newArray_` of the following type:

```
newArray_ :: (Int,Int) -> IO (IOArray Int a)
```

The `(Int,Int)` pair is the smallest and largest valid index. These arrays do not have to start at zero, although ours do. Initially, all elements of the array will be undefined. Reading an undefined element will cause an IO exception.

We arbitrarily decide that the initial capacity of our dynamic arrays will be 10.

```haskell
newDynamicArray :: IO (DynamicArray a)
newDynamicArray = do
  let capacity = 10
  arr <- newArray_ (0, capacity - 1)
  used_ref <- newIORef 0
  capacity_ref <- newIORef capacity
  arr_ref <- newIORef arr
  pure $
    DynamicArray
      { daUsed = used_ref,
        daCap = capacity_ref,
        daArr = arr_ref
      }
```

When indexing an array we check whether the index is in-bounds, returning `Nothing` if not.

```haskell
index :: Int -> DynamicArray a -> IO (Maybe a)
index i (DynamicArray used_ref _cap_ref arr_ref) = do
  used <- readIORef used_ref
  if i >= 0 && i < used
    then do
      arr <- readIORef arr_ref
      Just <$> readArray arr i
    else pure Nothing
```

Inserting always succeeds and produces no result beyond modifying the array, but we have to resize the underlying array if the new element causes the capacity to be exceeded. This is done by allocating a new array and copying the contents of the old one.

```haskell
insert :: a -> DynamicArray a -> IO ()
insert x (DynamicArray used_ref cap_ref arr_ref) = do
  used <- readIORef used_ref
  cap <- readIORef cap_ref
  arr <- readIORef arr_ref
  if used < cap
    then do
      writeArray arr used x
      writeIORef used_ref $ used + 1
    else do
      let cap' = cap * 2
      arr' <- newArray_ (0, cap')
      forM_ [0 .. used - 1] $ \i ->
        writeArray arr' i =<< readArray arr i
      writeArray arr' used x
      writeIORef arr_ref arr'
      writeIORef used_ref $ used + 1
      writeIORef cap_ref cap'
```

Overwriting an existing element is quite similar to indexing. We return `Nothing` on out-of-bounds, and `Just ()` on success.

```haskell
write :: Int -> a -> DynamicArray a -> IO (Maybe ())
write i x (DynamicArray used_ref _cap_ref arr_ref) = do
  used <- readIORef used_ref
  if i >= 0 && i < used
    then do
      arr <- readIORef arr_ref
      writeArray arr i x
      pure $ Just ()
    else pure Nothing
```

Our final operation allows the deletion of an element anywhere in the array. This is a somewhat costly operation, as we have to shift all elements after the deleted one left. Further, to avoid using too much memory, if the capacity is too large after the deletion, we shrink the array back down to a smaller size. As before, we return `Nothing` in case the index is out of bounds.

```
delete :: Int -> DynamicArray a -> IO (Maybe ())
delete i (DynamicArray used_ref cap_ref arr_ref) = do
  used <- readIORef used_ref
  if i >= 0 && i < used
    then do
      cap <- readIORef cap_ref
      arr <- readIORef arr_ref
      forM_ [i + 1 .. used - 1] $ \j ->
        writeArray arr (j - 1) =<< readArray arr j
      writeIORef used_ref $ used - 1
      if used < cap `div` 2
        then do
          let cap' = cap `div` 2
          arr' <- newArray_ (0, cap')
          forM_ [0 .. used - 1] $ \j ->
            writeArray arr' j =<< readArray arr j
          writeIORef used_ref $ used - 1
          writeIORef arr_ref arr'
          writeIORef cap_ref cap'
        else pure ()
      pure $ Just ()
    else pure Nothing
```

Defining a model

There is a lot of intricate index-fiddling code in the above. Can we really be sure it is correct? At a semantic level, what `DynamicArray` is doing is not complicated - it is merely maintaining a sequence of elements. The complexity comes from our desire for certain performance characteristics. Let us define a model that captures the central behaviour of `DynamicArray`, but without caring about operational matters such as performance.

```
data Model a = Model [a]
  deriving (Show)

initModel :: Model a
initModel = Model []
```

A model is described by its internal state and by which *commands* can be sent to it, and which *responses* may be produced in return. These commands usually resemble the API of the SUT, but perhaps simplified in various ways. Due to the simplicity of the system we are testing here, the commands are however very similar to the API of the SUT.

```
data Command a
  = Insert a
  | Index Int
  | Write Int a
  | Delete Int
  deriving (Eq, Show)

data Response a = Success | Failure | Elem a
  deriving (Eq, Show)
```



Command-query separation

Note that our commands fall into two groups: those that change the state (`Insert` , `Write` , `Delete`) and those that merely observe it (`Index`). A design discipline in which every operation is either a *command* that changes state and returns nothing, or a *query* that returns something and changes nothing, is known as *command-query separation*. It is not a rule you must follow, but systems that respect it are markedly easier to test, because the observations are cleanly separated from the state transitions. If you find that a stateful system is painful to model, an operation that both mutates and reports a result is often the reason.

We then define functions corresponding to executing each of the commands. Each function accepts the command parameters and the current model state, and returns a new model state, as well as a response. For example, the function corresponding to an insertion is defined as follows.

```
cmdInsert :: a -> Model a -> (Model a, Response a)
cmdInsert x (Model xs) = (Model $ xs ++ [x], Success)
```

We define functions for the remaining commands, and finally a `step` function for simulating the effect of running a certain `Command` on a `Model`:

```
cmdIndex :: Int -> Model a -> (Model a, Response a)
cmdIndex i (Model xs) =
  if i >= 0 && i < length xs
    then (Model xs, Elem (xs !! i))
    else (Model xs, Failure)

cmdWrite :: Int -> a -> Model a -> (Model a, Response a)
cmdWrite i x (Model xs) =
  if i >= 0 && i < length xs
    then (Model $ take i xs ++ [x] ++ drop (i + 1) xs, Success)
    else (Model xs, Failure)

cmdDelete :: Int -> Model a -> (Model a, Response a)
cmdDelete i (Model xs) =
  if i >= 0 && i < length xs
    then (Model $ take i xs ++ drop (i + 1) xs, Success)
    else (Model xs, Failure)

step :: Model a -> Command a -> (Model a, Response a)
step m (Insert x) = cmdInsert x m
step m (Index i) = cmdIndex i m
step m (Write i x) = cmdWrite i x m
step m (Delete i) = cmdDelete i m
```

Similarly, we also define a function for executing a `Command` on our SUT. This involves mapping the SUT API to the `Command` / `Response` types, which in our case is quite simple, but can sometimes be a bit laborious.

```
exec :: DynamicArray a -> Command a -> IO (Response a)
exec a (Insert x) = do
  insert x a
  pure Success
exec a (Index i) = do
  r <- index i a
  case r of
    Just x -> pure $ Elem x
    Nothing -> pure Failure
exec a (Write i x) = do
  r <- write i x a
  case r of
    Just () -> pure Success
    Nothing -> pure Failure
exec a (Delete i) = do
  r <- delete i a
  case r of
    Just () -> pure Success
    Nothing -> pure Failure
```

Specifying a property

We denote a sequence of commands as a *program*. The testing problem is now to ensure that the SUT and the model behave identically for all programs.

```
newtype Program a = Program [Command a]
  deriving (Show)
```

First of course, we have to define appropriate `Arbitrary` instances. We are going to define these in the simplest way imaginable, and then - as in the pure case above - find out how bad they are.

```haskell
instance (Arbitrary a) => Arbitrary (Command a) where
  arbitrary =
    oneof
      [ Insert <$> arbitrary,
        Index <$> arbitrary,
        Write <$> arbitrary <*> arbitrary,
        Delete <$> arbitrary
      ]

instance (Arbitrary a) => Arbitrary (Program a) where
  arbitrary = Program <$> listOf arbitrary
  shrink (Program l) = map Program (shrink l)
```

Let us look at what programs end up being generated.

```
> sample (arbitrary :: Gen (Program Int))
Program []
Program []
Program [Index (-3),Delete (-3),Delete 2]
Program []
Program [Insert (-8),Delete 6,Delete 7,Insert 2,Insert (-2)]
Program [Write (-1) (-10),Insert 9,Index 5]
Program [Delete (-9),Write (-6) 3,Index (-8),Write 10 0,Index
8,Insert (-10)]
Program [Index (-9),Write (-4) 0,Delete (-11),Write 2 (-9),Index
4,Index 2,Insert (-1),Insert 2,Delete 10,Delete (-14)]
Program [Write 14 (-12),Delete (-3),Delete (-1),Delete (-15),Index
(-2),Write (-5) (-15),Index (-9),Delete 4,Write (-7) (-2),Index
0,Index 10,Insert (-11),Delete (-6),Index 16,Delete (-13)]
Program [Index 11]
Program [Write (-14) (-11),Write (-11) (-13),Insert (-11),Index
(-9),Insert (-17),Index (-19),Delete (-18),Delete 6]
```

It is clear that once again, we generate a good number of empty programs, but we also generate a good many programs that barely touch the array at all. Look at how many of them perform `Index` or `Delete` commands - very often at negative indices

- without first inserting any elements. It is good and necessary to test these cases, but they are clearly generated too often.

Next we define a procedure for executing a `Program a` on both a `DynamicArray a` and a `Model a`, producing a boolean that indicates whether the same responses were observed. Recall that we cannot directly inspect the internal state of a `DynamicArray`, but we *can* check its observable behaviour.

```
runProgram :: (Eq a) => DynamicArray a -> Model a -> Program a -> IO
Bool
runProgram c0 m0 (Program cmds0) = go c0 m0 cmds0
  where
    go _c _m [] = pure True
    go c m (cmd : cmds) = do
      sut_resp <- exec c cmd
      let (m', model_resp) = step m cmd
      if sut_resp == model_resp
        then go c m' cmds
        else pure False
```

Finally, we just need to construct a property that uses `runProgram`. The crucial building block is the function `ioProperty`, which allows us to turn an arbitrary IO operation into a `Property`:

```
ioProperty :: Testable prop => IO prop -> Property
```

We must of course be very careful when using this function, as it is easy to use it to construct a property that behaves nondeterministically, which can make it difficult for QuickCheck to shrink test cases properly. In our case we are merely using it to construct the initial dynamic array and to execute `runProgram`:

```
prop_array :: Program Int -> Property
prop_array prog = ioProperty $ do
  c <- newDynamicArray
  runProgram c initModel prog
```

Finally we can verify the property.

```
> quickCheck prop_array
+++ OK, passed 100 tests.
```

Making failures informative

Passing tests are gratifying but uninformative. Let us introduce a bug: remove the line `writeArray arr' used x` from the `else`-branch in the definition of `insert`, so that a resize loses the element that triggered it. Rerunning the tests gives, some of the time:

```
> quickCheck prop_array
*** Failed! (after 99 tests and 14 shrinks):
Exception:
  MArray: undefined array element
  CallStack (from HasCallStack):
    error, called at libraries/array/Data/Array/Base.hs:812:16 in
array-0.5.8.0:Data.Array.Base
Program [Insert (-60),Insert (-12),Insert (-92),Insert 78,Insert
(-4),Insert (-67),Insert 66,Insert (-82),Insert (-65),Insert
(-38),Insert 38,Delete 6,Index 9]
```

We are not told exactly what is wrong (this is an exception thrown from the underlying array library), but we are given a sequence of commands that reliably leads to the problem, and which we can use to debug it.

When the SUT does not crash but merely disagrees with the model, `runProgram`'s `Bool` result is much less helpful: we are told *that* some program failed, but not *which command* diverged, nor what the two sides said. Since we have the information at hand, we may as well report it, using `counterexample` to attach a message to a failing property:

```
checkProgram ::
  (Eq a, Show a) =>
  DynamicArray a ->
  Model a ->
  Program a ->
  IO Property
checkProgram c0 m0 (Program cmds0) = go m0 cmds0
  where
    go _m [] = pure $ property True
    go m (cmd : cmds) = do
      sut_resp <- exec c0 cmd
      let (m', model_resp) = step m cmd
      if sut_resp == model_resp
        then go m' cmds
        else
          pure $
            counterexample
              ( unlines
                  [ "Diverged on command: " ++ show cmd,
                    "In model state:      " ++ show m,
                    "SUT response:        " ++ show sut_resp,
                    "Model response:      " ++ show model_resp
                  ]
              )
              False


prop_array' :: Program Int -> Property
prop_array' prog = ioProperty $ do
  c <- newDynamicArray
  checkProgram c initModel prog
```

This costs four lines and saves a great deal of debugging. As a rule, when a property is more complicated than a single equation, spend the effort to make its failure output say what actually went wrong.

Generating programs that make sense

The naive generator produces far too many commands that are rejected out of hand. The fix is the same one as for sorted lists: use the *model* - which we have

right there - to generate commands that make sense in the state reached so far.

```
genCommand :: (Arbitrary a) => Model a -> Gen (Command a)
genCommand (Model xs) = frequency $ unconstrained ++ inBounds
  where
    unconstrained =
      [ (insertWeight, Insert <$> arbitrary),
        (1, Index <$> arbitrary),
        (1, Write <$> arbitrary <*> arbitrary),
        (1, Delete <$> arbitrary)
      ]
    inBounds
      | null xs = []
      | otherwise =
          [ (3, Index <$> validIndex),
            (3, Write <$> validIndex <*> arbitrary),
            (3, Delete <$> validIndex)
          ]
    validIndex = chooseInt (0, length xs - 1)
    insertWeight = max 2 (30 - 2 * length xs)
```

Two things are going on here. First, whenever the array is non-empty we generate in-bounds indices a good fraction of the time, by choosing them from the range the model says is valid. Second, the weight of `Insert` *depends on the current model state*: it is high when the array is small and drops as the array fills up, so that programs tend to drive the array to an interesting size and then keep it there rather than growing without bound. This kind of state-dependent weighting is the bread and butter of writing generators for stateful systems.

Generating a program is then a fold over model states: generate a command for the current state, step the model, and continue.

```haskell
genProgram :: (Arbitrary a) => Gen (Program a)
genProgram = sized $ \n -> do
  k <- chooseInt (0, n)
  Program <$> go initModel k
  where
    go _ 0 = pure []
    go m k = do
      cmd <- genCommand m
      let (m', _) = step m cmd
      (cmd :) <$> go m' (k - 1)
```

```haskell
prop_array'' :: Property
prop_array'' =
  forAllShrink (genProgram :: Gen (Program Int)) shrink $ \prog ->
    ioProperty $ do
      c <- newDynamicArray
      checkProgram c initModel prog
```

Note the use of `forAllShrink`, which supplies a shrink function alongside the custom generator; plain `forAll` does not shrink at all, and a stateful counterexample that has not been shrunk is nearly unreadable.

Preconditions and shrinking

We got away with reusing the ordinary list `shrink` for `Program` because *every* command is meaningful in *every* state: an out-of-bounds `Index` is not a programmer error, it is a legitimate request with a `Failure` response. Many stateful systems are not like that. If your SUT would crash, or has undefined behaviour, when a command is issued in the wrong state, then that command has a *precondition*, and both generation and shrinking must respect it - shrinking especially, since dropping an earlier command can easily invalidate a later one.

The standard recipe is to make the precondition explicit and filter the shrinks through it:

```haskell
precondition :: Model a -> Command a -> Bool
precondition (Model xs) cmd = case cmd of
  Insert _ -> True
  Index i -> ok i
  Write i _ -> ok i
  Delete i -> ok i
  where
    ok i = i >= 0 && i < length xs

validProgram :: Program a -> Bool
validProgram (Program cmds0) = go initModel cmds0
  where
    go _ [] = True
    go m (cmd : cmds) =
      precondition m cmd && go (fst (step m cmd)) cmds

shrinkProgram :: Program a -> [Program a]
shrinkProgram (Program cmds) =
  filter validProgram $ map Program $ shrinkList (const []) cmds
```

Here `shrinkList (const []) cmds` produces the sub-lists of `cmds` obtained by deleting commands (but does not attempt to shrink the individual commands), and `validProgram` re-runs the precondition check along the model trace. For our `DynamicArray` this particular precondition would be too strong - we *want* to test out-of-bounds accesses - but the pattern is exactly what you need for a system where illegal commands must not be issued.

Partitioning the state space

For a pure function we partitioned the space of *inputs*. For a stateful system we must partition the space of *states*, and design commands that reach the interesting ones. For our dynamic array, the interesting states are those around the capacity boundaries: just below the initial capacity, just above it (so that a resize has happened), and having deleted back down again (so that a shrink has happened).

Since the model records the sequence of states a program passes through, we can compute these observations directly from the program:

```haskell
modelTrace :: Program a -> [(Model a, Command a, Response a)]
modelTrace (Program cmds0) = go initModel cmds0
  where
    go _ [] = []
    go m (cmd : cmds) =
      let (m', resp) = step m cmd
       in (m, cmd, resp) : go m' cmds

maxElems :: Program a -> Int
maxElems prog = maximum $ 0 : [length xs | (Model xs, _, _) <-
modelTrace prog]

deletedWhenLarge :: Program a -> Bool
deletedWhenLarge prog =
  or [length xs > 10 | (Model xs, Delete _, _) <- modelTrace prog]

anyRejected :: (Eq a) => Program a -> Bool
anyRejected prog = or [resp == Failure | (_, _, resp) <- modelTrace
prog]
```

and then state our coverage requirements:

```haskell
programCoverage :: Program Int -> Property -> Property
programCoverage prog =
  checkCoverage
    . tabulate "maximum number of elements" [bucket (maxElems prog)]
    . cover 30 (maxElems prog > 10) "grows past initial capacity"
    . cover 10 (deletedWhenLarge prog) "deletes while large"
    . cover 30 (anyRejected prog) "some command out of bounds"
  where
    bucket n
      | n == 0 = "0"
      | n <= 10 = "1-10"
      | n <= 20 = "11-20"
      | otherwise = ">20"
```

```
prop_arrayNaiveCoverage :: Program Int -> Property
prop_arrayNaiveCoverage prog =
  programCoverage prog $
    ioProperty $ do
      c <- newDynamicArray
      checkProgram c initModel prog

prop_arrayCoverage :: Property
prop_arrayCoverage =
  forAllShrink (genProgram :: Gen (Program Int)) shrink $ \prog ->
    programCoverage prog $
      ioProperty $ do
        c <- newDynamicArray
        checkProgram c initModel prog
```

Applying this to the naive generator is illuminating:

```
> quickCheck prop_arrayNaiveCoverage
*** Failed! Insufficient coverage (after 800 tests):
92.5% some command out of bounds
16.6% grows past initial capacity
14.9% deletes while large

maximum number of elements (799 in total):
63.8% 1-10
19.5% 0
14.5% 11-20
 2.1% >20

Only 16.6% grows past initial capacity, but expected 30.0%
```

Over 92% of programs contain an out-of-bounds command, but fewer than one in five ever grows the array past its initial capacity - which is to say that the resizing code, by far the most intricate part of the implementation, is barely tested at all. The state-aware generator does much better:

```
> quickCheck prop_arrayCoverage
+++ OK, passed 400 tests:
74.0% some command out of bounds
47.5% grows past initial capacity
43.0% deletes while large

maximum number of elements (400 in total):
47.5% 11–20
44.5% 1–10
 8.0% 0
```

This is not a cosmetic difference. Reintroduce the bug in `insert` from above and run each version twenty times, at the default 100 tests per run:

generator	runs that caught the bug
naive	7 out of 20
state-aware	20 out of 20

Test data that report success two times out of three on code that is definitely broken may be worse than no test data at all, because they give the impression of being a *test suite*---which is designed to maximise the likelihood of finding an error---when they are nothing of the sort. This is what the coverage requirement buys us: it tells us, *before* we come to rely on the tests, that our test data hardly ever reach the code we care about.

> **Tip**
>
> For a stateful system, the hard part is rarely writing the model; it is test design---generating command sequences that reach the states in which errors manifest themselves. Budget your effort accordingly.

Perspective

You may object that the amount of infrastructure needed to perform testing of this kind is rather large - our test harness is more lines of code than the `DynamicArray` implementation itself. This is a fair objection, but real systems

tend to grow in complexity much faster than their models, so this is a technique that scales well as the SUT grows complicated, although it is sometimes impractical for testing small systems.

It is also worth realising how little of the above depends on the SUT being Haskell. The technique requires only that there is some handle representing the SUT (in our case, `DynamicArray`) and some operation (which may be in `IO`) that performs state changes based on `Command`s. There is nothing that fundamentally prevents us from testing a remote network service this way, or the control system for a robot, or a C library invoked through a foreign-function interface. In these cases we might need to write more complicated `exec` functions, of course.

Because the pattern is so stereotyped, it has been packaged into libraries - `quickcheck-state-machine` and `quickcheck-dynamic` are well-known Haskell libraries that supply the generation, shrinking and reporting machinery so that you need only provide the model, the commands, and the `exec` function. We have built it by hand here because the machinery is worth understanding, and because in practice you will frequently need to adapt it.

Finally, a warning about the limits of the technique. Everything above assumes the commands are executed *sequentially*. Testing *concurrent* code is substantially harder, because a property must hold for *all* possible schedules - all interleavings of all concurrently executing threads - not merely for the schedule that your machine happened to choose today. The standard specification for a concurrent stateful system is *linearisability*: the observed behaviour of a set of concurrent operations must be explainable by *some* sequential ordering of them that respects the real-time order of non-overlapping operations. Checking this requires both a way to explore many schedules and a search for a valid linearisation, and is beyond the scope of AP - but it is built on exactly the model-based foundation developed here.

Concurrent Programming

Haskell offers a wide range of facilities for different styles of concurrent and parallel programming. In this chapter we focus on a limited set of primitives for *concurrent programming*, which can often be found in similar forms in other modern programming languages. We use:

- Lightweight independent **threads** of control.

  "Lightweight" means we can create many threads without worrying about overhead (up to a couple of hundred thousand is usually fine).

- **Channels** for communicating between threads.

The approach we take to concurrent programming in AP is based on the paradigm of *message passing*, and is heavily inspired by languages such as Erlang. The idea is to structure concurrent programs as independent *servers* (a.k.a. *concurrent objects* or *actors* in other languages and frameworks) that communicate with each other by sending asynchronous *messages*. Each server maintains its own private state, and servers cannot directly modify the state of other servers, except by sending messages.

Although the concurrent systems we will construct in AP will run only on a single machine (and within a single Haskell process), the approach is perfectly suitable for distributed systems, as seen in for example Cloud Haskell, or languages such as Erlang and Elixir.

Concurrency is closely related to (but not the same as) *parallelism*. While multi-threading in Haskell is indeed one way to take advantage of parallel multi-core computers, this is not an explicit aspect of our study of concurrency. Instead, we focus on concurrency as a programming model which happens to be convenient for expressing certain forms of event-driven systems. Indeed, while the purpose of parallelism is specifically to obtain better performance, we will for didactic reasons use an excessive amount of concurrency in many cases, such that our programs are likely to be slower than if they were written in a sequential style.

Concurrent programming in Haskell is done in the `IO` monad because threads are executed for their effects. Threads do not have a "return value" as such, so the only way they can influence a computation is through their side effects. Effects from multiple threads are interleaved *nondeterministically* at runtime.

Concurrent programming allows programs that interact with multiple external agents to be modular:

- The interaction with each agent is programmed separately
- Allows programs to be structured as a collection of interacting agents, sometimes called **actors** or (mini) **servers**.

This chapter is about a principled and systematic way of constructing concurrent programs as a collection of interacting servers.

Concurrency Primitives

We use the Haskell modules Control.Concurrent and Control.Concurrent.Chan. The latter is implicitly re-exported by the former. As always, we will use only a fairly small subset of the facilities provided by the Haskell standard library. You are welcome (and encouraged) to peruse the documentation to enlighten yourself, but in this particular case you should be careful not to be tempted by functions that subvert the notion of message-passing. In this section we will use the following facilities from `Control.Concurrent`:

```
import Control.Concurrent
  ( Chan,
    ThreadId,
    forkIO,
    newChan,
    readChan,
    writeChan,
  )
```

To create a new thread in Haskell, we use the `forkIO` function. The `forkIO` function has the following type:

```
forkIO :: IO () -> IO ThreadId
```

In other words, to create a thread we pass `forkIO` an action of type `IO ()`, meaning a monadic computation in the `IO` monad. Typically, this will be some kind of potentially infinite loop that receives and handles messages, as we will see in a moment. The thread will continue to run until this action terminates.

The `forkIO` function returns a `ThreadId` that can be used for interacting with the thread in low level ways, although we will not make much use of that in AP. Instead, we will communicate using channel-based messaging.

Example

```
runThread :: IO ()
runThread = do
  t <- forkIO $ putStrLn "Hello there."
  print t


> runThread
HellToh rtehaedrIed.
47
```

Note how the output of the new thread and the original computation is interleaved.

Channels and Messages

In Haskell, communication is done via *channels*. A channel is created using the `newChan` action:

```
newChan :: IO (Chan a)
```

The `newChan` action produces a channel that can be used for sending and receiving messages of type `a`. The precise type of `a` will be inferred by the compiler.

Messages can be both read and written to a channel, corresponding to receiving and sending messages, using the following two functions:

```
writeChan :: Chan a -> a -> IO ()
readChan :: Chan a -> IO a
```

Conceptually, a channel is an unbounded queue of messages. Writing to a channel is an asynchronous operation - it immediately and always succeeds. Reading from a channel retrieves the oldest message in the channel. If the channel is empty, reading blocks until a message is available.

Example

```
channelExample :: IO ()
channelExample = do
  c <- newChan
  _ <- forkIO $ do
    r <- readChan c
    putStrLn $ "Received message: " <> r
  writeChan c "Hello there."


> channelExample
Received message: Hello there.
```

Whenever we create a thread, we will also create a channel through which we can communicate with the thread. Typically the thread will run a loop that repeatedly reads from the channel and responds to messages.

Example

```
channelLoopExample :: IO ()
channelLoopExample = do
  c <- newChan
  let threadLoop = do
        r <- readChan c
        putStrLn $ "Received message: " <> r
        threadLoop
  _ <- forkIO threadLoop
  writeChan c "The first"
  writeChan c "The second"
  writeChan c "The third"


> channelLoopExample
Received message: The first
Received message: The second
Received message: The third
```

When two different threads have a reference to the same channel (`c` in the example above), they can communicate. However, completely arbitrary use of a shared channel will quickly lead to chaos, so we step in to restore order.

Single-reader principle: we adopt the rule that a channel may have only a *single* reader, meaning only a single thread is allowed to call `readChan` on any given channel. This is typically the thread that we created the channel for. This is not enforced by the Haskell type system, and there are indeed forms of concurrent programming that are more flexible, but they are outside the scope of AP.

It is perfectly acceptable (and often necessary) for a channel to have multiple writers.

If we call `readChan` on a channel where we hold the only reference (meaning we would in principle wait forever), the Haskell runtime system will raise an exception that will cause the thread to be terminated. This is a natural and safe way to shut down a thread that is no longer necessary, assuming the thread

does not hold resources (e.g., open files) that must be manually closed. Handling such cases is outside the scope of this note.

Remote procedure calls (RPC)

The Haskell message passing facility is asynchronous, but quite often we wish to send a message to a server and then wait for it to respond with some kind of result, corresponding to a procedure call. To implement synchronous *remote procedure calls* (RPC), we need to invent a bit of machinery on top of the basic message passing machinery. The way we make it work is by creating a new channel that is used for transmitting the result. This channel is then sent along as part of the message.

To demonstrate the idea, consider a rather contrived server that receives message containing an integer, and responds with that integer incremented by one. The starting point (and always good practice) is to define an explicit type for the messages we would like to send.

```
data Msg = MsgInc Int (Chan Int)
```

We then define our thread loop as follows:

```
threadLoop :: Chan Msg -> IO ()
threadLoop c = do
  msg <- readChan c
  case msg of
    MsgInc x from ->
      writeChan from (x + 1)
  threadLoop c
```

Given a handle to a channel of type `Chan Msg`, we can send a message, and wait for a response, as follows:

```
performRPC :: Chan Msg -> Int -> IO Int
performRPC c x = do
  from <- newChan
  writeChan c $ MsgInc x from
  readChan from
```

And tying it all together:

```
ex2 :: IO ()
ex2 = do
  c <- newChan
  _ <- forkIO $ threadLoop c
  print =<< performRPC c 0
  print =<< performRPC c 1
```

Implementation of the `GenServer` module

Using the concurrency primitives directly is somewhat error-prone, particularly for the constrained form of concurrency we study in AP. Therefore, we wrap these primitive in a module `GenServer` that defines a canonical way of using the techniques discussed above, and in the rest of this note we use the `GenServer` module to write our servers. There may still be cases where we have to break out of the `GenServer` abstraction, but we will largely try to work within it.

We will make use of the following imports:

```
import Control.Concurrent
  ( Chan,
    ThreadId,
    forkIO,
    killThread,
    newChan,
    readChan,
    threadDelay,
    writeChan,
  )
```

Servers

The `forkIO` procedure provides a low-level way to create a new thread. However, we want a canonical way to communicate with our servers. Thus, we

introduce the `Server` abstract type. We represent a server as a pair: a `ThreadId` and an *input channel*:

```
data Server msg = Server ThreadId (Chan msg)
```

Here we use the type variable `msg` to denote the type of messages that a server can receive, which can be different for each kind of server.

```
spawn :: (Chan a -> IO ()) -> IO (Server a)
spawn serverLoop = do
  input <- newChan
  tid <- forkIO $ serverLoop input
  pure $ Server tid input
```

Note that `Server` will be an abstract type - users cannot directly access its components, except through the interface we define below.

Channels

In the `GenServer` abstraction, channels are unchanged from their primitive form, except that we define some more concise function names.

```
send :: Chan a -> a -> IO ()
send = writeChan

receive :: Chan a -> IO a
receive = readChan
```

However, to users of a server the channel is hidden away in the `Server` type, and so we provide a dedicated `sendTo` function for sending a message to the server.

```
sendTo :: Server a -> a -> IO ()
sendTo (Server _tid input) msg =
  send input msg
```

Request-Reply Pattern

We saw above how to implement RPC on top of asynchronous messages. To cut down on the boilerplate and avoid incorrect usage, `GenServer` provides a structured facility for performing RPCs.

First, we define an abstract type that encapsulates the *reply channel*. Under the hood, this is just a normal channel, but the wrapper type denotes that its purpose is to reply to an RPC.

```
newtype ReplyChan a = ReplyChan (Chan a)
```

The idea is that only one message is ever sent to this channel. This is not something we can express within Haskell's type system (at least not without extensions that go beyond what we discuss in AP). We provide a function `reply` for sending a reply on a `ReplyChan`:

```
reply :: ReplyChan a -> a -> IO ()
reply (ReplyChan chan) x = send chan x
```

Finally, we provide a function `requestReply` that encapsulates the notion of creating a reply channel, providing it to a message constructor, and reading the response from the reply channel.

```
requestReply :: Server a -> (ReplyChan b -> a) -> IO b
requestReply serv con = do
  reply_chan <- newChan
  sendTo serv $ con $ ReplyChan reply_chan
  receive reply_chan
```

If we avoid exporting the definition of `ReplyChan` from `GenServer` (meaning it is an abstract type), then `requestReply` is the *only* place one can read from the reply channel, which is exactly what we want.

Method

The following is a five step method to systematically designing and implementing a server. The steps are presented as sequential phases, but in practise there will be a bit of going back and forward between steps.

1. Determine what data (the internal state) the server should keep track of, declare a type for this.

   If you lack imagination for a name for this type you can always go with `InternalData`.

2. Determine the interface for the server. That is, a set of functions where each function takes the server as the first argument and possibly other arguments as well. Furthermore, for each function we should determine if the function is *blocking* or *non-blocking*.

   In general a function is *blocking* if we need to wait for a result depending on the state of the server.

   The blocking behaviour can sometimes be refined into *limited blocking* if there is some upper limit on how long a function can be blocked. We might not know what exactly the upper limit is (as it might depend on various system specific constants and dynamic behaviours). Or *unlimited blocking* if the function might block forever.

   These functions constitute the external interface for the server.

3. Declare an internal type for the kind of messages (both external and internal messages) a server can receive.

   We use the pattern where we make a constructor for each kind of message and the argument(s) to be send with that kind of message. If there is an expected reply to the given kind of message, we use the convention that the last argument for that constructor is a channel for sending back the response. This convention is to make sure that we can use the `requestReply` function without too much bother.

4. Implement a *server-loop* function.

We use the convention that a server-loop function should take the input channel for the server as the first argument. This convention makes it convenient to use the `spawn` function.

The server loop will usually start by receiving a message on the input channel and then use a `case` -expression to pattern match on each kind of message and determine what action to do. Each action will usually compute a (potentially unchanged) internal state, and potentially send some messages, for instance replies to requests.

5. Implement API functions.

   When we have declared the type for messages and implemented the server loop, the last step is to implement the API functions. Where we use `requestReply` for each blocking function and `sendTo` for each non-blocking function.

It is usually best practise to declare each server in a separate module. Thus when we talk about *internal* types and functions, it is types and functions not exported from the module. However, there are cases where a server may define its own internal servers for utility purposes, that would be awkward to define in a separate module.

Worked Example: Counter Server

In this example we want to make a server that keeps track of a count, a *counter server*. It should be possible to *get the value* of the counter, to *increment* the counter by one, or to *decrement* the counter by positive amount `n` . We will maintain the invariant that the counter is always non-negative. While this is perhaps not a terribly useful server, it does demonstrate facilities that most servers will need; namely keeping some kind of state, responding to requests for changes to that state, and maintaining invariants for that state.

Step 1: Internal state

The server should keep track of an integer as the internal state:

```
type InternalData = Int
```

Step 2: API functions

The API for the counter server is:

- `newCounter initial` for creating a new counter server with the initial value `initial`. It is an error if `initial` is negative.
- `getValue cnt` for getting the value of the counter server, `cnt`. This is a *blocking* function, because we need to wait for a result.
- `incr cnt` for incrementing the value of the counter server, `cnt`, by one. This is a *non-blocking* function, because it cannot fail and we don't need to wait for a result.
- `decr cnt n` for decrementing the value of the counter server, `cnt`, by `n`. This is a *blocking* function, because it can fail if `n` is larger than the current value of the counter server. Thus, we need to wait for a result that tells us if the function succeeded.

  Note, this is the interesting function of the example. Because it isn't clear what it should mean to decrement a counter with too large an amount. We have two options:

  1. The function blocks for a short amount of time, and reports if the decrement was successful or not (our choice for now). We call this *limited blocking*.
  2. The function blocks until it is possible to decrement the counter with the given amount. This means that the function might block forever (in principle). We call this *unlimited blocking*.

Step 3: A type for messages

This step is to make an internal type for the messages that will be send to the server.

For the counter server we have a message for each of the interface functions and don't have any internal messages:

```
data Msg
  = GetValue (ReplyChan Int)
  | Incr
  | Decr Int (ReplyChan Bool)
```

Note how the messages for the blocking functions `getValue` and `decr` have a channel as the last argument.

Step 4: Implement the server-loop function

The server-loop function for a counter server is:

```
counterLoop :: InternalData -> Chan Msg -> IO ()
counterLoop state input = do
  msg <- receive input
  case msg of
    GetValue from -> do
      let (newState, res) = (state, state)
      reply from res
      counterLoop newState input
    Incr -> do
      let newState = state + 1
      counterLoop newState input
    Decr n from -> do
      let (newState, res) =
            case state of
              value | value > n -> (value - n, True)
              _ -> (state, False)
      reply from res
      counterLoop newState input
```

Step 5: Implement API functions

The API functions for a counter server can now be implemented straight forward by using the `spawn`, `sendTo` and `requestReply` functions from the `GenServer` module:

```
type CounterServer = Server Msg

newCounter :: Int -> IO CounterServer
newCounter initial | initial >= 0 = spawn $ counterLoop initial
newCounter _ = error "Initial value should be non-negative"

getValue :: CounterServer -> IO Int
getValue cnt = requestReply cnt GetValue

incr :: CounterServer -> IO ()
incr cnt = sendTo cnt Incr

decr :: CounterServer -> Int -> IO Bool
decr cnt n | n >= 0 = requestReply cnt $ Decr n
decr _ _ = error "Cannot decrement with negative amount"
```

Example use of a counter server

```
main = do
  c <- newCounter 0
  incr c
  replicateM_ 5 $ incr c
  _ <- decr c 1
  v <- getValue c
  putStrLn $ "The counter should now be 5, and it is " ++ show v
```

Extending `GenServer` with support for timeouts

The server abstraction does not directly support timeouts for blocking calls. However, we can build our own support for timeouts. The technique we employ

is based on three ingredients:

1. we have a channel where we allow the reply to be either the intended value *or* a special timeout value.
2. we start a worker thread, which will evaluate an action, and send the result back to us.
3. we also launch an extra thread that sleeps for some period of time, then sends the timeout value to us.

If the non-timeout response is the first to arrive, then the timeout value is ignored and harmless.

Then we define a type `Timeout` with a single value `Timeout`.

```
data Timeout = Timeout
```

We can use this to build a function for performing an action with a timeout:

```
actionWithTimeout :: Int -> IO a -> IO (Either Timeout a)
```

That is, `actionWithTimeout s act` will perform the action `act` within a time limit of `s` seconds; it returns an action of type `IO (Either Timeout a)` where the special value `Timeout` is returned if `act` did not complete within the time limit:

```
actionWithTimeout :: Int -> IO a -> IO (Either Timeout a)
actionWithTimeout seconds action = do
  chan <- newChan
  _ <- forkIO $ do
    -- worker thread
    x <- action
    send chan $ Right x
  _ <- forkIO $ do
    -- timeout thread
    threadDelay (seconds * 1000000)
    send chan $ Left Timeout
  receive chan
```

You will note that this is not a server in the `GenServer` sense, as it does not loop: it is simply a utility function that launches two threads. Note also that

`threadDelay` accepts an argument in microseconds, so we have to multiply the provided timeout by one million.

One downside of this function is that the worker thread (the one that runs `action`, and might take too long) is not terminated after the timeout. This is a problem if `action` is, for example, stuck in an infinite loop that consumes ever more memory. To fix this, we can have the timeout thread explicitly kill the worker thread:

```
actionWithTimeoutKill :: Int -> IO a -> IO (Either Timeout a)
actionWithTimeoutKill seconds action = do
  chan <- newChan
  worker_tid <- forkIO $ do
    -- worker thread
    x <- action
    send chan $ Right x
  _ <- forkIO $ do
    -- timeout thread
    threadDelay (seconds * 1000000)
    killThread worker_tid
    send chan $ Left Timeout
  receive chan
```

Note that killing a thread is a dangerous operation in general. It may be the case that the worker thread is stuck in some loop or waiting for a network request, in which case it is harmless, but killing it may also leave some shared state in an unspecified state. We will (hopefully) not encounter such cases in AP, but it is something to be aware of in the future.

A Larger Example: A Module for Asynchronous Computation

In this section we will look at the design of a GenServer-based module for executing pure Haskell functions in an asynchronous manner. It demonstrates several important programming techniques, including the use of sub-threads to ensure responsivity and robustness. We are concerned here only with pure functions. The API of the system we will implement is as follows:

```haskell
data Async a

type Seconds = Int

async :: Seconds -> (a -> b) -> a -> IO (Async b)

data Result a
  = Timeout
  | Exception String
  | Value a
  deriving (Eq, Ord, Show)

poll :: Async a -> IO (Maybe (Result a))

wait :: Async a -> IO (Result a)
```

A value of type `Async a` represents an asynchronous computation that produce a value of type `a`. They are created using the function `async`, which takes as argument a maximum allowed runtime, a function `a -> b`, and an argument value `a`, returning an `Async b`. The `async` function itself must return immediately.

The status of an `Async a` value can be inspected using the functions `poll` and `wait`. The `poll` function immediately returns the state of the asynchronous computation, returning `Nothing` if the computation is still ongoing, and otherwise a result of type `Result a`:

- `Timeout` is returned if the execution of the function exceeded the runtime specified by the original `async` invocation.
- `Exception`, along with the exception error message, is returned if the computation resulted in an exception being thrown.
- `Value` is returned if the computation finished without exceptions and within the allotted runtime.

The `wait` function is similar to `poll`, except that it *blocks* until the computation finishes.

We will use the following definition of `fib` in our examples:

```haskell
fib :: Int -> Int
fib 0 = 1
fib 1 = 1
fib n =
  if n < 0
    then error "negative n"
    else fib (n - 1) + fib (n - 2)
```

Because it returns `Int`, it is easy to use the `evaluate` function from `Control.Exception` to ensure that the result is fully evaluated.

Example

```haskell
simpleDemo :: IO ()
simpleDemo = do
  putStrLn "a"
  a <- async 1 fib 10
  print =<< poll a
  print =<< wait a
```

Initial Design

Our server thread can be seen as a state machine with two main states:

1. Before we know the result of the computation. At this point we must also maintain a list of those clients that have called `wait` and must be informed when the computation finishes.

2. After the result of the computation is known, which requires storing the result.

Further, when we transition from stage 1 to 2, we must inform all of the waiting clients of the result.

For simplicity, we will initially not worry about timeouts and exceptions. It will turn out to be quite simple to add support for these features.

Our design will be to launch a separate *worker thread* that computes the value. This allows the main server thread to immediately answer `poll` requests even though the worker thread is engaged in a long-running computation. Once the worker thread has computed the desired value, it will be sent to the server thread in a message. Further, we need messages for the `poll` and `wait` functions. Our resulting message type is the following:

```
data Msg a
  = MsgPutVal a
  | MsgPoll (ReplyChan (Maybe (Result a)))
  | MsgWait (ReplyChan (Result a))
```

And the `poll` and `wait` functions are merely wrappers around sending the corresponding messages:

```
poll :: Async a -> IO (Maybe (Result a))
poll (Async s) =
  requestReply s MsgPoll

wait :: Async a -> IO (Result a)
wait (Async s) =
  requestReply s MsgWait
```

To represent the two main states of the server, we use two recursive functions:

```haskell
noValueLoop :: Chan (Msg a) -> [ReplyChan (Result a)] -> IO ()
noValueLoop c waiters = do
  msg <- receive c
  case msg of
    MsgPutVal v' -> do
      forM_ waiters $ \from ->
        reply from $ Value v'
      valueLoop c (Value v')
    MsgPoll from -> do
      reply from Nothing
      noValueLoop c waiters
    MsgWait from ->
      noValueLoop c (from : waiters)

valueLoop :: Chan (Msg a) -> Result a -> IO ()
valueLoop c v = do
  msg <- receive c
  case msg of
    MsgPutVal _ ->
      valueLoop c v
    MsgPoll from -> do
      reply from $ Just v
      valueLoop c v
    MsgWait from -> do
      reply from v
      valueLoop c v
```

Note how the `MsgWait` case in `noValueLoop` simply adds the `ReplyChan` to a list. Then, once we receive a `MsgPutVal`, we reply to all of these pending calls. In contrast, when we receive a `MsgWait` in `valueLoop`, we immediately respond with the result. If we are in `valueLoop` and receive another `MsgPutVal` message, we simply ignore it. In practice this will never occur (at least not until we add timeouts and exceptions).

Instead of using two function to represent two states, we could also have used a single function and stored the server state as a sum type with two constructors. I prefer using mutually recursive functions to represent the high level states of a server (if such a distinction makes sense), but this is ultimately a matter of personal taste.

An `Async a` is now just a type that wraps a `Server (Msg a)`:

```haskell
data Async a = Async (Server (Msg a))
```

The `async` function uses `spawn` to create a server. Before entering the server loop, we use `forkIO` to create the worker thread, which has a reference to a channel (`c`) from which the server reads messages:

```haskell
async :: Seconds -> (a -> b) -> a -> IO (Async b)
async timeout f x = do
  s <- spawn $ \c -> do
    void $ forkIO $ do
      x' <- evaluate $ f x
      send c $ MsgPutVal x'
    noValueLoop c []
  pure $ Async s
```

Initially the `noValueLoop` has no waiters.

This completes the basic functionality of the `Async` server.

Handling Timeouts

Timeouts are (almost) always handled by creating a thread that waits for some period of time, then sends a message or takes some other action. Here, we will add a new message that indicates that the timeout has passed:

```haskell
data Msg a
  = ...
  | MsgTimeout
```

We modify `async` such that after creating the worker thread, we also create a thread that sends this message after the timeout has passed. Remember that `threadDelay` expects its argument in microseconds, therefore we multiply the provided timeout by one million.

```haskell
async :: Seconds -> (a -> b) -> a -> IO (Async b)
async timeout f x = do
  s <- spawn $ \c -> do
    void $ forkIO $ do
      x' <- evaluate $ f x
      send c $ MsgPutVal x'
    void $ forkIO $ do
      threadDelay $ timeout * 1000000
      send c MsgTimeout
    noValueLoop c []
  pure $ Async s
```

Finally, we modify `noValueLoop` and `valueLoop` to handle the new `MsgTimeout`. In the former, reception of such a message denotes a timeout, and so we switch to the `valueLoop` state, with `Timeout` as our value:

```haskell
noValueLoop :: Chan (Msg a) -> [ReplyChan (Result a)] -> IO ()
noValueLoop c waiters = do
  msg <- receive c
  case msg of
    ...
    MsgTimeout -> do
      forM_ waiters $ \from ->
        reply from Timeout
      valueLoop c Timeout
```

In `valueLoop`, `MsgTimeout` is simply ignored, as all it indicates is that the original timeout has expired - which is unimportant once the value has been received.

```haskell
valueLoop :: Chan (Msg a) -> Result a -> IO ()
valueLoop c v = do
  msg <- receive c
  case msg of
    ...
    MsgTimeout ->
      valueLoop c v
```

Example

```
timeoutDemo :: IO ()
timeoutDemo = do
  a <- async 1 fib 100
  print =<< poll a
  print =<< wait a


> timeoutDemo
Nothing
Timeout
```

Arguably, if the timeout is reached, we should use `killThread` to liquidate the worker thread, in case it is stuck in an infinite loop. This would require us to augment the state with a reference to the `ThreadId` of the worker thread.

Handling Exceptions

At the protocol level, the handling of exceptions is very similar to the handling of values. We add a new message type:

```
data Msg a
  = ...
  | MsgPutException String
```

The handling of `MsgPutException` is then essentially identical to the handling of `MsgPutVal`:

```
noValueLoop :: Chan (Msg a) -> [ReplyChan (Result a)] -> IO ()
noValueLoop c waiters = do
  msg <- receive c
  case msg of
    ...
    MsgPutException err -> do
      forM_ waiters $ \from ->
        reply from $ Exception err
      valueLoop c (Exception err)



valueLoop :: Chan (Msg a) -> Result a -> IO ()
valueLoop c v = do
  msg <- receive c
  case msg of
    ...
    MsgPutException _ ->
      valueLoop c v
```

Exceptions are caught in the worker thread, using the technique discussed in Chapter 4 of these notes:

```
async :: Seconds -> (a -> b) -> a -> IO (Async b)
async timeout f x = do
  s <- spawn $ \c -> do
    void $ forkIO $ do
      let computeValue = do
            x' <- evaluate $ f x
            send c $ MsgPutVal x'
          onException :: SomeException -> IO ()
          onException e = do
            send c $ MsgPutException $ show e
      catch computeValue onException
    void $ forkIO $ do
      threadDelay $ timeout * 1000000
      send c MsgTimeout
    noValueLoop c []
  pure $ Async s
```

Example

```
exceptionDemo :: IO ()
exceptionDemo = do
  a <- async 1 fib (-1)
  print =<< poll a
  print =<< wait a


> exceptionDemo
Nothing
Exception "negative n"
```

Structured Concurrent Programming

Concurrency with Free Monads

In Chapter 4 we saw an example of Asynchronous Programming with Free Monads. The principles underlying asynchronous programming are quite similar to those of concurrent programming. We can actually express a concurrent system using a free monad, and then interpret the system using either true IO-based concurrency, or simulation. This raises the potential of using the simulation to systematically (or probabilistically) explore the potential execution paths possible in the system, in order to discover race conditions.

In the following we will design a concurrency abstraction that is rather similar to the one described in Chapter 6, based on threads, channels, and messages. One concession we will make for simplicity is that our channels will be *monomorphic*, and capable of sending only a single type of messages, for which we pick `String`:

```
type Msg = String
```

This restriction can be lifted, but requires a significant amount of Haskell type-level trickery, which is not the point of this chapter.

An Initial Attempt

Let us now define a data type `CCOp` for representing concurrency effects. We allow four effects: forking a thread, creating a channel, sending a message on a channel, and receiving a message from a channel. We also instantiate the `Free` monad with the `CCOp` effect to produce `CC`, a monadic representation of concurrency:

```haskell
data CCOp a
  = CCFork (CC ()) a
  | CCNewChan (Chan Msg -> a)
  | CCSend (Chan Msg) Msg a
  | CCReceive (Chan Msg) (Msg -> a)

type CC a = Free CCOp a
```

Note that we are using the `Control.Concurrent.Chan` type as our channel representation - we will change that later. We must of course also define the usual `Functor` instance for `CCOp`:

```haskell
instance Functor CCOp where
  fmap f (CCFork m c) = CCFork m (f c)
  fmap f (CCNewChan c) = CCNewChan $ f . c
  fmap f (CCSend chan msg c) = CCSend chan msg $ f c
  fmap f (CCReceive chan c) = CCReceive chan $ f . c
```

And finally we define accessor functions for constructing monadic operations with these effects:

```haskell
ccNewChan :: CC (Chan Msg)
ccNewChan = Free $ CCNewChan pure

ccFork :: CC () -> CC ()
ccFork m = Free $ CCFork m $ pure ()

ccSend :: Chan Msg -> Msg -> CC ()
ccSend chan msg = Free $ CCSend chan msg $ pure ()

ccReceive :: Chan Msg -> CC Msg
ccReceive chan = Free $ CCReceive chan pure
```

Interpreting `CC` computations in `IO` is quite straightforward, due to how closely our effects match the interface provided by `Control.Concurrency`:

```haskell
interpCCIO :: CC a -> IO a
interpCCIO (Pure x) =
  pure x
interpCCIO (Free (CCFork m c)) = do
  _ <- forkIO $ interpCCIO m
  interpCCIO c
interpCCIO (Free (CCNewChan c)) = do
  chan <- newChan
  interpCCIO $ c chan
interpCCIO (Free (CCSend chan msg c)) = do
  writeChan chan msg
  interpCCIO c
interpCCIO (Free (CCReceive chan c)) = do
  msg <- readChan chan
  interpCCIO $ c msg
```

And now we can write a contrived little program that passes a message through a chain of threads, each adding a token to the message and passing it to the next thread:

```haskell
pipeline :: CC String
pipeline = do
  chan_0 <- ccNewChan
  chan_1 <- ccNewChan
  chan_2 <- ccNewChan
  chan_3 <- ccNewChan
  chan_4 <- ccNewChan
  let passOn tok from to = do
        x <- ccReceive from
        ccSend to $ x ++ tok
  ccFork $ passOn "a" chan_0 chan_1
  ccFork $ passOn "b" chan_1 chan_2
  ccFork $ passOn "c" chan_2 chan_3
  ccFork $ passOn "d" chan_3 chan_4
  ccSend chan_0 ""
  ccReceive chan_4
```

Running it yields the expected results:

```
> interpCCIO pipeline
"abcd"
```

Abstract Channels

Unfortunately, the definition of concurrent effects given above is not suitable for simulation. The reason is that the we specified that a channel is always of type `Chan Msg`, meaning it is intrinsically tied to the interface provided by `Control.Concurrent`. In order to allow multiple interpretations of concurrency, we need to make `CCOp` *polymorphic* in its representation of channel. To this end, we add a type parameter `chan`, which we use instead of `Chan Msg`:

```
data CCOp chan a
  = CCFork (CC chan ()) a
  | CCNewChan (chan -> a)
  | CCSend chan Msg a
  | CCReceive chan (Msg -> a)

instance Functor (CCOp chan) where
  fmap f (CCFork m c) = CCFork m (f c)
  fmap f (CCNewChan c) = CCNewChan $ f . c
  fmap f (CCSend chan msg c) = CCSend chan msg $ f c
  fmap f (CCReceive chan c) = CCReceive chan $ f . c
```

In our definition of the `CC` type alias, we also add `chan` as a type parameter:

```
type CC chan a = Free (CCOp chan) a
```

Now a value of type `CC chan a` represents a concurrent execution that produces a value of type `a`, and uses a channel representation `chan`. When we actually write computations in `CC`, we will leave `chan` polymorphic - only the interpretation functions will impose constraints on it. This sounds a bit abstract (because it is), but will become clear later. First, however, we have to redefine the accessor functions, which will also have to use a `chan` type parameter:

```haskell
ccNewChan :: CC chan chan
ccNewChan = Free $ CCNewChan pure

ccFork :: CC chan () -> CC chan ()
ccFork m = Free $ CCFork m $ pure ()

ccSend :: chan -> Msg -> CC chan ()
ccSend chan msg = Free $ CCSend chan msg $ pure ()

ccReceive :: chan -> CC chan Msg
ccReceive chan = Free $ CCReceive chan pure
```

Now we can make the `pipeline` example work with our new definition. This is quite straightforward - in fact, we only have to change the type, and the definition can be unchanged:

```haskell
pipeline :: CC chan String
pipeline = ...
```

A similar situation arises for the interpretation function `interpCCIO`. Here we simply require that the channel representation is `Chan Msg`, but otherwise the implementation is the same:

```haskell
interpCCIO :: CC (Chan Msg) a -> IO a
interpCCIO = ...
```

Now consider what happens when we run the example:

```
> interpCCIO pipeline
"abcd"
```

This type checks because `pipeline` has the polymorphic type `CC chan String` where `chan` can be instantiated with any type, and in particular it can be instantiated with `Chan Msg` - which is what `interpCCIO` requires. In this way we can write generic code that delays the concrete choice of channel representation. Let us now exploit this to actually write a pure interpreter for `CC`.

A Pure Interpreter

The pure interpreter will be more complicated than `interpCCIO`, because we cannot piggyback on the existing Haskell runtime system for concurrency. Our approach will essentially be that of a state monad, where we maintain the following main bits of state:

- A collection of all channels and the messages they currently contain.
- A collection of all threads that can be executed further.

Since channels have a notion of identity, we need a way to uniquely identify them, which we will do with the `ChanId` type:

```
type ChanId = Int
```

Each channel will be associated with a unique integer. This means we also need to have a source of fresh integers, which we will accomplish by maintaining a counter in our state.

Now we are ready to define a Haskell type encapsulating our concurrency simulator state:

```
data CCState = CCState
  { ccCounter :: ChanId,
    ccChans :: [(ChanId, [Msg])],
    ccThreads :: [CC ChanId ()]
  }
```

The `ccChans` field stores all existing channels, keyed by a `ChanId`, with each channel storing a list of messages with the oldest first. The `ccThreads` stores suspended threads, which are represented as monadic computations of type `CC ChanId ()`.

The actual monad we will use is the `State` monad from Chapter 2 with `CCState` as the state. When programming with state monads it is usually a good idea to define higher-level utility functions rather than using `get` / `put` directly. First we define a function `getChan` that retrieves the messages associated with a specific channel:

```
getChan :: ChanId -> State CCState [Msg]
getChan chan_id = do
  state <- get
  pure $
    fromMaybe (error "unknown channel") $
      lookup chan_id $
        ccChans state
```

And its counterpart, `setChan`, that sets the messages associated with a channel.

```
setChan :: ChanId -> [Msg] -> State CCState ()
setChan chan_id msgs = do
  state <- get
  put $
    state
      { ccChans =
          (chan_id, msgs)
            : filter ((/= chan_id) . fst) (ccChans state)
      }
```

It is not difficult to imagine how we will implement reading from a channel: use `getChan` to fetch the inbox, remove the first message, and use `setChan` to put back the remainder. The tricky part is how to handle the situation when no messages are available, but we will come back to that.

We also need to be able to add threads to the state, which is done by `addThread`.

```
addThread :: CC ChanId () -> State CCState ()
addThread m = do
  state <- get
  put $ state {ccThreads = m : ccThreads state}
```

And finally, `incCounter` increments the counter in the state and returns the old value. It serves as our mechanism for obtaining fresh `ChanId`s.

```haskell
incCounter :: State CCState ChanId
incCounter = do
  state <- get
  put $ state {ccCounter = ccCounter state + 1}
  pure $ ccCounter state
```

We can now define a function `step` that evaluates a `CC Int ()` computation *as far as possible*, meaning until it blocks or terminates. Remember that the only way a thread can block in our system is by trying to read from an empty channel. The `step` function is *not* a full interpretation function, but we will use it to build one.

The `step` function has the following type:

```haskell
step :: CC Int a -> State CCState (CC ChanId a)
```

Note that it returns a `CC ChanId a` because there is no guarantee that it is able to run the computation to completion (which would produce an `a`). The simplest case is the one for `Pure`, which represents a finished computation for which there is nothing further to do:

```haskell
step (Pure x) = pure $ Pure x
```

Now we need to handle the various effects. Creating a channel is done by retrieving an unused `ChanId`, then adding a channel with an initially empty message queue:

```haskell
step (Free (CCNewChan c)) = do
  chan_id <- incCounter
  setChan chan_id []
  step $ c chan_id
```

Forking a thread simply adds the computation to the state with `addThread`:

```haskell
step (Free (CCFork m c)) = do
  addThread m
  step c
```

The `CCSend` effect is executed by appending the given message to the specified channel, then executing the continuation:

```
step (Free (CCSend chan_id msg c)) = do
  msgs <- getChan chan_id
  setChan chan_id $ msgs ++ [msg]
  step c
```

Finally, the most interesting effect is `CCReceive`, because it can block when the channel is empty. This is represented by simply returning the monadic computation unchanged:

```
step (Free (CCReceive chan_id c)) = do
  msgs <- getChan chan_id
  case msgs of
    [] -> pure $ Free $ CCReceive chan_id c
    msg : msgs' -> do
      setChan chan_id msgs'
      step $ c msg
```

We now have a `step` function for running as much of a single computation (i.e., thread!) as possible. But if `step` is stuck on a `CCReceive`, then no amount of re-running `step` is going to make progress - rather, some other thread must be given a chance to run, which may end up putting a message in the queue of some channel that the original thread was stuck on. To accomplish this, the function `stepThreads` invokes `step` on every thread in the system. First we write an incorrect implementation:

```
-- BEWARE: WRONG!
stepThreads :: State CCState ()
stepThreads = do
  state <- get
  threads <- mapM step $ ccThreads state
  put $ state {ccThreads = threads}
```

At first glance, this may look right: fetch all the threads, advance them a step, then put them back in the state. But that final `put` is a problem, because it effectively reverts any state modifications done inside `step`. We can attempt a fix:

```
-- BEWARE: STILL WRONG!
stepThreads :: State CCState ()
stepThreads = do
  state <- get
  threads <- mapM step $ ccThreads state
  new_state <- get
  put $ new_state {ccThreads = threads}
```

This is still wrong. While we now maintain *some* state modifications done in `step`, we completely overwrite the list of threads. This means that if `step` creates any new threads, they are thrown away. Let us try again:

```
-- BEWARE: STILL WRONG!
stepThreads :: State CCState ()
stepThreads = do
  state <- get
  threads <- mapM step $ ccThreads state
  new_state <- get
  put $ new_state {ccThreads = threads ++ ccThreads new_state}
```

Instead of overwriting the list of threads, we now simply prepend to it. But now we end up *duplicating* the threads, since the original threads from `state` (the ones we pass to `step`) are still present in `new_state`. A correct solution requires us to *remove* the threads from the state before we step them:

```
stepThreads :: State CCState ()
stepThreads = do
  state <- get
  put $ state {ccThreads = []}
  threads <- mapM step $ ccThreads state
  new_state <- get
  put $ new_state {ccThreads = threads ++ ccThreads new_state}
```

The point of this progression through incorrect implementations of `stepThreads` was not to demonstrate how to write incorrect code, but as an illustration of the subtleties of working with mutable state.

Now we can write a function that evaluates a "main thread" (the one producing the ultimate execution result), but first uses `stepThreads` to advance all "background threads".

```haskell
interp :: CC ChanId a -> State CCState a
interp (Pure x) = pure x
interp (Free op) = do
  stepThreads
  op' <- step $ Free op
  interp op'
```

As soon as the "main thread" reaches the `Pure` constructor, the system is considered terminated. This is different from `Control.Concurrency`, which has no notion of a "main thread", and where forked threads can continue to do arbitrary side effects forever.

The final bit of machinery we need is a bit of boilerplate for running our state monad with an initial state, and projecting out the result we care about:

```haskell
interpCCPure :: CC ChanId a -> a
interpCCPure orig =
  fst $ runState initial_state $ interp orig
  where
    initial_state =
      CCState
        { ccCounter = 0,
          ccChans = [],
          ccThreads = []
        }
```

And we can see that things work:

```
> interpCCPure pipeline
"abcd"
```

One limitation of this approach, which is not present in `interpCCIO`, is that we cannot handle infinite loops in *pure* code. We are only able to switch between threads when `step` encounters an effect.

Remaining Issues

Although the interpreter developed above works in some cases, it has undesirable behaviour for others. Consider a program that creates a channel

and thread that runs an infinite loop that continuously writes to the channel. The main thread reads twice from the channel, concatenates the messages, and returns:

```
infiniteWrite :: CC chan String
infiniteWrite = do
  chan <- ccNewChan
  ccFork $ forever $ ccSend chan "x"
  a <- ccReceive chan
  b <- ccReceive chan
  pure $ a ++ b
```

Using the IO-based interpreter, we observe the following result, as expected:

```
> interpCCIO infiniteWrite
"xx"
```

Since the main thread terminates in finite time, it doesn't matter that some secondary thread is still running (although it's not great that it continues to run and causes the channel to grow infinitely - it would be better if `interpCCIO` kept track of launched threads and killed them at the end).

Now let us try running the program using the pure interpreter:

```
> interpCCPure infiniteWrite
```

You will find that this goes into an infinite loop. This occurs due to `stepThreads`, which will evaluate every thread with `step` until the next time it blocks on a channel. But since the thread we have forked will *never* read from a channel, `step` will continue recursively evaluating it forever - preventing any other thread from being executed. To fix this, we modify the `CCSend` case in `step` such that we do not evaluate the continuation, but merely return it:

```
step (Free (CCSend chan_id msg c)) = do
  msgs <- getChan chan_id
  setChan chan_id $ msgs ++ [msg]
  pure c -- This line was previously 'step c'.
```

This does not prevent the computation from progressing, since we still do *some* work whenever `step` is invoked, and `step` is anyway invoked repeatedly in a loop.

Now computation terminates as expected:

```
> interpCCPure infiniteWrite
"xx"
```

But consider now this program:

```
infiniteLoop :: CC chan String
infiniteLoop = do
  chan <- ccNewChan
  ccFork $ forever $ pure ()
  ccFork $ ccSend chan "x"
  ccReceive chan
```

Here we fork two threads: one goes into an infinite loop that does nothing, while the other thread sends a message on a channel. The main thread receives a message from a channel, which is returned. Execution with `interpCCIO` works fine, but `interpCCPure` goes into an infinite loop. In this case, however, the loop is in *pure* code, rather than being an infinite list of effects. This means that we have no way of interrupting it. Operationally, `step` is stuck on evaluating the (never terminating) thread to figure out which kind of effect it evaluates to.

There is a deep lesson here: when interpreting a free monad, the only time we can "interrupt" computation and get back control is when an effect occurs, so there is no way we can avoid this problem in `interpCCPure`. The IO-based concurrency from `Control.Concurrency` uses machine-level interrupts to suspend even pure computation, and therefore does not have this problem. If we could somehow force computations to issue an effect from time to time, this would not be a problem. In fact, one could easily imagine that Haskell itself could have implemented concurrency by injecting "suspension effects" into the generated code. However, Haskell does not allow us to inject effects into otherwise pure code. If we have control over how the computations using the free monad are constructed, we can of course ensure that effects occur

regularly - we could even imagine a "step" effect that serves no purpose except to interrupt computation.

Tricks with Channels

The channel model we have used in AP is quite simple. Real message passing systems often add additional features, such as:

1. *Peeking* at a channel, which determines whether a message is available for reading, but without blocking if there is not.
2. Reading from two (or more) channels at the same time, blocking only if none of them have messages.
3. Broadcasting to multiple channels simultaneously.

There are often performance reasons to implement these as primitives in a message passing system, but as we shall see, they can be implemented nicely using the primitives we are already familiar with. In particular, the trick that may be a bit counterintuitive to most programmers is to use a larger number of auxiliary threads.

We will start with the obligatory imports.

```
import Control.Concurrent (Chan, forkIO, newChan, readChan,
writeChan)
import Control.Monad (forM_, forever)
import GenServer
```

Peeking

Our goal is to define an API that has a type `PeekChan a` that supports an operation

```
peek :: PeekChan a -> IO (Maybe a)
```

that will return `Nothing` if the channel has no outstanding messages, and otherwise fetch a message as with `receive`. (This is a slight abuse of nomenclature, as "peeking" normally implies that we do *not* remove anything.)

The idea is that `PeekChan` will wrap an original `Chan` and provide the additional support for peeking. We will implement `PeekChan` as a server that maintains a *buffer* of messages that have been sent to the underlying channel. The server then supports two operations: appending a message to the queue, or reading a message from the queue. An auxiliary thread is responsible for repeatedly reading from the original channel and forwarding to the `PeekChan` server. The trick is that the blocking behaviour of `receive` is isolated in a thread whose responsivity is not important.

This is our definition of `PeekChan`:

```
data Peek a
  = Peek (ReplyChan (Maybe a))
  | Put a

type PeekChan a = Server (Peek a)

peek :: PeekChan a -> IO (Maybe a)
peek s = requestReply s Peek
```

Creating a `PeekChan` from a `Chan` is with the `peekChan` function:

```haskell
peekChan :: Chan a -> IO (PeekChan a)
peekChan c = do
  s <- spawn $ loop []
  _ <- forkIO $ forever $ sendTo s . Put =<< receive c
  pure s
  where
    loop buffer sc = do
      msg <- receive sc
      case msg of
        Put x -> loop (buffer ++ [x]) sc
        Peek rc ->
          case buffer of
            [] -> do
              reply rc Nothing
              loop buffer sc
            x : xs -> do
              reply rc $ Just x
              loop xs sc
```

Reading from multiple channels

Sometimes a thread must read messages from multiple channels in any order. Simply calling `receive` is undesirable, because it will block on an empty channel even if there are available messages on the other. We can use `peekChan` above to implement a loop that repeatedly peeks at the different channels, but such *busywaiting* is rather inefficient.

Let us start with a solution for reading from two channels, which can have different message types. Our solution is to launch two threads that infinitely forward messages onto a third channel.

```haskell
joinChans :: Chan a -> Chan b -> IO (Chan (Either a b))
joinChans chan_a chan_b = do
  chan_c <- newChan
  _ <- forkIO $ forever $ send chan_c . Left =<< receive chan_a
  _ <- forkIO $ forever $ send chan_c . Right =<< receive chan_b
  pure chan_c
```

It is not so difficult to generalise this to any number of threads, although in that case we require that they all have the same message type.

```haskell
joinChansAny :: [Chan a] -> IO (Chan a)
joinChansAny ins = do
  outc <- newChan
  forM_ ins $ \inc ->
    forkIO $ forever $ send outc =<< receive inc
  pure outc
```

In some cases we may care about which channel a message is received from, which can be handled by associating each channel with a *tag* that is paired with each received message.

```haskell
joinChansAnyTagged :: [(tag, Chan a)] -> IO (Chan (tag, a))
joinChansAnyTagged ins = do
  outc <- newChan
  forM_ ins $ \(tag, inc) ->
    forkIO $ forever $ do
      msg <- receive inc
      send outc (tag, msg)
  pure outc
```

Broadcasting

Now let us consider the opposite of reading from multiple channels: *sending* a message to many channels. On the face of it, this is much simpler: just do a loop (say with `mapM` or `forM`) and `send` to the channels in question. This is certainly correct from a semantic perspective, but it is not very concurrent: each message is sent serially. If we are sending to a large number of channels, which may well be the case for a fine-grained concurrent system, this is inefficient.

Our solution is to construct a tree of threads. When a message is send to the root, it transmits the message to each of its children, which then transmit it on to their children, on to the leaves which send the message to the desired channels. For simplicity each node in our tree will have only two children, which is likely not particularly efficient because the tree ends up fairly tall, but it *is* maximally concurrent.

```haskell
broadcast :: [Chan a] -> IO (Chan a)
broadcast [] = error "broadcast: empty list"
broadcast [x] = pure x
broadcast cs = do
  let (as, bs) = splitAt (length cs `div` 2) cs
  as_c <- broadcast as
  bs_c <- broadcast bs
  c <- newChan
  _ <- forkIO $ forever $ do
    msg <- receive c
    send as_c msg
    send bs_c msg
  pure c
```

Now `broadcast cs` produces a channel `bc` such that whenever a message is sent to `bc`, it is sent to every channel in `cs`. One caveat one must be aware of is that the order in which these messages are sent (let alone received) is not deterministic, as it depends on how the threads are scheduled. It is however possible to construct a variant of `broadcast` that blocks until all messages have been sent, using a technique similar to the `requestResponse` implementation for `GenServer`.